



UNIVERSITÀ DELLA CALABRIA

DIPARTIMENTO DI
INGEGNERIA INFORMATICA,
MODELLISTICA, ELETTRONICA
E SISTEMISTICA

DIMES

Corso di Laurea Magistrale in Ingegneria Informatica

Corso di High Performance Computing

Progetto: Monte Carlo per Radioterapia in CUDA

Prof. Domenico Talia

Angelica Porco

Ing. Riccardo Cantini

Anno Accademico 2025/2026

Sommario

Introduzione	6
1. Il problema fisico	7
1.1. La radioterapia e il calcolo della dose	7
1.2. La rappresentazione del paziente	8
1.2.1. Dalla TAC alla griglia tridimensionale	9
1.2.2. Hounsfield Units	9
1.2.3. Validità e limiti della rappresentazione voxelizzata	10
1.3. Interazione fotone-materia	11
1.3.1. Effetto fotoelettrico	11
1.3.2. Diffusione Compton	12
1.3.3. Produzione di coppie	13
1.3.4. Processi nel contesto clinico	13
1.4. I coefficienti di attenuazione e le sezioni d'urto	14
1.4.1. Monte Carlo e database fisici	15
1.5. La legge di Beer Lambert	15
1.5.1. La legge di Beer Lambert e la distribuzione del cammino libero [4]	15
1.5.2. Voxel traversal	16
1.6. Distribuzione di Klein Nishina e metodo di Kahn	16
1.6.1. La distribuzione di Klein-Nishina	17
1.6.2. Metodo di Kahn	17
1.7. KERMA, dose e deposito locale	18
2. Il metodo Monte Carlo per il trasporto di particelle	19
2.1. Principio generale Monte Carlo	19
2.2. Il ciclo di simulazione	19
2.3. I generatori di numeri pseudo casuali	20
2.3.1. Scelta dei PRNG	21
2.4. Convergenza statistica	21
2.5. Differenza rispetto al Monte Carlo generico	22
2.6. Problema HPC	22
2.6.1. La warp divergence da stocasticità	23
2.6.2. La contesa sulle sezioni atomiche	23
3. Il phantom	24
3.1. Proprietà fisiche di phantom	24

3.2.	Struttura del phantom.....	25
3.3.	Il LINAC e il fascio da 6MV	26
3.4.	Il campionamento dello spettro	28
3.4.1.	Distribuzione spaziale dei fotoni	28
3.5.	Database NIST XCOM	29
3.6.	Energie di cutoff	29
3.7.	Tabella dei parametri	30
4.	Analisi della letteratura	30
4.1.	Codici Monte Carlo su CPU	30
4.1.1.	EGSnrc	30
4.1.2.	DOSXYZnrc	31
4.1.3.	Penelope	31
4.2.	Implementazioni su GPU	32
4.2.1.	MC-GPU	32
4.2.2.	gDPM	32
4.2.3.	GPUMCD	32
4.3.	Architetture ibride.....	33
4.3.1.	ARCHER	33
4.3.2.	MC-GPU 1.3	33
4.4.	Problemi delle soluzioni su architetture attuali	33
5.	Implementazione.....	34
5.1.	Versione CPU sequenziale	34
5.1.1.	Physics.h.....	35
5.1.2.	Random.h	37
5.1.3.	Compton.h.....	39
5.1.4.	Phantom.h.....	41
5.1.5.	Output.h.....	42
5.1.6.	Main.cpp	43
5.1.7.	BeerLambert.cpp.....	46
5.1.8.	Conclusioni	47
5.2.	Versione CPU parallela	47
5.2.1.	Main.cpp	48
5.2.2.	BeerLambert.cpp.....	50
5.3.	Versione base GPU	50
5.3.1.	Caratteristiche GPU	50

5.3.2.	Kernel CUDA.....	51
5.3.3.	Struttura dei moduli	51
5.3.4.	Gestione della memoria	52
5.3.5.	Generazione numeri casuali	52
5.3.6.	AtomicAdd su double.....	53
5.3.7.	Gestione delle particelle secondarie.....	53
5.3.8.	Misurazione del tempo	54
5.3.9.	Versione Beer Lambert.....	55
5.3.10.	Criticità e analisi implementativa	55
5.4.	Versione 2 GPU.....	56
5.4.1.	Considerazioni e profilazione della versione base	56
5.4.2.	Ottimizzazione con Work Stealing.....	56
5.4.3.	Versione Beer Lambert.....	58
5.5.	Versione 3 GPU.....	59
5.5.1.	Criticità versione 2	59
5.5.2.	Conversione float.....	60
5.5.3.	Utilizzo texture memory	61
5.5.4.	Versione Beer Lambert.....	63
5.6.	Profiling Nsight	64
5.6.1.	Analisi risultati NVPROF	64
5.6.2.	Analisi risultati Nsight	65
5.6.3.	Analisi complessiva dei risultati.....	65
6.	Test dei risultati.....	66
6.2.	Test 1 – Analisi del coefficiente di attenuazione lineare	67
6.3.	Test 2 – Validazione della legge di Beer Lambert	68
6.4.	Test 3 – Verifica delle sezioni d’urto e probabilità di interazione.....	69
6.5.	Test 4 – Validazione stocastica della deviazione compton	69
6.6.	Test 5 – Analisi del bilancio energetico globale	70
6.7.	Test 6 – Validazione della forma del PDD clinico	70
6.8.	Test 7 – Verifica dell’eterogeneità acqua-osso	71
6.9.	Test 8 – Analisi della penombra laterale e diffusione compton	72
6.10.	Test 9 – Gamma Index	72
6.11.	Test 10 – Validazione della convergenza statistica	73
7.	Risultati e metriche di prestazione	74
7.1.	Tempo di esecuzione	75

7.2.	Speedup assoluto.....	76
7.3.	Speedup relativo	76
7.4.	Throughput.....	77
7.5.	Heatmap dello Speedup assoluto	78
7.6.	Ripartizione del tempo GPU	78
7.7.	Riepilogo dei risultati	79
8.	Visualizzazione dei risultati	79
9.	Conclusioni	83
9.1.	Sviluppi futuri	83
	Indice delle figure	84
	Indice delle tabelle.....	84
	Bibliografia	84

Introduzione

La radioterapia rappresenta una delle modalità terapeutiche basate sull'utilizzo controllato di radiazioni per indurre danni alle cellule tumorali minimizzando, allo stesso tempo, l'irradiazione dei tessuti sani circostanti. L'accuratezza nella determinazione della dose assorbita è un fattore critico per la sicurezza del paziente. Uno scostamento superiore alla soglia di tolleranza raccomandata del 5% può comportare conseguenze negative sull'esito clinico del paziente.

Per questi motivi, i moderni sistemi di pianificazione del trattamento devono modellare il trasporto della radiazione e l'interazione fotone-materia all'interno dell'eterogeneità anatomica del paziente. In questo scenario, il metodo Monte Carlo è quello più utilizzato in quanto simula in modo stocastico le traiettorie e le singole interazioni delle particelle, come l'effetto fotoelettrico, la diffusione Compton e la produzione di coppie, all'interno di una griglia di voxel derivata dalle immagini TAC del paziente. Tuttavia, l'accuratezza del metodo Monte Carlo deriva dalla legge della convergenza statistica, la quale impone che l'incertezza decresca lentamente con l'aumentare del numero di storie simulate. Raggiungere una precisione clinicamente accettabile richiede un volume di particelle dell'ordine di $10^7 - 10^8$, il che comporta tempi di calcolo eccessivi se l'esecuzione avviene su architetture CPU sequenziali o parallele.

Sebbene la letteratura scientifica presenti delle soluzioni ottimizzate identificate come standard, queste risalgono al periodo 2009/2015, pertanto risultano obsolete rispetto alle moderne tecnologie GPU che presentano una diversa architettura della memoria cache e una differente gestione della warp divergence indotta dalla natura stocastica del Monte Carlo.

Il presente lavoro si propone di implementare un sistema Monte Carlo in ambiente CUDA per la simulazione del trasporto di fotoni ad alta energia, con riferimento al fascio clinico da 6MV generato da acceleratori lineari LINAC, all'interno di phantom voxelizzati omogenei ed eterogenei. Oltre alla riproduzione fisica del trasporto, vengono implementate una gestione del carico tramite algoritmi di work stealing, un'ottimizzazione della memoria, una conversione a precisione singola senza compromettere l'accuratezza fisica del modello, e l'utilizzo della texture memory. La validità fisica e l'accuratezza del sistema implementato verranno verificate tramite una serie di test comparativi con il codice internazionale di riferimento DOSXYZnrc.

1. Il problema fisico

1.1. La radioterapia e il calcolo della dose

La radioterapia oncologica è una modalità terapeutica basata sull'impiego controllato di radiazioni ionizzanti¹ per indurre danni alle cellule tumorali, preservando il più possibile i tessuti sani circostanti. Attualmente i fasci che vengono utilizzati sono prevalentemente fotoni ad alta energia generati da acceleratori lineari, i LINAC, con energie nominali comprese tra circa 1.25 MeV e 6 – 18MV.

Il processo fondamentale è il trasferimento di energia dal campo di radiazione al mezzo biologico. Fisicamente i fotoni non depositano direttamente la maggior parte della loro energia nel tessuto. La dose, infatti, viene rilasciata principalmente attraverso la produzione di elettroni secondari. In ambito clinico il meccanismo principale è quello dell'effetto Compton in cui un fotone incidente:

- Interagisce con un elettrone quasi libero;
- Trasferisce parte della propria energia cinetica;
- Genera un elettrone secondario ad alta energia e un fotone diffuso.

Gli elettroni secondari prodotti secondo questo meccanismo percorrono delle distanze dell'ordine dei millimetri o centimetri, a seconda dell'energia e della densità del mezzo, depositando energia attraverso ionizzazioni ed eccitazioni lungo la loro traccia. Questo processo determina la distribuzione spaziale della dose assorbita. Dal punto di vista oncologico queste ionizzazioni comportano la formazione di radicali liberi² ma anche un danno diretto e indiretto al DNA tramite le rotture a singolo e doppio filamento.

L'obiettivo del trattamento è il superamento della capacità di riparazione cellulare, con conseguente perdita della capacità riproduttiva delle cellule tumorali. L'efficacia della radioterapia è formalizzata attraverso due grandezze probabilistiche:

- TCP o Tumor Control Probability, che rappresenta la probabilità di distruggere tutte le cellule che fanno crescere il tumore;
- NTCP o Normal Tissue Complication Probability, che rappresenta la possibilità di indurre complicanze nei tessuti sani.

Il trattamento ottimale consiste nel massimizzare la TCP mantenendo la NTCP entro i limiti accettabili che definiscono la finestra terapeutica.

Questo equilibrio è diventato sempre più critico con l'introduzione di tecniche avanzate come l'IMRT, Intensity Modulated Radiation Therapy, o la tecnica adattativa VMAT, Volumetric Modulated Arc Therapy. Queste metodologie consentono una distribuzione di dose altamente conformata alla massa tumorale ma introducono anche gradienti di dose estremamente ripidi con elevata complessità spaziale e dosimetrica [1]. Con queste metodologie anche piccoli errori possono avere conseguenze cliniche rilevanti. Ad esempio, un errore geometrico o dosimetrico può causare sottodosaggio del PTV o Planning Target Volume, oppure trasferire una dose elevata ad un organo a rischio adiacente al tumore. Di conseguenza, la precisione del calcolo della dose è un fattore determinante per la sicurezza del paziente e per la riuscita della terapia.

¹Onde elettriche ad alta energia in grado di strappare elettroni dagli atomi.

² Molecole instabili particolarmente reattive.

Il vincolo sull'accuratezza dosimetrica deriva direttamente dalla relazione non lineare tra dose e risposta biologica. In un lavoro del 1984 [2] Anders Brahme ha analizzato la relazione tra la curva dose/risposta e la terapia evidenziando come:

- Nella regione clinicamente rilevante queste curve presentano una pendenza elevata;
- Una variazione relativamente piccola della dose si traduce in una variazione significativa dell'esito clinico.

In particolare, si evidenzia come una deviazione del 5% nella dose possa comportare variazioni del 10%-20% nella TCP o nella NTCP a seconda del tessuto considerato. Questo effetto è particolarmente critico nei trattamenti dove il sistema opera vicino alla soglia di controllo tumorale.

L'accuratezza nella prescrizione, nel calcolo e nell'erogazione della dose è un requisito fondamentale per garantire la qualità e la comparabilità dei trattamenti [1]. In questo contesto l'incertezza complessiva raccomandata è dell'ordine del 3.5%. Nella pratica clinica, considerando tutte le sorgenti di errore, come il calcolo della dose e il posizionamento del fascio, si accetta un limite globale di circa 5%. La precisione geometrica offerta dalle tecniche moderne è significativa solo se accompagnata da un'accuratezza dosimetrica equivalente. In caso contrario, il beneficio tecnologico viene annullato da errori sistematici nel calcolo della dose.

Per rispettare questi requisiti i sistemi di pianificazione del trattamento o TPS moderni devono essere in grado di modellare accuratamente :

- Il trasporto di fotoni;
- La produzione e il trasporto degli elettroni secondari;
- L'eterogeneità dei tessuti, ad esempio polmone, osso e cavità aeree.

Questi aspetti sono particolarmente critici nelle regioni a bassa densità come il polmone, nelle aree ad alto gradiente di dose e in prossimità di interfacce tra materiali diversi.

Il lavoro di J. Van Dyk [3] ha definito i protocolli di commissioning³ e quality assurance⁴ dei TPS introducendo criteri pratici per verificare la loro accuratezza rispetto alla realtà fisica. Tra questi si ha la validazione tramite misure sperimentali in fantocci, detti phantom, la verifica in condizioni di eterogeneità e l'utilizzo di metriche quantitative come il Gamma Index⁵ per confrontare le distribuzioni di dose calcolate e misurate. Questi protocolli rappresentano ancora oggi la base qualitativa in radioterapia.

1.2. La rappresentazione del paziente

Nel contesto della radioterapia moderna, il calcolo della distribuzione di dose richiede la costruzione di un modello numerico del paziente che sia fisicamente significativo e computazionalmente trattabile. La transizione dall'anatomia reale, continua e complessa, a una rappresentazione discreta costituisce un passaggio fondamentale per l'applicazione dei metodi Monte Carlo, i quali simulano in modo stocastico il trasporto della radiazione attraverso la materia. Questa rappresentazione prende la forma di un phantom voxelizzato, ovvero *“una discretizzazione tridimensionale del volume anatomico in una*

³ Processo di verifica e calibrazione degli acceleratori lineari LINAC e dei sistemi TPS prima dell'uso.

⁴ Insieme di procedure mirate a garantire l'accuratezza del trattamento e ottimizzare la dose sul tumore.

⁵ Metrica di controllo qualità che confronta la distribuzione di dose calcolata dal TPS con quella misurata.

griglia regolare di elementi volumetrici discreti, all'interno dei quali le proprietà fisiche del mezzo sono considerate uniformi".

1.2.1. Dalla TAC alla griglia tridimensionale

La costruzione del phantom voxelizzato parte dalla Tomografia Assiale Computerizzata o TAC che fornisce una descrizione volumetrica dell'anatomia del paziente sotto forma di immagini bidimensionali. Ogni immagine è costituita da una matrice di pixel la cui estensione tridimensionale lungo l'asse longitudinale definisce i voxel. Il volume del paziente viene rappresentato, pertanto, come una matrice:

$$N_x \times N_y \times N_z$$

in cui a ciascun elemento sono associati una posizione spaziale e un valore numerico derivato dalla TAC. Le dimensioni tipiche dei voxel in ambito clinico sono dell'ordine di:

- 2-3 mm nel piano trasversale ;
- 2-5 mm lungo l'asse cranio-caudale.

Questa discretizzazione rappresenta un'approssimazione della geometria reale del paziente. Tale approssimazione è giustificata dal fatto che le grandezze fisiche rilevanti per il trasporto della radiazione variano su scale spaziali compatibili o superiori. Il voxel rappresenta, pertanto, un volume elementare entro cui le proprietà del mezzo possono essere considerate costanti senza una perdita significativa di accuratezza.

Dal punto di vista implementativo, il phantom voxelizzato è memorizzato come una struttura dati tridimensionale in cui ogni voxel è identificato da indici e associato a parametri fisici. Questa struttura è particolarmente adatta al trasporto Monte Carlo poiché consente di determinare in modo efficiente la posizione della particella, il materiale attraversato e le probabilità locali di interazione.

1.2.2. Hounsfield Units

Il dato primario fornito dalla TAC è rappresentato dalle Hounsfield Units (HU) che costituiscono una misura relativa del coefficiente di attenuazione lineare dei tessuti rispetto all'acqua. Sono definite come:

$$HU = 1000 * \frac{\mu - \mu_{acqua}}{\mu_{acqua}}$$

Dove μ rappresenta il coefficiente di attenuazione lineare del tessuto per i raggi X utilizzati nella scansione.

Le HU rappresentano le proprietà di attenuazione a energie tipicamente dell'ordine di decine di keV⁶, mentre il problema dosimetrico in radioterapia riguarda fotoni con energie molto più elevate nell'ordine dei MeV⁷. Ne deriva il fatto che le HU non possono essere utilizzate direttamente nei codici Monte Carlo ma devono essere convertite in grandezze fisicamente rilevanti per il trasporto della radiazione.

Il passaggio dalla descrizione radiologica a quella fisica avviene attraverso una procedura di calibrazione che associa a ciascun valore di HU una densità di massa, una densità elettronica e una

⁶ Indica l'energia di particelle o raggi X e rappresenta un kiloelettronvolt ($1keV = 1.602 * 10^{-16}J$).

⁷ Misura l'energia delle particelle subatomiche ed è pari a $10^6eV = 1.602 * 10^{13}J$.

classe di materiale biologico. La relazione risultante, implementata come una funzione bi-lineare o multilineare, è nota come CT-to density calibration curve.

In seguito a questa relazione il range di HU viene suddiviso in intervalli ai quali vengono assegnati materiali come aria, polmone, tessuto molle, osso, I materiali con densità simili possono rappresentare composizioni elementari differenti con conseguenze sulle sezioni d'urto e quindi sul trasporto delle particelle. In questo modo, ogni voxel è caratterizzato da:

- Una densità ρ ;
- Un indice di materiale;
- Una composizione elementare associata.

Per rendere possibile la simulazione Monte Carlo è necessario associare a ciascun materiale delle proprietà fisiche tra cui:

- Frazioni in peso degli elementi chimici;
- Densità elettronica;
- Numero atomico effettivo.

Queste informazioni derivano da database standardizzati e permettono di calcolare le sezioni d'urto per fotoni ed elettroni, la capacità di arresto e le distribuzioni angolari e spettrali delle particelle secondarie.

La densità elettronica, in particolare, rappresenta un fattore determinante nel campo energetico della radioterapia in quanto determina la possibilità dell'effetto Compton e il trasporto degli elettroni secondari. Il numero atomico effettivo diventa invece rilevante per processi come l'effetto fotoelettrico e la produzione di coppie, soprattutto ai limiti inferiori e superiori dello spettro energetico.

In questo modo, il phantom voxelizzato non è solamente una descrizione geometrica, ma anche una vera e propria mappa tridimensionale delle proprietà fisiche del mezzo.

1.2.3. Validità e limiti della rappresentazione voxelizzata

Nel contesto del metodo Monte Carlo, il phantom voxelizzato costituisce il dominio entro cui avviene il trasporto delle particelle. Durante la simulazione ogni particella viene tracciata lungo la sua traiettoria e la posizione corrente determina il voxel attraversato, mentre le proprietà del voxel definiscono le probabilità di interazione.

All'interno di ciascun voxel il mezzo è omogeneo, il che rende possibile il campionamento delle interazioni senza dover risolvere variazioni continue delle proprietà del mezzo. L'energia che viene depositata viene accumulata voxel per voxel producendo una distribuzione tridimensionale della dose che può essere direttamente confrontata con il volume anatomico del paziente.

La rappresentazione voxelizzata è considerata adeguata al trasporto Monte Carlo poiché la scala spaziale della discretizzazione è compatibile con le lunghezze caratteristiche dei processi fisici coinvolti. Il cammino libero medio dei fotoni e il range degli elettroni sono tipicamente maggiori o comparabili alla dimensione del voxel, il che rende valida l'approssimazione di omogeneità locale. Inoltre, questa rappresentazione permette di descrivere in modo realistico le eterogeneità anatomiche, incluse le interfacce tra tessuti a diversa densità, dove fenomeni come la variazione del range elettronico e la retrodiffusione diventano rilevanti. Infine, la struttura a griglia regolare consente una navigazione computazionalmente efficiente in cui il passaggio da un voxel all'altro richiede operazioni geometriche semplici, ottimizzando i tempi di simulazione senza compromettere l'accuratezza.

I limiti principali, invece, sono legati alla risoluzione della TAC, alla discretizzazione delle proprietà dei materiali e alla possibile perdita di dettaglio nelle interfacce molto sottili. Tuttavia, entro questi limiti, il phantom voxelizzato rappresenta un compromesso ottimale tra fedeltà fisica e sostenibilità computazionale. Proprio per questo rappresenta lo standard per la dosimetria avanzata.

Attraverso la conversione della TAC in una mappa tridimensionale di densità e composizione, il modello consente di applicare il metodo Monte Carlo al problema del calcolo della dose. L'accuratezza complessiva di questa fase dipende da:

- Qualità e calibrazione delle immagini TAC;
- Correttezza della conversione da HU a densità/materiale;
- Risoluzione spaziale della discretizzazione.

Una volta soddisfatte queste condizioni, l'incertezza residua nel calcolo della dose è dominata dalla statistica della simulazione rendendo possibile il rispetto dei requisiti clinici di accuratezza.

1.3. Interazione fotone-materia

Nel range energetico rilevante per la radioterapia esterna, l'interazione dei fotoni con la materia è dominata da tre meccanismi fondamentali [4]:

- Effetto fotoelettrico;
- Diffusione Compton;
- Produzione di coppie.

Questi processi costituiscono la base del trasporto della radiazione e, di conseguenza, della deposizione di dose nei tessuti biologici [5]. Ogni fotone incidente può subire uno solo di questi processi in un dato evento di interazione. La competizione tra i tre meccanismi è determinata dalle rispettive sezioni d'urto, che dipendono dall'energia del fotone e dalle proprietà atomiche del mezzo attraversato. Nel metodo Monte Carlo questa selezione avviene attraverso un processo stocastico in cui l'interazione viene scelta campionando una distribuzione costruita a partire dalle probabilità relative dei singoli contributi.

1.3.1. Effetto fotoelettrico

L'effetto fotoelettrico rappresenta il caso più semplice in quanto, il fotone incidente viene completamente assorbito da un elettrone tipicamente appartenente agli strati più interni dell'atomo. L'energia del fotone viene trasferita quasi interamente all'elettrone, che viene espulso con energia cinetica pari alla differenza tra l'energia incidente e l'energia di legame:

$$E_k = h\nu - E_b$$

Il processo è quindi caratterizzato dall'assorbimento completo del fotone e dalla generazione di un elettrone secondario energetico. L'atomo residuo, lasciato in uno stato eccitato si rilassa attraverso una cascata di transizioni elettroniche, accompagnata dall'emissione di raggi X caratteristici. Nel calcolo della dose, questi contributi secondari vengono trattati come un deposito locale di energia soprattutto quando le loro lunghezze di attenuazione sono inferiori alla dimensione del voxel.

La sezione d'urto fotoelettrica presenta una forte dipendenza dal numero atomico e una rapida decrescita con l'energia del fotone. Si può descrivere come proporzionale a:

$$\frac{Z^n}{E^3}$$

Con $4 \leq n \leq 5$.

Questa dipendenza implica che il processo sia estremamente sensibile alla composizione del materiale risultando dominante a basse energie, mentre diventa rapidamente trascurabile al crescere dell'energia.

Nel contesto della radioterapia con fasci a megavoltaggio, il contributo dell'effetto fotoelettrico nei tessuti molli è generalmente secondario. Tuttavia, non è completamente trascurabile in presenza di materiali ad alto numero atomico, come il tessuto osseo, o in condizioni in cui lo spettro energetico contiene componenti a bassa energia.

Dal punto di vista del trasporto Monte Carlo, la gestione del processo è diretta. Una volta selezionato l'evento fotoelettrico, la storia del fotone viene terminata e viene generato un elettrone con energia E_k . La direzione di emissione per energie elevate si calcola utilizzando distribuzioni angolari relativistiche mentre in molte applicazioni cliniche tramite approssimazioni semplificate. La successiva cascata di rilassamento atomico può essere simulata esplicitamente oppure incorporata in un deposito locale di energia.

1.3.2. Diffusione Compton

La diffusione Compton rappresenta il meccanismo predominante nel range energetico della radioterapia clinica. In questo processo, il fotone interagisce con un elettrone debolmente legato⁸, che può essere trattato come libero rispetto all'energia incidente. L'interazione comporta un trasferimento parziale di energia dal fotone all'elettrone, accompagnato da una deviazione della direzione del fotone stesso. Ne risultano due particelle secondarie, che sono:

- Un fotone diffuso, con energia ridotta e direzione modificata;
- Un elettrone che trasporta l'energia ceduta.

La relazione tra energia incidente, energia diffusa e angolo di scattering è determinata dalla conservazione relativistica di energia e quantità di moto, portando ad avere:

$$E' = \frac{E}{1 + \frac{E}{m_e c^2} (1 - \cos \theta)} \quad ^9$$

La descrizione del processo è rappresentata dalla sezione d'urto differenziale di Klein-Nishina, che descrive la probabilità di scattering in funzione dell'angolo e dell'energia e mostra come, all'aumentare dell'energia, la distribuzione angolare diventi sempre più concentrata in direzione forward.

A differenza dell'effetto fotoelettrico la probabilità di interazione Compton dipende in modo relativamente debole dal numero atomico, essendo proporzionale alla densità elettronica del mezzo. Questo aspetto implica che, materiali con composizione chimica diversa ma densità elettronica simile, presentano dei comportamenti di dosimetria simili.

⁸ Elettrone che orbita intorno a un nucleo atomico con una bassa energia di legame. Questo indica che è necessaria poca energia per liberarlo dal suo stato.

⁹ Formula che descrive la cinematica dell'urto, quindi quanta energia resta al fotone dopo che è stato deviato di un angolo θ .

La diffusione Compton è il contributo dominante nei tessuti biologici ed è il principale responsabile della produzione di elettroni secondari e, di conseguenza, del deposito di dose nei trattamenti con fasci di fotoni. Il campionamento del processo Compton prevede il campionamento dell'angolo di scattering secondo la distribuzione di Klein-Nishina. Per questo motivo servono delle tecniche numeriche come i metodi di accettazione/rifiuto e degli algoritmi dedicati. Una volta determinato l'angolo, l'energia del fotone diffuso e quella dell'elettrone sono univocamente determinate. Entrambe le particelle vengono quindi tracciate nel mezzo, contribuendo alla distribuzione finale della dose.

1.3.3. Produzione di coppie

La produzione di coppie diventa possibile quando l'energia del fotone supera la soglia di 1.022 MeV , che è pari al doppio dell'energia di massa a riposo dell'elettrone. In questo processo, il fotone interagisce con il campo coulombiano di un nucleo e viene convertito in una coppia elettrone-positrone¹⁰. L'energia del fotone viene quindi utilizzata per creare la massa delle due particelle e per fornire loro energia cinetica:

$$E = 2m_e c^2 + T_{e^-} + T_{e^+}^{11}$$

La presenza del nucleo è necessaria per garantire la conservazione della quantità di moto. Il positrone prodotto dopo aver rallentato nel mezzo, si distrugge generando due fotoni emessi in direzioni opposte.

La sezione d'urto per la produzione di coppie cresce con l'energia del fotone ed è proporzionale a Z^2 rendendo il processo più probabile in materiali ad alto numero atomico. Nel contesto clinico, il contributo della produzione di coppie diventa significativo per energie superiori a 10 MeV . In questi casi, il processo contribuisce non solo alla deposizione di dose ma anche alla generazione di radiazione secondaria, inclusi i fotoni di annichilazione.

Nel formalismo Monte Carlo, una volta selezionato il processo, si procede al campionamento della distribuzione di energia tra elettrone e positrone, che è mediamente simmetrica ma con fluttuazioni significative. Le direzioni di emissione sono concentrate lungo la direzione incidente del fotone, soprattutto ad alte energie. Entrambe le particelle vengono quindi tracciate fino al loro arresto.

1.3.4. Processi nel contesto clinico

Nel contesto della radioterapia con fasci fotonici di energia tipica di $6 - 10\text{ MV}$, la diffusione Compton rappresenta il meccanismo dominante nella maggior parte dei tessuti biologici. Tuttavia, la presenza di eterogeneità anatomiche introduce variazioni locali nelle probabilità di interazione. In particolare, nelle interfacce tra tessuti con diverso numero atomico o densità, come osso e tessuto molle, anche processi secondari come l'effetto fotoelettrico possono contribuire in modo non trascurabile.

Allo stesso modo, nei trattamenti ad alta energia, la produzione di coppie diventa progressivamente più rilevante e deve essere modellata con accuratezza per garantire la correttezza del calcolo dosimetrico. Nonostante la dominanza di un singolo meccanismo in specifici intervalli energetici, una descrizione fisica completa richiede la modellizzazione accurata di tutti e tre i processi, in accordo con i requisiti di precisione imposti dalla pratica clinica.

¹⁰ Chiamato anche antielettrone, rappresenta l'antiparticella dell'elettrone.

¹¹ Energia totale di un sistema composto da una coppia elettrone-positrone.

1.4. I coefficienti di attenuazione e le sezioni d'urto

La descrizione del trasporto dei fotoni in un mezzo materiale richiede il passaggio da una formulazione microscopica, basata sulle interazioni elementari fotone-materia, a una rappresentazione macroscopica che consente di trattare l'evoluzione di un fascio in un sistema esteso. Questo è possibile grazie al coefficiente di attenuazione lineare μ , che rappresenta la probabilità che un fotone subisca un'interazione nel mezzo. Si può considerare come una densità di probabilità, ovvero se un fotone percorre un tratto infinitesimo ds , la probabilità che interagisca in questo intervallo è pari a:

$$\mu ds$$

Il coefficiente di attenuazione costituisce il parametro fondamentale che collega la struttura microscopica della materia alla descrizione statistica del trasporto. Se si considera un mezzo costituito da atomi con densità numerica n , il coefficiente di attenuazione lineare può essere espresso come:

$$\mu = n\sigma_{tot}$$

Dove σ_{tot} è la sezione d'urto totale, ovvero la somma dei contributi dei diversi processi di interazione. Questa relazione evidenzia il legame diretto tra la scala microscopica e quella macroscopica. Poiché la densità numerica dipende dalla densità di massa e dalla composizione del materiale si ha:

$$\frac{\mu}{\rho}$$

Che rappresenta una proprietà intrinseca del materiale, indipendente dal suo stato fisico. Il coefficiente di attenuazione totale può essere espresso come somma dei contributi associati ai tre processi fondamentali di interazione fotone-materia:

$$\mu = \tau + \sigma + k$$

Che rappresentano rispettivamente il contributo fotoelettrico, effetto Compton e la produzione di coppie. Si ottiene una relazione in termini di attenuazione pari a:

$$\frac{\mu}{\rho} = \left(\frac{\tau}{\rho}\right) + \left(\frac{\sigma}{\rho}\right) + \left(\frac{k}{\rho}\right)$$

Questa consente di analizzare il contributo relativo dei diversi meccanismi in funzione dell'energia e del materiale. Dal punto di vista computazionale permette di costruire le probabilità relative dei singoli processi utilizzati nel metodo Monte Carlo per la selezione stocastica dell'interazione.

L'andamento del coefficiente di attenuazione in funzione dell'energia riflette direttamente la competizione tra i tre processi fondamentali e la loro diversa dipendenza dalle proprietà atomiche del mezzo. A basse energie, il contributo dominante è quello fotoelettrico, caratterizzato da una forte dipendenza dal numero atomico e da una rapida decrescita con l'energia. In questa regione, il coefficiente di attenuazione varia significativamente tra materiali diversi e anche piccole variazioni di composizione possono produrre differenze rilevanti nel comportamento del fascio.

All'aumentare dell'energia, il contributo fotoelettrico diventa trascurabile e viene sostituito dalla diffusione Compton. In questa regione, la dipendenza dal numero atomico si attenua drasticamente, poiché il processo è governato principalmente dalla densità elettronica. Questo implica che, materiali biologici con composizione differente ma densità simile presentano coefficienti di attenuazione comparabili, una proprietà che ha importanti implicazioni per la dosimetria in radioterapia. A energie

ancora più elevate, sopra la soglia di produzione delle coppie, si ha un nuovo contributo che cresce con l'energia e con il quadrato del numero atomico.

1.4.1. Monte Carlo e database fisici

I coefficienti di attenuazione e le sezioni d'urto non vengono calcolati direttamente ma derivano da database fisici che integrano risultati sperimentali e calcoli teorici accurati [6]. Le tabelle dei coefficienti di attenuazione costituiscono il riferimento standard per applicazioni dosimetriche. Forniscono i valori di μ/ρ e μ_{en}/ρ per un ampio intervallo di energie e per diversi elementi e materiali composti, consentendo di caratterizzare in modo accurato il comportamento della radiazione nei tessuti biologici.

In ambito Monte Carlo, tuttavia, è necessario conoscere anche le informazioni differenziali, come la distribuzione angolare e spettrale dei prodotti di interazione. Per questo motivo vengono utilizzate delle librerie fisiche che includono le sezioni d'urto differenziali e dati per processi secondari permettendo una simulazione completa delle interazioni e dei fenomeni atomici.

Un aspetto fondamentale è rappresentato anche dal valore di μ che non è costante nello spazio ma dipende dalle proprietà locali del mezzo. Nei phantom voxelizzati utilizzati nella radioterapia, il coefficiente di attenuazione viene aggiornato dinamicamente in funzione della densità e della composizione del voxel attraversato. In questo modo, la geometria anatomica del paziente viene trasformata in una mappa tridimensionale di probabilità di interazione che guida il trasporto stocastico delle particelle.

1.5. La legge di Beer Lambert

Il percorso di un fotone non è deterministico ma è costituito da una successione di eventi casuali, come tratti di volo intervallati da interazioni discrete. Il primo problema da risolvere è quindi quello di stabilire quanto spazio percorre il fotone prima del prossimo urto.

1.5.1. La legge di Beer Lambert e la distribuzione del cammino libero [4]

Si considera un fascio di fotoni che attraversa un materiale omogeneo. Ogni fotone ha una certa probabilità di interagire descritta dal coefficiente di attenuazione μ . Se $N(x)$ è il numero di fotoni che hanno attraversato una distanza x senza interagire, la variazione infinitesima lungo un tratto dx è proporzionale a quanti fotoni sono ancora presenti, ovvero:

$$dN = -\mu N dx$$

Più fotoni ci sono, più interazioni avvengono. Integrando questa equazione si ottiene la legge esponenziale:

$$N(x) = N_0 e^{-\mu x}$$

Dove $e^{-\mu x}$ rappresenta la probabilità che un singolo fotone non abbia ancora interagito dopo aver percorso una distanza x . La posizione dell'interazione non è pertanto fissa ma distribuita secondo una legge statistica ben precisa.

Nel contesto Monte Carlo non si lavora con distribuzioni continue, ma con eventi discreti simulati uno alla volta [7]. Per ogni fotone bisogna quindi generare una distanza di volo s coerente con la distribuzione esponenziale. Questo si ottiene utilizzando un numero casuale uniforme $0 \leq \xi \leq 1$. Si arriva tramite inversione della funzione di distribuzione cumulativa alla relazione:

$$s = -\frac{\ln(\xi)}{\mu}$$

Questo indica che l'algoritmo estrae un numero casuale e lo trasforma in una distanza fisica. Se ξ è grande il logaritmo è piccolo per cui il fotone interagisce subito, al contrario se è piccolo il fotone percorre una distanza maggiore prima di urtare. Questo processo si ripete per un numero elevato di particelle così da ottenere una distribuzione delle interazioni ricostruita numericamente e coincidente con quella prevista dalla legge fisica.

1.5.2. Voxel traversal

La derivazione della legge di Beer Lambert si basa sul fatto che il mezzo sia omogeneo, ovvero il coefficiente μ sia costante lungo il percorso. Questa condizione è però raramente soddisfatta nel corpo umano, che risulta caratterizzato da forti discontinuità: polmone, osso, tessuti molli, Di conseguenza anche μ varia nello spazio.

Per gestire questa complessità Monte Carlo sfrutta la rappresentazione voxelizzata del paziente. Si applica la legge di Beer-Lambert localmente, all'interno di ciascun voxel, dove il materiale può essere considerato uniforme.

Il fotone si trova in un voxel con un dato valore di μ e si calcola una distanza teorica utilizzando questo valore. Si verifica se questa distanza rimane interamente all'interno del voxel oppure se il fotone raggiunge prima un confine. Se il percorso è interamente contenuto nel voxel, l'interazione avviene alla distanza calcolata. Se invece il fotone incontra il bordo del voxel prima di completare il percorso, il calcolo viene interrotto esattamente al confine. A questo punto il fotone entra in un nuovo voxel con proprietà diverse e il processo continua analizzando il nuovo valore di μ . Questo meccanismo rappresenta il voxel traversal, e permette di combinare la semplicità della legge esponenziale con la complessità anatomica reale. Il percorso della particella rappresenta, pertanto, una sequenza di segmenti, ciascuno trattato come se il mezzo fosse localmente omogeneo.

Il campionamento del cammino libero medio in Monte Carlo è il punto in cui le proprietà fisiche del materiale vengono tradotte in una distribuzione spaziale di eventi reali. Da questo meccanismo dipende direttamente:

- La posizione delle interazioni;
- La generazione delle particelle secondarie;
- La distribuzione finale della dose.

L'accuratezza del modello dipende quindi sia dalla correttezza dei valori di μ sia dalla fedeltà con cui viene implementato il campionamento stocastico.

La legge di Beer-Lambert implica che la distanza tra interazioni segue una distribuzione esponenziale, che può essere campionata in modo semplice ed efficiente attraverso la trasformazione di un numero casuale uniforme. Nel contesto reale della radioterapia, l'ipotesi di omogeneità viene gestita suddividendo il paziente in voxel e applicando localmente il modello.

1.6. Distribuzione di Klein Nishina e metodo di Kahn

Una volta determinato dove avviene un'interazione si deve stabilire come avviene l'urto, cioè in quale direzione viene deviato il fotone e quante energia trasferisce all'elettrone. Tuttavia, non tutte le direzioni di scattering sono ugualmente probabili.

1.6.1. La distribuzione di Klein-Nishina

Quando un fotone interagisce tramite effetto Compton viene deviato di un certo angolo θ rispetto alla direzione iniziale. La probabilità che lo scattering avvenga a un certo angolo non è uniforme ma è descritta dalla distribuzione di Klein-Nishina, che rappresenta la sezione d'urto differenziale del processo. Questa distribuzione afferma che:

- A basse energie, il fotone può essere diffuso in quasi tutte le direzioni con probabilità simile;
- Ad alte energie lo scattering è fortemente in avanti, ovvero il fotone tende a continuare quasi nella stessa direzione.

Questa asimmetria influenza la direzione del trasporto dell'energia del paziente. Inoltre, l'angolo di scattering determina univocamente anche l'energia del fotone diffuso: più grande è l'angolo e maggiore è l'energia trasferita all'elettrone.

Nel metodo Monte Carlo si deve essere in grado di estrarre numeri casuali che seguano esattamente quella distribuzione. Per alcune distribuzioni semplici come quella esponenziale del cammino libero medio, questo si fa invertendo analiticamente la funzione, ma nel caso più complesso come quello di Klein-Nishina, questo non basta. Questo perché la distribuzione ha una forma matematica complessa, non è invertibile in modo semplice e non permette di ottenere direttamente l'angolo θ a partire da un numero casuale uniforme.

1.6.2. Metodo di Kahn

Il metodo più utilizzato per risolvere questo problema è il rejection sampling o campionamento per accettazione/rifiuto, nella versione di Kahn. Con questo meccanismo, invece di cercare di campionare direttamente la distribuzione complessa di Klein-Nishina:

- Si genera un candidato angolare secondo una distribuzione più semplice;
- Si decide se accettarlo oppure no, in base alla distribuzione reale.

Si sceglie una distribuzione da cui è semplice estrarre angoli, come ad esempio una distribuzione uniforme. Questa distribuzione deve essere sempre maggiore o uguale alla distribuzione reale. A questo punto:

- Si estrae un angolo candidato;
- Si calcola quanto è probabile secondo la distribuzione reale;
- Si genera un secondo numero casuale per decidere se accettarlo.

Se il numero casuale cade sotto la curva della distribuzione reale, l'angolo viene accettato altrimenti viene scartato e si ripete il processo.

Questo meccanismo garantisce che, dopo molte iterazioni, gli angoli accettati seguano esattamente la distribuzione di Klein-Nishina. Questo metodo non cerca di invertire la distribuzione ma la ricostruisce indirettamente attraverso un filtro probabilistico in cui la distribuzione semplice genera proposte mentre la distribuzione reale decide quali sono fisicamente corrette.

Una volta accettato un angolo di scattering θ allora l'energia del fotone diffuso si calcola dalle leggi di conservazione, l'energia ceduta all'elettrone è data dalla differenza rispetto all'energia iniziale mentre la direzione dell'elettrone deriva dalla stessa sua cinematica. Con questo meccanismo il metodo Monte Carlo traccia entrambe le particelle contribuendo alla distribuzione finale della dose. Il campionamento

corretto della distribuzione di Klein-Nishina controlla la direzione dei fotoni secondari, determina come l'energia si propaga nello spazio e influenza direttamente la forma del profilo di dose.

Questa combinazione consente di riprodurre fedelmente la fisica dell'interazione e costituisce uno degli elementi fondamentali che rendono il metodo Monte Carlo uno strumento accurato nella distribuzione della dose in radioterapia.

1.7. KERMA, dose e deposito locale

L'energia trasportata dal fotone non viene depositata direttamente nel punto di interazione attraverso particelle secondarie cariche. Quando un fotone interagisce con la materia, la sua energia non viene depositata, infatti, direttamente nel mezzo, ma viene prima trasferita a particelle cariche. Questo trasferimento iniziale è definito come KERMA o Kinetic Energy Released per unit MAAss e rappresenta l'energia cinetica trasferita dai fotoni alle particelle cariche per unità di massa. La dose assorbita, invece, è definita come l'energia effettivamente depositata nel mezzo per unità di massa. Il KERMA descrive quanta energia viene messa in moto sotto forma di elettroni, mentre la dose descrive quanta energia viene realmente dissipata localmente.

Questa distinzione deriva dal fatto che gli elettroni secondari prodotti nelle interazioni, soprattutto Compton, non si fermano nel punto in cui sono stati generati ma percorrono una certa distanza depositando energia lungo il loro cammino. Questo implica due effetti importanti:

- Parte dell'energia trasferita in un punto può essere depositata altrove;
- Parte dell'energia depositata in un punto può provenire da alterazioni avvenute in altre regioni.

Di conseguenza KERMA e dose coincidono solo in casi particolari. La condizione in cui ciò avviene è chiamata equilibrio delle particelle cariche o Charged Particle Equilibrium CPE. Questa condizione si verifica quando l'energia trasportata dagli elettroni che entrano in un volume è uguale a quella trasportata dagli elettroni che ne escono.

In radioterapia il CPE è generalmente soddisfatto nelle regioni sufficientemente lontane dalle interfacce, nei campi uniformi e per energie non troppo basse. Tuttavia, può essere violato vicino ai bordi del campo, in presenza di forti disomogeneità e in prossimità della superficie. La dose è determinata interamente dal trasporto degli elettroni secondari. I fotoni agiscono come generatori di elettroni ma sono quest'ultimi a depositare energia nel mezzo attraverso collisioni. Per calcolare la dose in modo esatto bisognerebbe simulare non solo i fotoni ma anche il trasporto dettagliato di tutti gli elettroni secondari aumentando però il costo computazionale.

Per rendere il problema più gestibile si introduce l'approssimazione di deposito locale che consiste nel supporre che l'energia trasferita agli elettroni venga depositata nello stesso punto in cui è stata generata, pertanto si assume implicitamente che:

$$Dose \approx KERMA$$

Questa approssimazione è valida quando il range degli elettroni è piccolo rispetto alla scala spaziale del problema, ovvero quando i voxel sono più grandi del cammino degli elettroni o in regioni dove il mezzo è relativamente omogeneo. Diventa invece meno accurata vicino alle interfacce tra materiali diversi, in presenza di gradienti di dose elevati o per energie elettroniche elevate.

Questa semplificazione, in Monte Carlo, implica che ogni interazione fotonica contribuisce direttamente alla dose locale, l'energia ceduta viene accumulata nel voxel corrente e non viene

simulato il trasporto successivo degli elettroni. Il fotone viene trasportato in modo dettagliato mentre il deposito energetico viene trattato in modo locale.

2. Il metodo Monte Carlo per il trasporto di particelle

Il metodo Monte Carlo oggi è ampiamente utilizzato nella fisica medica per il calcolo della dose in radioterapia. Tuttavia, non nasce come strumento di analisi medica ma per il trasporto di particelle in sistemi complessi. Il suo sviluppo è legato allo studio del comportamento dei neutroni nei materiali durante il progetto Manhattan¹². Il metodo Monte Carlo, invece di cercare una soluzione analitica globale simula direttamente il comportamento di un gran numero di particelle individuali, lasciando che la statistica ricostruisca il comportamento medio del sistema. Usa un uso sistematico di numeri casuali per risolvere problemi fisici complessi.

2.1. Principio generale Monte Carlo

Si considera una particella alla volta e si simula il suo percorso come una sequenza di eventi tra cui: un tratto di volo, un'interazione e la generazione di particelle secondarie. Ogni passaggio non è determinato in modo fisso, ma viene estratto da una distribuzione di probabilità che rappresenta la fisica del processo. Ripetendo questa simulazione per un numero elevato di particelle, le grandezze di interesse come la distribuzione di energia depositata, emergono come medie statistiche. Proprio per questo motivo si ha che il metodo Monte Carlo è stato sviluppato per il trasporto di particelle, compito primario della radioterapia, non è un metodo adattato successivamente. Le stesse tecniche utilizzate per il trasporto dei neutroni nell'ambito del progetto Manhattan sono state adattate al trasporto di fotoni e di elettroni nei tessuti biologici.

Il Monte Carlo per il trasporto di particelle è un algoritmo iterativo in cui ogni particella viene inizializzata con una certa energia, posizione e direzione, e il suo percorso viene simulato passo dopo passo. La distanza fino alla prossima interazione viene determinata in modo stocastico a partire dai coefficienti di attenuazione¹³. Il tipo di interazione viene selezionato in base alle probabilità relative dei processi fisici. Le caratteristiche dell'evento vengono campionate dalle distribuzioni appropriate e l'energia trasferita al mezzo viene registrata contribuendo alla grandezza finale che è la dose. Nel contesto della radioterapia, la legge di attenuazione esponenziale, la distribuzione di Klein-Nishina e il legame KERMA-dose rappresentano i singoli passi di queste iterazioni.

L'intero processo è reso possibile dai generatori di numeri pseudo-casuali o PRNG. Questi algoritmi producono sequenze numeriche che approssimano il comportamento di variabili casuali uniformi e costituiscono la base di tutti i campionamenti effettuati durante la simulazione. Il tipo di PRNG è fondamentale, in quanto sequenze non sufficientemente casuali possono introdurre correlazioni artificiali o compromettere l'accuratezza del risultato. In ambito medico vengono utilizzati dei generatori specificatamente progettati e testati per garantire le proprietà statistiche adeguate su lunghi periodi.

2.2. Il ciclo di simulazione

Nel trasporto di particelle per radioterapia ogni particella viene trattata in modo indipendente. Si simulano sequenze di eventi discreti, ciascuno dei quali è determinato tramite campionamento

¹² Programma di ricerca e sviluppo che portò alla realizzazione delle prime bombe atomiche durante la Seconda Guerra Mondiale.

¹³ Il coefficiente di attenuazione indica quanto è facile per un materiale fermare o deviare una radiazione che lo attraversa.

stocastico basato su leggi fisiche. La distribuzione finale della dose emerge come risultato statistico dell'insieme delle storie. Il trasporto di un fotone è rappresentato da una sequenza ordinata di operazioni che si ripetono fino a quando la particella esaurisce la propria energia o esce dal dominio della simulazione. I passaggi iterativi sono:

- Generazione della particella dalla sorgente
 - La simulazione inizia con l'inizializzazione della particella primaria, dove posizione direzione ed energia vengono assegnate secondo il modello di sorgente. Questa dipende dal fascio clinico e dalla geometria del LINAC.
- Campionamento del cammino libero
 - La distanza fino alla prossima interazione viene estratta dalla distribuzione esponenziale derivante dalla legge di attenuazione. Questo passaggio determina quanto lontano la particella si muove prima di interagire.
- Traversal della griglia voxelizzata
 - Poiché il paziente è rappresentato come un insieme di voxel con proprietà diverse, il cammino della particella deve essere adattato alla geometria. Viene verificato se la distanza campionata attraversa uno o più voxel e aggiorna localmente il coefficiente di attenuazione.
- Selezione del tipo di interazione
 - Una volta determinato il punto di interazione viene deciso quale processo fisico avviene, quindi se quello fotoelettrico, quello Compton o la produzione di coppie, sulla base delle probabilità relative derivate dalle sezioni d'urto.
- Esecuzione dell'interazione
 - Il processo selezionato viene simulato, in particolare, vengono campionati angoli ed energie delle particelle coinvolte secondo le distribuzioni fisiche appropriate.
- Deposito di energia
 - L'energia trasferita al mezzo viene convertita in dose nel voxel corrente. Questo si fa utilizzando l'approssimazione di deposito locale per cui l'energia ceduta agli elettroni viene considerata depositata nello stesso punto.
- Generazione e gestione delle particelle secondarie
 - Le interazioni possono produrre nuove particelle. Queste vengono aggiunte alla simulazione e tracciate a loro volta, spesso tramite una struttura dati tipo stack o coda.
- Verifica delle condizioni di terminazione
 - La storia della particella termina quando ha un'energia inferiore alla soglia, quando è uscita dal dominio o per assorbimento completo
- Ripetizione del ciclo
 - Il ciclo si ripete per tutte le particelle primarie e secondarie fino al raggiungimento della statistica desiderata.

2.3. I generatori di numeri pseudo casuali

L'intero metodo Monte Carlo si basa sulla disponibilità di numeri casuali. Un calcolatore non è in grado di generare numeri realmente casuali. Si utilizzano algoritmi deterministici che sono generatori di numeri pseudo-casuali PRNG che producono sequenze numeriche con proprietà statistiche che approssimano quelle del caso ideale. Questo indica che il metodo Monte Carlo è stocastico ma operativamente deterministico.

La qualità del PRNG è fondamentale per l'accuratezza della simulazione. Sequenze con correlazioni residue o con periodo troppo breve possono introdurre bias sistematici difficili da individuare, soprattutto per simulazioni di trasporto di particelle. Nel caso di Monte Carlo per radioterapia, e in generale per la fisica medica, si utilizzano dei generatori con:

- Lungo periodo, che tipicamente è maggiore di 10^8 ;
- Buona distribuzione multidimensionale;
- Assenza di correlazioni a corto e lungo raggio.

Su implementazioni su GPU poiché ogni thread simula una particella indipendente, è necessario garantire che ciascun thread utilizzi una sequenza pseudo-casuale indipendente dalle altre. Questo viene ottenuto tramite tecniche di stream splitting¹⁴ evitando sovrapposizioni tra sequenze. La correttezza statistica dell'intera simulazione dipende direttamente dalla qualità delle sequenze generate.

2.3.1. Scelta dei PRNG

Storicamente uno dei generatori di numeri casuali più utilizzati in ambito scientifico è il Mersenne Twister. Questo generatore è caratterizzato da un periodo estremamente lungo, circa 2^{19937} che lo hanno reso uno standard nelle simulazioni Monte Carlo su CPU. Tuttavia, questo presenta un limite strutturale nel contesto delle GPU che deriva dal fatto che richiede uno stato interno molto grande, circa 624 interi per thread, e in un'architettura CUDA, dove migliaia di thread vengono eseguiti simultaneamente, comporta un elevato consumo di memoria e accessi non efficienti allo stato.

Nelle implementazioni GPU di problemi con queste caratteristiche si utilizza Philox 4x32 in cui, invece di mantenere uno stato che evolve nel tempo, il generatore produce numeri pseudo casuali come funzione deterministica di un contatore. Questo comporta, in CUDA, avere:

- Stato minimo;
- Assenza di dipendenza sequenziale tra numeri successivi;
- Parallelizzazione naturale, poiché ogni thread può essere inizializzato con un contatore diverso.

Lo stesso Philox viene implementato nella libreria cuRAND per cui è possibile assegnare a ciascun thread una sequenza indipendente variando il contatore iniziale senza necessità di tecniche complesse di stream splitting.

2.4. Convergenza statistica

Il risultato della simulazione Monte Carlo è affetto da fluttuazioni statistiche. La distribuzione di dose ottenuta rappresenta una stima della soluzione reale e la sua precisione dipende dal numero di particelle simulate. Il principio di convergenza segue la legge dei grandi numeri per cui all'aumentare del numero di storie simulate, la media delle grandezze osservate converge al valore atteso fisico:

$$\sigma \propto \frac{1}{\sqrt{N}}$$

Con N è il numero delle particelle simulate. In questo modo si ha una riduzione dell'errore di un fattore 2 e si ottengono precisioni clinicamente accettabili solo con simulazioni molto estese.

¹⁴ Divisione di un flusso in parti piccole per elaborazione parallela.

Nel contesto della radioterapia questo vuol dire simulare un numero elevato di storie per rispettare il vincolo di accuratezza imposto a livello clinico. Inoltre, l'incertezza non è uniforme nello spazio in quanto regioni a bassa dose o piccoli volumi richiedono delle statistiche ancora più elevate. La convergenza è un vincolo computazionale diretto che influenza l'architettura del codice e la scelta delle tecniche di accelerazione.

2.5. Differenza rispetto al Monte Carlo generico

Il metodo Monte Carlo per il trasporto di particelle è diverso rispetto alle applicazioni generiche. La prima distinzione riguarda l'origine delle distribuzioni campionate. Nel Monte Carlo generico, le distribuzioni di probabilità sono parte del modello e pertanto vengono scelte per rappresentare un fenomeno o per semplificare un problema numerico. Nel trasporto di particelle, queste derivano direttamente da leggi fisiche e da dati sperimentali per cui c'è una riproduzione evento per evento.

Un'altra differenza è rappresentata dalla correttezza del risultato, che nel caso del Monte Carlo generico è corretto se la simulazione converge numericamente rispetto al modello scelto. Nel trasporto Monte Carlo per radioterapia questa equivalenza non vale, infatti, un codice può convergere e quindi produrre risultati stabili ma verso delle soluzioni fisicamente errate. In questi casi, la distribuzione di dose risultante appare accettabile ma in realtà risulta clinicamente sbagliata. Per questo motivo la validazione ha un ruolo fondamentale rispetto al Monte Carlo generico in quanto rappresenta una componente strutturale del metodo stesso. La validazione include:

- Confronto con dati sperimentali;
- Benchmark contro configurazioni standard;
- Confronto con risultati validati sperimentalmente.

Codici come DOSXYZnrc [8] sono il riferimento standard in quanto sono stati confrontati con misure reali e si utilizzano come misura di paragone nelle nuove implementazioni, che devono produrre gli stessi risultati entro le incertezze accettabili.

Queste esigenze non esistono nel Monte Carlo generico, dove il modello non ha necessariamente la necessità di descrivere una realtà fisica misurabile con precisione, a differenza del contesto della radioterapia dove il legame con la realtà è fondamentale in quanto comporta delle conseguenze cliniche rilevanti.

2.6. Problema HPC

Il calcolo del Monte Carlo della dose rappresenta un problema di High Performance Computing non tanto per la complessità delle singole operazioni ma per la combinazione di diversi fattori tra cui:

- Volume computazionale elevato;
- Comportamento stocastico;
- Accessi concorrenti alla memoria.

Il primo vincolo è di natura statistica. L'errore Monte Carlo decresce lentamente con il numero di campioni secondo una legge proporzionale a $1/\sqrt{N}$. Questo implica che, per raggiungere incertezze dell'ordine di pochi punti percentuali su grandezze locali come la dose voxelizzata, è necessario simulare un numero molto elevato di storie, tipicamente dell'ordine di 10^8 .

Dal punto di vista computazionale questo requisito, considerando che ogni particella rappresenta una traiettoria indipendente, comporta che il numero totale di operazioni cresce non solo con il numero

delle particelle ma anche con la complessità del trasporto. In questo contesto la GPU porta un vantaggio in quanto si può assegnare una o più particelle a ciascun thread, tenendo conto dei vincoli strutturali.

2.6.1. La warp divergence da stocasticità

In CUDA i thread sono organizzati in warp che eseguono le stesse istruzioni, tuttavia la natura stocastica del metodo Monte Carlo, implica che, ogni particella segua una storia diversa causata da:

- Lunghezza del cammino variabile;
- Numero di interazioni non deterministico;
- Tipi di interazione differenti;
- Generazione asincrona di secondari.

Di conseguenza all'interno di uno stesso gruppo i thread divergono rapidamente in rami di esecuzione differenti. Questa divergenza è strutturale per cui si devono adottare delle strategie di:

- Riordinamento dinamico delle particelle per stato o energia;
- Separazione del trasporto di fotoni ed elettroni;
- Utilizzo di code per uniformare il lavoro tra i thread.

Il Monte Carlo è naturalmente parallelo ma non è semplice vettorizzarlo a causa delle diverse ramificazioni.

2.6.2. La contesa sulle sezioni atomiche

Un altro problema che presenta il Monte Carlo per radioterapia riguarda la memoria durante la fase di accumulo della dose. Ogni evento di interazione contribuisce con una quantità di energia depositata in un voxel. Poiché molte particelle possono depositare energia nello stesso voxel, l'aggiornamento della dose richiede operazioni atomiche per evitare race conditions. Questo fenomeno è particolarmente critico nelle regioni ad alta fluenza, dove la probabilità che molti thread aggiornino lo stesso voxel è elevata.

Si crea così un collo di bottiglia che limita la scalabilità del codice. A differenza della divergenza che influisce sul flusso di esecuzione in questo caso il problema risiede nella contesa delle risorse di memoria. Per cercare di mitigare questo effetto si potrebbe effettuare:

- Accumulo locale per thread o per blocco;
- Riduzioni intermedie prima della scrittura globale;
- Suddivisione della griglia per ridurre la probabilità di collisione.

Queste tecniche tengono conto del fatto che il problema di sincronizzazione globale diventa composto da una serie di operazioni locali.

Il problema Monte Carlo per radioterapia è quindi un problema HPC non perché richiede delle operazioni matematiche complesse ma perché combina un numero estremamente elevato di eventi da simulare, una struttura di controllo fortemente irregolare e un accesso concorrente alla memoria condivisa.

3. Il phantom

Per la validazione del codice Monte Carlo si utilizza un phantom standard costituito da un volume omogeneo di acqua. La geometria è quella di un cubo di dimensioni $30 \times 30 \times 30 \text{ cm}^3$ discretizzato in voxel cubici di lato 1mm. Questo corrisponde ad una griglia regolare di $100 \times 100 \times 100$ per un totale di 10^6 voxel. Questa configurazione è quella che si usa nei test di validazione dei sistemi di calcolo della dose e nelle procedure di TPS. L'acqua viene utilizzata come materiale di riferimento perché approssima il tessuto molle e consente un confronto diretto dei risultati. La scelta di queste dimensioni del voxel è un compromesso tra risoluzione e costo computazionale. Un passo di 3mm è sufficiente per descrivere i gradienti di dose tipici dei fasci fotonici clinici, mantenendo allo stesso tempo un numero di voxel gestibili in memoria GPU. Questa stessa geometria viene utilizzata nello standard di riferimento DOSXYOnrc.

Un altro test di valutazione si esegue in un caso di eterogeneità. A parità di volume viene inserito al centro un cubo di materiale osseo delle dimensioni di $5 \times 5 \times 5 \text{ cm}^3$. In questo modo viene introdotta discontinuità nelle proprietà fisiche del mezzo e permette di verificare il comportamento del codice nelle interfacce tra materiali diversi. In particolare, consente di osservare le variazioni del trasporto degli elettroni secondari e modifiche locali della dose.

L'utilizzo del phantom omogeneo e di quello eterogeneo si usano per seguire la procedura standard di validazione che prevede con il primo la verifica del corretto funzionamento di base del codice e con il secondo la capacità di gestire condizioni più realistiche.

3.1. Proprietà fisiche di phantom

Per la simulazione Monte Carlo ogni voxel del phantom è associato a un materiale definito attraverso le sue proprietà fisiche. In particolare, per ciascun materiale devono essere specificati:

- Densità di massa ρ ;
- Composizione elementare in frazioni di peso;
- Densità elettronica;
- Coefficiente di attenuazione lineare μ .

Questi valori sono presi dal report ICRU [9] che contiene un database standard di riferimento.

Per quanto riguarda il primo phantom di validazione, questo è composto da acqua che è il materiale di riferimento per la dosimetria in radioterapia e viene utilizzata con i seguenti parametri:

<i>Acqua</i>	
<i>Densità</i>	$\rho = 1.0 \text{ g/cm}^3$
<i>Composizione (frazioni in peso)</i>	$H = 0.1119 \quad O = 0.8881$
<i>Densità elettronica</i>	$\sim 3.34 \times 10^{23} e^-/\text{cm}^3$
<i>Coefficiente di attenuazione</i>	$\mu = 0.0494 \text{ cm}^{-1} \text{ a } 2\text{MeV}$

L'acqua è utilizzata come riferimento in quanto le sue proprietà radiologiche sono vicine a quelle del tessuto molle dei pazienti. Questo consente di avere un confronto diretto con i risultati delle simulazioni sperimentali di riferimento.

Per quanto riguarda il secondo phantom di validazione, che contiene un cubo di materiale osseo, serve per gestire la discontinuità dei materiali e viene utilizzato con i seguenti parametri:

Osso

Densità	$\rho = 1.85 \text{ g/cm}^3$		
Composizione (frazioni in peso)	$H \approx 0.034$	$C \approx 0.155$	$N \approx 0.042$
	$O \approx 0.435$	$P \approx 0.103$	$Ca \approx 0.225$
Densità elettronica	Proporzionale alla densità e al contenuto di elementi ad alto Z		
Coefficiente di attenuazione	$\mu = 0.0441 \text{ cm}^{-1}$ a 2 MeV		

Nonostante la densità più elevata il coefficiente di attenuazione a energie dell'ordine dei MeV è comparabile con quello dell'acqua, poiché il processo predominante è il Compton che dipende dalla densità elettronica e non fortemente da Z .

Un altro materiale presente è il polmone che è modellato come tessuto molle a bassa densità:

Polmone

Densità	$\rho = 0.26 \text{ g/cm}^3$		
Composizione (frazioni in peso)	$H \approx 0.034$	$C \approx 0.155$	$N \approx 0.042$
	$O \approx 0.435$	$P \approx 0.103$	$Ca \approx 0.225$
Densità elettronica	Significativamente ridotta rispetto all'acqua		
Coefficiente di attenuazione	$\mu = 0.0086 \text{ cm}^{-1}$ a 2 MeV		

La riduzione della densità comporta un aumento del cammino libero medio e quindi una minore probabilità di interazione per unità di percorso.

L'ultimo materiale è quello dell'aria che rappresenta il limite di bassa densità e si usa per modellare cavità o per effettuare dei test:

Aria

Densità	$\rho = 0.00120 \text{ g/cm}^3$		
Composizione (frazioni in peso)	$N \approx 0.755$	$O \approx 0.232$	$Ar \approx 0.013$
Densità elettronica	Trascurabile rispetto ai tessuti biologici		
Coefficiente di attenuazione	$\mu = 0.0000457 \text{ cm}^{-1}$ a 2 MeV (trascurabile)		

A queste densità, la probabilità di interazione dei fotoni nel range MV è molto bassa.

Questi coefficienti di attenuazione sono ottenuti a partire dai coefficienti di attenuazione di massa μ/ρ tabulati dal database NIST e poi moltiplicati per la densità di materiale, mentre per quanto riguarda l'energia di riferimento, presa come 2 MeV questa è rappresentativa dello spettro medio di un fascio clinico da 6 MV . Queste proprietà del mezzo si usano per determinare il cammino libero medio delle particelle, selezionare il tipo di interazione e modellare poi il trasporto nei diversi materiali.

3.2. Struttura del phantom

Nel codice Monte Carlo, il phantom non è gestito come una struttura tridimensionale astratta ma viene rappresentato come un array lineare in memoria contigua. Viene memorizzato come un array monodimensionale di dimensione $N_x \times N_y \times N_z$ con ordinamento row-major. L'indice lineare associato a un voxel di coordinate (i, j, k) è dato da:

$$idx = i + N_x \times (j + N_y \times k)$$

Questa rappresentazione consente di memorizzare l'intero volume in un blocco contiguo di memoria globale, evitando strutture dati annidate che introdurrebbero solamente overhead. Per ogni voxel vengono memorizzate le informazioni minime necessarie al trasporto che sono un identificativo intero di materiale e la densità di massa.

In CUDA, i thread sono organizzati in warp che accedono alla memoria globale in modo efficiente solo se le richieste sono coalescenti e cioè indirizzate a locazioni contigue. Durante il trasporto delle particelle, thread adiacenti tendono a tracciare particelle con posizioni spazialmente vicine. Se la mappatura dei voxel in memoria è coerente con questa località spaziale, gli accessi alla memoria risultano contigui e quindi coalescenti.

In particolare, quando i thread accedono a voxel adiacenti lungo l'asse z , la linearizzazione garantisce che gli indirizzi in memoria siano consecutivi o quasi consecutivi. Questo permette alla GPU di servire le richieste con un numero minimo di transazioni in memoria globale, migliorando significativamente il throughput. Al contrario una rappresentazione non lineare porterebbe ad avere accessi disallineati con conseguente aumento della latenza.

Questa rappresentazione ad array lineare con mapping esplicito degli indici è lo stesso che viene utilizzato nei codici Monte Carlo per GPU, come in MC-GPU. Questo consente di:

- Ridurre il costo degli accessi alla memoria globale;
- Mantenere elevata l'occupazione dei multiprocessori;
- Garantire la scalabilità con il numero di thread.

3.3. IL LINAC e il fascio da 6MV

Nei trattamenti di radioterapia esterna, la sorgente di radiazione è un acceleratore lineare, detto LINAC, che produce fasci di fotoni ad alta energia tramite un processo di radiazione di frenamento. Elettroni accelerati a energie dell'ordine di alcuni MeV impattano su un target metallico generando un fascio di fotoni con spettro continuo. Elettroni accelerati a energie dell'ordine di alcuni MeV impattano su un target metallico generando un fascio di fotoni con spettro continuo.

Un fascio nominale da 6MV non è quindi monoenergetico ma l'energia dei fotoni è distribuita in un intervallo ampio che va da pochi keV fino a un valore massimo prossimo all'energia degli elettroni incidenti. La distribuzione energetica è fortemente asimmetrica, con una concentrazione di fotoni a basse energie e un'energia media tipicamente intorno ai 2 MeV.

La forma dello spettro non dipende solo dal processo di produzione nel target ma anche da diversi elementi del LINAC, quali:

- Il target metallico che determina la produzione iniziale;
- Il filtro di appiattimento che modifica lo spettro rimuovendo preferenzialmente i fotoni a bassa energia e rendendo il fascio più uniforme lateralmente;
- Il collimatore che introduce ulteriori effetti geometrici e di scattering.

Il risultato che si ottiene è uno spettro policromatico complesso specifico per ogni configurazione di macchina.

Nella simulazione Monte Carlo la scelta dei parametri del LINAC è un elemento fondamentale. Un fascio monoenergetico si usa solamente in contesti di verifica della fisica di base ma non delle rappresentazioni cliniche reali. Nel caso in cui si ha un fascio policromatico, la distribuzione di dose

dipende dall'interazione combinata di fotoni con energie diverse. Questo si riflette direttamente su grandezze dosimetriche fondamentali come il Percent Depth Dose o PDD. La profondità di massima dose, ad esempio, è determinata dalla componente energetica dominante. Il decadimento in profondità è più granulare in quanto i fotoni ad alta energia penetrano più a fondo.

Per evitare di calcolare e modellare in dettaglio l'intero LINAC si usano spettri pre-calcolati e validati. Questi sono una approssimazione del fascio clinico reale e permettono di isolare il problema del trasporto nel corpo del paziente. In particolare, viene utilizzato uno spettro di riferimento per fascio a 6MV che è uno standard nella letteratura e produce risultati realistici senza introdurre sorgenti di incertezza legate alla modellazione dell'intera macchina.

Per modellare in modo realistico la sorgente senza simulare esplicitamente il LINAC, viene utilizzata la libreria di spettri fotonici IAEA-TECDOC-1540. Questo documento raccoglie una serie di spettri energetici di riferimento per fasci clinici generati da acceleratori lineari, ottenuti tramite simulazione Monte Carlo. Ogni spettro è in forma tabulata come distribuzione discreta energia-fluenza, ovvero è composto da un insieme di coppie:

$$(E_i, \Phi_i)$$

Dove:

- E_i è il centro dell'intervallo energetico, il bin;
- Φ_i è la fluenza relativa normalizzata associata al centro dell'intervallo.

Le tabelle rappresentano direttamente una distribuzione di probabilità discreta dell'energia dei fotoni. Questa distribuzione contiene tutti gli effetti della produzione e del filtraggio dovuto al LINAC per cui può essere utilizzato direttamente come sorgente primaria nella simulazione.

Nel codice Monte Carlo lo spettro tabulato viene utilizzato per assegnare un'energia iniziale a ciascun fotone simulato. Poiché la distribuzione è discreta, è necessario trasformarla in una distribuzione cumulativa:

$$F(E_k) = \sum_{i=1}^k \Phi_i$$

A partire da questa, il campionamento può essere effettuato in modo efficiente, in quanto:

- Si genera un numero casuale uniforme $\xi \in (0,1)$;
- Si individua il bin k tale che $F(E_{k-1}) < \xi \leq F(E_k)$;
- Si assegna al fotone un'energia compresa nell'intervallo del bin selezionato, eventualmente con interpolazione uniforme all'interno del bin.

L'uso di questo spettro consente di separare il problema della generazione del fascio da quello del trasporto nel phantom. In questo modo si evita la simulazione esplicita del LINAC, si mantiene coerenza con la letteratura e con i codici di riferimento e si garantisce che la distribuzione energetica iniziale sia fisicamente realistica.

3.4. Il campionamento dello spettro

Una volta definito lo spettro energetico come distribuzione discreta si deve poi estrarre per ogni fotone simulato, un'energia iniziale coerente con questa distribuzione. Questo processo è stocastico e rappresenta il primo passo del metodo Monte Carlo per radioterapia.

Lo spettro viene memorizzato come una tabella di valori discreti e convertito in una distribuzione cumulativa CDF che rappresenta la probabilità che l'energia del fotone sia inferiore o uguale al valore stabilito.

Si effettua un campionamento sulla CDF, in particolare:

- Si genera un numero pseudo-casuale uniforme $\xi \in (0,1)$;
- Si cerca nell'array della CDF il primo elemento $F(E_k)$ tale che $F(E_k) \geq \xi$;
- Il bin k identifica l'intervallo energetico da cui viene estratta l'energia del fotone.

La ricerca del bin si effettua tramite ricerca binaria così da essere stabile anche per distribuzioni con molti bin. Una volta individuato il bin l'energia può essere assegnata al valore centrale del bin, oppure campionata uniformemente all'interno dell'intervallo $[E_k, E_{k+1}]$ per evitare effetti di discretizzazione. Questo metodo è particolarmente adatto all'implementazione su GPU perché evita operazioni costose e utilizza solo confronti e accessi in memoria.

In CUDA la CDF viene memorizzata in memoria globale e nel caso in cui è di dimensioni ridotte nella memoria costante. Ogni thread esegue il campionamento in modo indipendente, utilizzando il proprio generatore pseudo-casuale. L'accesso alla CDF è regolare e prevedibile, il che favorisce il caching e riduce la latenza. Inoltre, la ricerca binaria ha un flusso di controllo limitato, riducendo l'impatto della divergenza tra thread. Per questi motivi il costo computazionale del campionamento dello spettro è trascurabile rispetto alle fasi successive del trasporto.

Oltre alla distribuzione energetica la sorgente Monte Carlo deve essere definita anche dal punto di vista geometrico. In questa implementazione si adotta una configurazione standard per cui la sorgente viene modellata come un fascio incidente lungo l'asse z , con direzione perpendicolare alla superficie del phantom. Il punto di emissione è posto lungo l'asse centrale del sistema in modo da allineare il fascio con il centro geometrico del volume. Nel caso specifico del phantom $30 \times 30 \times 30 \text{ cm}^3$ il piano superiore del volume si trova a una distanza inferiore rispetto all'isocentro¹⁵.

3.4.1. Distribuzione spaziale dei fotoni

Per quanto riguarda la distribuzione spaziale iniziale dei fotoni si assume come prima approssimazione una distribuzione uniforme all'interno dell'apertura del fascio. Vengono campionate le coordinate (x, y) uniformemente distribuite nel campo proiettato e si costruisce il vettore direzione che collega il punto sorgente con il punto campionato sul piano isocentrico. In questo modo si modella un fascio idealmente uniforme, coerente con l'effetto del filtro di appiattimento del LINAC.

Si introduce poi una distribuzione gaussiana centrata sull'asse del fascio, che rappresenta la non uniformità residua del profilo laterale. Nel contesto della validazione del codice, tuttavia, si utilizza la distribuzione uniforme per ridurre il numero di parametri liberi, garantire la coerenza nel confronto con

¹⁵ L'isocentro è il punto geometrico fisso nello spazio tridimensionale coincidente con il centro del tumore attorno al quale ruotano le apparecchiature radioterapiche.

DOSXYZnrc. Tutti i parametri geometrici della sorgente devono essere identici tra i due sistemi, quello sviluppato e lo standard di riferimento per rendere valida e coerente la validazione dei risultati.

3.5. Database NIST XCOM

Il calcolo Monte Carlo del trasporto dei fotoni richiede come input fondamentale i coefficienti di attenuazione dei materiali attraversati. Questi non vengono calcolati al momento ma sono prelevati da database fisici validati. Il riferimento standard in ambito radioterapico per questi valori è il database NIST XCOM. XCOM è una libreria di dati sviluppata dal National Institute of Standards and Technology che fornisce i coefficienti di interazione dei fotoni con la materia per tutti gli elementi della tavola periodica e per i composti di interesse medico. Nel codice Monte Carlo questi vengono memorizzati come tabelle funzione dell'energia. Poiché l'energia del fotone varia continuamente durante la simulazione è necessario interpolare tra i valori tabellari. In particolare:

- Dato un valore di energia E , si individuano i due punti tabulati E_i e E_{i+1} ;
- Si esegue un'interpolazione lineare per ottenere $\mu(E)$.

Dal punto di vista dell'HPC la gestione di questi dati è un aspetto fondamentale in quanto i coefficienti di attenuazione sono utilizzati da tutti i thread durante la simulazione e rappresentano un caso di accesso condiviso. Queste tabelle sono allocate nella constant memory della GPU invece che nella global memory. Questo perché la constant memory è ottimizzata per accessi broadcast, pertanto, quando più thread leggono lo stesso indirizzo il dato viene gestito in modo efficiente nella cache con latenza ridotta.

Nel contesto del Monte Carlo per radioterapia molti thread si trovano in condizioni energetiche simili nello stesso momento della simulazione e l'accesso ai coefficienti è frequente e ripetuto. L'uso della constant memory permette quindi di ridurre significativamente il costo degli accessi e migliorare le prestazioni complessive del kernel.

3.6. Energie di cutoff

Nel Monte Carlo per radioterapia, non tutte le interazioni vengono simulate fino a energie basse ma si introducono delle soglie energetiche di cutoff al di sotto delle quali la particella non viene più tracciata esplicitamente. In particolare, si distinguono:

- ECUT o energia di cutoff per fotoni;
- PCUT o energia di cutoff per le particelle cariche.

Quando l'energia di una particella scende sotto la soglia impostata il suo trasporto viene interrotto e l'energia residua viene depositata localmente nel voxel corrente.

Questa approssimazione è giustificata dal fatto che a basse energie il range delle particelle diventa estremamente ridotto. Di conseguenza, tracciare esplicitamente il suo trasporto non modifica in modo significativo la distribuzione spaziale della dose e aumenta di tanto il numero di interazioni da simulare.

Nel presente progetto si utilizzano valori standard di riferimento che sono:

- $ECUT = 10 \text{ keV}$ per i fotoni;
- $PCUT = 100 \text{ keV}$ per gli elettroni in caso di trasporto esplicito.

Il parametro PCUT, considerando che si considera l'approssimazione di deposito locale degli elettroni secondari non ha rilevanza in quanto gli elettroni vengono trasportati esplicitamente e la loro energia viene depositata direttamente nel punto di interazione.

La scelta del cutoff influisce sull'accuratezza della simulazione e sul tempo di calcolo. Abbassare il suo valore significa avere maggiore accuratezza in quanto vengono simulate anche interazioni a bassa energia ma si ha un aumento significativo del numero di eventi da tracciare. Al contrario, aumentare il cutoff riduce il costo computazionale ma produce errori sistemici soprattutto nelle regioni superficiali.

3.7. Tabella dei parametri

La tabella dei parametri utilizzati nella simulazione e confrontabili con lo standard DOSXYZ di riferimento è:

<i>Parametro</i>	<i>Valore</i>	
<i>Dimensioni phantom</i>	$30 \times 30 \times 30 \text{ cm}^3$	
<i>Numero voxel</i>	$100 \times 100 \times 100$	
<i>Dimensione voxel</i>	$3 \times 3 \times 3 \text{ mm}^3$	
<i>Materiali</i>	Acqua	Inserito osseo
<i>Densità</i>	1.0 g/cm^3	1.85 g/cm^3
<i>Energia fascio</i>	6 MV	
<i>Energia media</i>	$\sim 2 \text{ MeV}$	
<i>SAD</i>	100 cm	
<i>Campo</i>	$10 \times 10 \text{ cm}^2$	
<i>Distribuzione spaziale</i>	Uniforme	
<i>Tipo di interpolazione</i>	Lineare in energia	
<i>ECUT</i>	10 keV	
<i>PCUT</i>	100 keV	
<i>Numero particelle per esecuzione</i>	$10^7/10^8$	

Tabella 1 - Tabella dei parametri

4. Analisi della letteratura

4.1. Codici Monte Carlo su CPU

4.1.1. EGSnrc

EGSnrc rappresenta il riferimento principale per la simulazione Monte Carlo del trasporto di fotoni ed elettroni nella fisica medica. Il codice è stato sviluppato dal National Research Council Canada ed è l'evoluzione del sistema EGS4. EGSnrc rappresenta una formulazione avanzata dell'algoritmo di condensed history per il trasporto delle particelle cariche, che consente di trattare in modo efficiente il gran numero di interazioni a bassa energia senza perdere accuratezza nella distribuzione della dose.

Dal punto di vista fisico implementa:

- Una modellazione completa delle interazioni fotone-materia, quindi effetto fotoelettrico, Compton e produzione di coppie;
- Un trasporto accurato degli elettroni secondari con gestione delle perdite energetiche, scattering multiplo e produzione di radiazione secondaria;
- L'utilizzo di database fisici validati per le sezioni d'urto e le proprietà di interazione.

Un elemento caratteristico di questa implementazione è la validazione sperimentale. Il codice EGSnrc è stato confrontato con misure dosimetriche in phantom e in condizioni cliniche reali diventando di fatto lo standard di riferimento per la verifica di nuovi algoritmi Monte Carlo in radioterapia. Qualsiasi nuova implementazione per essere considerata affidabile deve produrre dei risultati che siano coerenti con quelli prodotti da queste simulazioni, entro i limiti clinici accettabili.

EGSnrc non viene utilizzato direttamente ma costituisce il riferimento fisico sottostante al codice DOSXYZnrc che viene utilizzato oggi. EGSnrc definisce lo stato dell'arte per quanto riguarda l'accuratezza del trasporto Monte Carlo in fisica medica su CPU. Le limitazioni principali non sono di natura fisica ma computazionale. Infatti, i tempi di simulazione richiesti per ottenere incertezze statistiche sono elevate, il che rende impossibile l'utilizzo in contesti clinici in tempo reale.

4.1.2. DOSXYZnrc

DOSXYZnrc è il codice di riferimento per il calcolo della dose in geometrie voxelizzate ed è parte integrante del sistema EGSnrc. A differenza del codice EGSnrc che si occupa del trasporto fisico delle particelle, DOSXYZnrc è progettata per risolvere il problema clinico della distribuzione di dose all'interno di un paziente o di un phantom discretizzato.

Il codice è strutturato su una geometria definita come una griglia tridimensionale di voxel, ciascuno caratterizzato da materiale e densità standard. Questa struttura lo rende adatto alla simulazione in radioterapia, dove la rappresentazione voxelizzata del paziente arriva direttamente dalle immagini TAC. Dal punto di vista operativo, questa implementazione consente di calcolare:

- Curve di dose percentuale in profondità (PDD);
- Profili laterali del fascio;
- Distribuzioni tridimensionali di dose.

L'input del progetto è rappresentato da una serie di file che definiscono la geometria del phantom, le proprietà dei materiali, la sorgente e i parametri di simulazione. L'output è rappresentato da un file in formato *.3ddose* che contiene la mappa tridimensionale della dose e l'incertezza statistica associata a ciascun voxel. Questo formato è uno standard nella letteratura e comunità scientifica ed è spesso utilizzato per i confronti statistici. Il confronto effettuato su questi risultati, tramite le curve PDD e i profili di dose, permette di verificare che la modellizzazione delle interazioni e del trasporto sia corretta entro le tolleranze cliniche richieste.

DOSXYZ non simula direttamente il LINAC ma utilizza il codice BEAMnrc che modella la testa acceleratrice e genera le distribuzioni di particelle incidenti.

4.1.3. Penelope

PENELOPE è uno dei principali codici Monte Carlo alternativi allo standard EGSnrc per la simulazione del trasporto di fotoni, elettroni e positroni. La sua caratteristica distintiva è rappresentata dal trasporto degli elettroni che avviene sulla base di un algoritmo misto hard/soft. Le collisioni hard vengono trattate in modo esplicito e stocastico, mentre le collisioni soft sono trattate in modo condensato attraverso medie statistiche. Questo approccio consente un controllo fine dell'accuratezza, in particolare in un regime di basse energie, dove gli elettroni subiscono un numero elevato di interazioni e dove i dettagli microscopici del trasporto diventano rilevanti.

Questo metodo viene utilizzato come codice indipendente per la validazione incrociata in quanto utilizza modelli fisici e strategie numeriche diverse rispetto a EGSnrc per cui è più semplice trovare eventuali bias sistematici.

4.2. Implementazioni su GPU

4.2.1. MC-GPU

MC-GPU di Badal e Badano del 2009 rappresenta la prima implementazione completa di un codice Monte Carlo per il trasporto di fotoni in radioterapia sviluppato su architetture GPU con CUDA. In questa implementazione, il problema del trasporto delle particelle che storicamente è affrontato su CPU con tempi di calcolo elevati è stato riformulato in maniera tale da sfruttare il parallelismo massivo delle GPU.

Il codice implementa il trasporto di fotoni in un phantom voxelizzato derivato da immagini TAC, mantenendo una modellizzazione fisica coerente con i codici di riferimento su CPU. Sono inclusi i tre processi fondamentali di interazione fotone-materia, insieme alla gestione dei secondari. La sorgente LINAC è modellata come un fascio policromatico, con spettro energetico tipico del LINAC stesso.

Ogni thread GPU è responsabile del trasporto di una o più particelle, sfruttando il fatto che le storie delle particelle sono statisticamente indipendenti. Questo consente di eseguire simultaneamente decine di migliaia di traiettorie, ottenendo un'accelerazione significativa rispetto all'esecuzione seriale su CPU. Ha la rappresentazione del phantom come array lineare in memoria globale per garantire accessi coalescenti, fa un uso di generatori pseudo-casuali indipendenti per thread. Inoltre, effettua una minimizzazione delle dipendenze tra thread per evitare sincronizzazioni costose. Il suo Speedup è di circa 27x rispetto alla CPU.

4.2.2. gDPM

Il codice GPU based Dose Planning Method introduce esplicitamente il compromesso tra accuratezza fisica e prestazioni computazionali. A differenza di MC-GPU che mantiene una modellizzazione completa del trasporto, gDPM si basa sul modello DPM o Dose Planning Method, che è una formulazione semplificata del trasporto fotone-elettrone progettata per il calcolo rapido della dose in radioterapia.

La semplificazione riguarda principalmente il trattamento degli elettroni secondari che, invece di simulare in dettaglio tutte le interazioni, utilizzano approssimazioni che riducono il numero di eventi da tracciare. Questo comporta una perdita controllata di accuratezza fisica ma consente di avere una notevole riduzione del costo computazionale. In questo contesto viene modellato il concetto della warp divergence secondo cui ogni particella segue una traiettoria stocastica indipendente e i thread appartenenti alla stessa warp tendono a divergere rapidamente. Dal punto di vista delle prestazioni riporta uno speedup compreso tra 69x e 115x rispetto alle implementazioni CPU dimostrando che accettando un livello controllato di approssimazione fisica è possibile ottenere guadagni di diversi ordini di grandezza superiori.

4.2.3. GPUMCD

GPUMCD è la prima implementazione che effettua un trasporto completo fotoni-elettroni mantenendo un livello di accuratezza quasi comparabile a quello delle CPU di riferimento. Preserva la fedeltà del trasporto includendo la generazione e il tracciamento degli elettroni secondari. Durante la simulazione, ogni interazione comporta il deposito di energia in un voxel che deve essere aggiornato in memoria

condivisa o globale. Per mitigare questa problematica tiene conto del fatto che molti thread depositano energia negli stessi voxel e che la risoluzione spaziale del phantom è elevata.

4.3. Architetture ibride

4.3.1. ARCHER

ARCHER rappresenta uno dei primi tentativi di estendere il Monte Carlo per radioterapia con una implementazione eterogenea e distribuita. Il sistema combina CPU, GPU e acceleratori many-core gestiti da MPI con l'obiettivo di scalare il calcolo su più nodi mantenendo elevata efficienza.

Tiene conto del fatto che il trasporto Monte Carlo è intrinsecamente parallelizzabile perché le storie delle particelle sono indipendenti. Distribuisce il numero totale di storie tra i diversi processi MPI e al termine della simulazione le distribuzioni di dose parziali vengono combinate tramite una operazione di *MPI_Reduce* sulla griglia tridimensionale. Questo meccanismo evita le comunicazioni durante la simulazione che rimane locale e indipendente su ogni nodo. CPU, GPU e MIC non hanno le stesse prestazioni per storia simulata pertanto la distribuzione del carico non può essere uniforme. Il sistema utilizza un meccanismo di bilanciamento che tiene conto delle capacità relative dei dispositivi assegnando un numero di storie proporzionale alla loro potenza computazionale.

4.3.2. MC-GPU 1.3

La versione MC-GPU 1.3 del 2015 è un'evoluzione del codice di Badal e Badano e introduce una configurazione multi-GPU e un miglioramento dell'ambiente HPC. L'estensione principale riguarda l'utilizzo di MPI per distribuire il lavoro tra più GPU seguendo un approccio simile a quello adottato in ARCHER. Ogni processo MPI controlla una GPU e simula un sottoinsieme indipendente delle particelle e nella fase finale si ha una combinazione delle distribuzioni di dose tramite riduzione.

Questa implementazione utilizza le CUDA streams, pertanto, mentre una parte dei dati viene trasferita tra host e device le altre operazioni possono continuare ad essere eseguite sulla GPU. Questo meccanismo evita i colli di bottiglia legati alle comunicazioni tipiche delle configurazioni distribuite. Essendo basato sul codice originale si può utilizzare come confronto diretto come DOSXYZnrc.

4.4. Problemi delle soluzioni su architetture attuali

La letteratura sul metodo Monte Carlo per radioterapia su GPU è tutta concentrata nel periodo 2009/2015 per cui sono tutte basate su architetture obsolete. Con l'introduzione dell'architettura Ampere(A100) e successive questi codici sono diventati non trasferibili direttamente. La cache L2, ad esempio, è passata da pochi megabyte a decine di megabyte. Un aumento di questa scala modifica il comportamento degli accessi al phantom voxelizzato, che nei codici Monte Carlo per radioterapia sono irregolari. Accessi, infatti, che in precedenza causavano miss frequenti e latenza elevata possono essere parzialmente assorbiti dalla cache alterando il bilancio tra il tempo di calcolo e il tempo di memoria.

Un altro aspetto riguarda il throughput e la gestione del parallelismo. Le strategie di parallelizzazione ottimali sulle tecnologie precedenti non sono più ottimali sulle nuove, soprattutto quando si considerano gli effetti come la warp divergence o la contesa sulle operazioni atomiche. Alla luce di queste implementazioni e delle criticità architetturali il presente lavoro si pone l'obiettivo di sviluppare un nuovo codice Monte Carlo che tenga conto della caratterizzazione sistematica delle prestazioni del trasporto Monte Carlo su GPU moderne isolando e quantificando il contributo delle principali ottimizzazioni HPC.

5. Implementazione

La parte implementativa inizia da una versione CPU sequenziale che funge da baseline fisica del problema e viene utilizzata come metro di validazione di tutte le implementazioni successive. A partire da questa viene implementata una versione CPU parallela in cui viene distribuito il carico tra i core disponibili tramite meccanismo Round Robin. Sono state implementate poi tre versioni GPU in CUDA, in particolare, è presente una versione base che rappresenta il porting del codice sequenziale in cui viene assegnato un thread per fotone, poi si passa a una seconda versione con distribuzione dinamica del lavoro tra i vari thread eliminando lo sbilanciamento del carico per concludere poi con una versione tre che implementa una conversione parziale a singola precisione per le operazioni non critiche e una gestione efficace della memoria tramite texture memory. Per ciascuna versione si ha l'implementazione semplificata Beer Lambert, con gli stessi meccanismi di ottimizzazione da usare nella fase di testing per validare il modello fisico.

5.1. Versione CPU sequenziale

La versione CPU sequenziale rappresenta la baseline del progetto sulla quale poi verranno misurati i guadagni delle diverse versioni parallelizzate. Questo codice, inoltre, viene considerato come base fisica validata attraverso appositi test.

Questa versione, scritta in C++, è composta da diversi moduli tra cui:

Tipo modulo	
File header	Physics.h
	Compton.h
	Phantom.h
	Output.h
	Random.h
File sorgente	Main.cpp
	BeerLambert.cpp

Tabella 2 - Moduli versione CPU sequenziale

Dal punto di vista fisico è stato implementato il trasporto di fotoni in un phantom voxelizzato tridimensionale, riproducendo i tre processi di interazione fondamentali nell'intervallo energetico della radioterapia che sono: effetto fotoelettrico, scattering Compton e produzione di coppie. L'energia depositata viene accumulata voxel per voxel secondo l'approssimazione KERMA locale che equipara la dose al punto di interazione con l'energia trasferita agli elettroni secondari, trascurandone il range. Questa approssimazione è fisicamente giustificata dal fatto che per fasci di mega voltaggio in acqua, il range degli elettroni Compton è dell'ordine del centimetro e la griglia ha una risoluzione di 3mm per voxel.

Viene simulato, in particolare, un fascio parallelo $10 \times 10 \text{ cm}^2$ centrato sul phantom, il cui spettro energetico riproduce quello di un acceleratore lineare LINAC e in particolare il *Varian 2100C* a 6MV secondo i dati tabulati da *Sheikh-Bagheri & Rogers* [10]. A partire da questo fascio ogni fotone viene tracciato voxel per voxel usando l'algoritmo di traversal [11] finché non esce dal volume, non scende sotto la soglia energetica $ECUT = 10 \text{ keV}$ o non viene assorbito per effetto fotoelettrico.

5.1.1. Physics.h

Questo modulo contiene tutte le costanti, le tabelle di sezione d'urto e le funzioni di interpolazioni che vengono chiamate ad ogni passo di trasporto. La scelta di tenerle in un header separato, invece di distribuirle nei vari file è dovuta al fatto che il compilatore può fare inlining di tutte le chiamate, eliminando l'overhead di funzione su un codice che chiama questa routine di milioni di volte per simulazione.

Le costanti fondamentali sono:

```
static const double ME_C2 = 0.51099895; // MeV massa a riposo elettrone
static const double ECUT = 0.010; // MeV cutoff fotoni (10 keV)
static const double PCUT = 0.100; // MeV cutoff elettroni
static const int NX = 100, NY = 100, NZ = 100; // voxel per asse
static const double VOXEL_CM = 0.30; // lato voxel [cm] = 3 mm
```

Le sezioni d'urto di attenuazione (μ/ρ) sono derivate direttamente dal database NIST XCOM che è il riferimento standard internazionale per questo tipo di calcoli. Vengono definite separatamente per i due materiali considerati che sono acqua e osso e per quattro processi fisici che sono attenuazione totale, effetto fotoelettrico, scattering Compton e produzione di coppie. La griglia energetica copre 28 punti da 0.01 MeV a 20 MeV coprendo tutto l'intervallo utile per la radioterapia a fotoni.

```
static const int PUNTI_CAMPIONAMENTO = 28;
static const double GRIGLIA_ENERGIA[PUNTI_CAMPIONAMENTO] = { ... }
static const double PROBABILITA_TOTALE[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO] = { ... }
static const double PROBABILITA_FOTOELETTRICO[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO] = { ... }
static const double PROBABILITA_COMPTON[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO] = { ... }
static const double PROBABILITA_COPPIA[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO] = { ... }
```

Invece di simulare un fascio monoenergetico, lo spettro di un LINAC reale viene riprodotto con 24 bin da 0.25 MeV a 6.00 MeV. I valori di fluensa relativa sono tratti dalla letteratura e normalizzati a somma unitaria. Lo spettro ha un picco intorno a 1 MeV con una coda che si estende fino a 6 MeV riproducendo la forma classica della radiazione di frenamento di megavoltaggio.

```
static const int NUMERO_BINS_SPETTRO = 24;
static const double ENERGIE_SPETTRO[NUMERO_BINS_SPETTRO] = {
    0.25, 0.50, 0.75, 1.00, 1.25, 1.50, 1.75, 2.00,
    2.25, 2.50, 2.75, 3.00, 3.25, 3.50, 3.75, 4.00,
    4.25, 4.50, 4.75, 5.00, 5.25, 5.50, 5.75, 6.00
};
static const double FLUENZA_SPETTRO[NUMERO_BINS_SPETTRO] = {
    0.0243, 0.0676, 0.0862, 0.0929, 0.0919, 0.0868, 0.0794, 0.0712,
    0.0628, 0.0548, 0.0471, 0.0399, 0.0334, 0.0276, 0.0224, 0.0178,
    0.0138, 0.0104, 0.0075, 0.0052, 0.0034, 0.0020, 0.0010, 0.0004
};
```

Poiché l'energia di un fotone dopo uno scattering Compton è un valore continuo che non coincide necessariamente con uno dei 28 nodi della griglia, è necessario interpolare. La funzione *interpolazione_lineare_energia()* implementa una ricerca binaria sui nodi della griglia seguita da un'interpolazione lineare nello spazio energetico. Tecnicamente si potrebbe usare un'interpolazione log-log che è più accurata perché le sezioni d'urto variano quasi esponenzialmente, ma per l'intervallo

0.1 – 6 Mev dominato dal Compton l'errore dell'interpolazione lineare è trascurabile rispetto al rumore statistico Monte Carlo con 10^6 fotoni.

```
inline double interpolazione_lineare_energia(double energia_mev, const double tabella[PUNTI_CAMPIONAMENTO]) {
    if (energia_mev <= GRIGLIA_ENERGIA[0])
        return tabella[0];
    if (energia_mev >= GRIGLIA_ENERGIA[PUNTI_CAMPIONAMENTO-1])
        return tabella[PUNTI_CAMPIONAMENTO-1];
    int indice_limite_inferiore = 0;
    int indice_imite_superiore = PUNTI_CAMPIONAMENTO - 1;
    while (indice_imite_superiore - indice_limite_inferiore > 1) {
        int punto_centrale = (indice_limite_inferiore + indice_imite_superiore) / 2;
        if (GRIGLIA_ENERGIA[punto_centrale] <= energia_mev){
            indice_limite_inferiore = punto_centrale;
        }else{
            indice_imite_superiore = punto_centrale;
        }
    }
    double fattore_interpolazione = (energia_mev - GRIGLIA_ENERGIA[indice_limite_inferiore]) /
    (GRIGLIA_ENERGIA[indice_imite_superiore] - GRIGLIA_ENERGIA[indice_limite_inferiore]);
    return tabella[indice_limite_inferiore] * (1.0 - fattore_interpolazione) + tabella[indice_imite_superiore] *
    fattore_interpolazione;
}
```

Le funzioni *calcolo_attenuazione_totale()*, *calcolo_attenuazione_fotoelettrico()*, *calcolo_attenuazione_compton()* e *calcolo_attenuazione_coppie()* vengono utilizzate per moltiplicare il coefficiente di massa per la densità materiale, restituendo il coefficiente lineare in cm^{-1} . Questo valore è quello che entra nella legge di Beer Lambert per il campionamento del cammino libero come:

$$\lambda = -\ln(\xi)/\mu$$

La funzione *seleziona_tipo_interazione()* implementa il campionamento del tipo di interazione utilizzando le probabilità relative. Un singolo numero casuale viene confrontato con le soglie cumulate per determinare quale processo si verifica, riproducendo la competizione tra i tre canali fisici.

```
inline int seleziona_tipo_interazione(double energia, int materiale, double xi) {
    double probabilita_totale = calcolo_attenuazione_totale(energia, materiale);
    if (probabilita_totale <= 0.0)
        return 1;
    double probabilita_fotoelettrico = calcolo_attenuazione_fotoelettrico(energia, materiale) / probabilita_totale;
    double probabilita_compton = calcolo_attenuazione_compton(energia, materiale) / probabilita_totale;
    if (xi < probabilita_fotoelettrico)
        return 0; // fotoelettrico
    if (xi < probabilita_fotoelettrico + probabilita_compton)
        return 1; // compton
    return 2; // produzione di coppie
}
```

Per i fotoni sotto $2\text{ mec}^2 = 1.022\text{ MeV}$ la produzione di coppie è cinematicamente impossibile e i relativi coefficienti nella tabella NIST sono zero. La funzione restituisce, infatti, sempre 0 o 1 per energie inferiori a questa soglia.

La funzione `phantom_idx()` calcola l'indice lineare di un voxel (i_x, i_y, i_z) con ordinamento row-major. Questa scelta favorisce la località della cache quando si scansiona il volume lungo z che è la direzione del fascio, che è anche il pattern di accesso più frequente nel calcolo del PDD.

```
inline int phantom_idx(int ix, int iy, int iz) {  
    return ix + NX * (iy + NY * iz);  
}
```

5.1.2. Random.h

Il modulo `random.h` rappresenta il generatore pseudocasuale. Nel metodo Monte Carlo per radioterapia sono richiesti periodi molto lunghi così da non avere correlazioni con 10^6 o più particelle, alta velocità di generazione e buona qualità statistica per superare i test fisici. L'algoritmo Xoshiro256 di Blackman rappresenta l'approccio più utilizzato in questo ambito in quanto soddisfa tutte le condizioni. In particolare:

- Il periodo è di $2^{256}-1$;
- La generazione di un double richiede circa 4 operazioni a 64 bit;
- Mantiene una buona qualità statistica.

```
struct Xoshiro256 {  
    uint64_t stato[4];  
    ...  
    double operator()() {  
        double numero_decimale;  
        do {  
            numero_decimale = (double)(genera_prossimo_numero() >> 11) * (1.0 / (double)(1ULL << 53));  
        } while (numero_decimale <= 0.0);  
        return numero_decimale;  
    }  
};
```

Dove `ULL` è un intero a 64 bit senza segno

L'inizializzazione usa l'algoritmo SplitMix64 per espandere il seed scalare a 64 bit nei quattro word dello stato. Questo è fondamentale perché un'inizializzazione naïve produrrebbe stati correlati con artefatti statistici nei primi milioni di campioni. Con SplitMix64 si garantisce invece che seed adiacenti producano stati iniziali decorrelati. Questo è importante soprattutto se si lanciano dei run paralleli con seed diversi.

```
explicit Xoshiro256(uint64_t seed) {  
    auto SplitMix64 = [](uint64_t &stato_corrente) -> uint64_t {  
        stato_corrente += 0x9e3779b97f4a7c15ULL;  
        uint64_t valore_temporaneo = stato_corrente;  
        valore_temporaneo=(valore_temporaneo^ (valore_temporaneo >> 30)) * 0xbf58476d1ce4e5b9ULL;  
        valore_temporaneo=(valore_temporaneo^(valore_temporaneo>>27))* 0x94d049bb133111ebULL;  
        return valore_temporaneo ^ (valore_temporaneo >> 31);  
    };
```

```

};
stato[0] = SplitMix64(seed);
stato[1] = SplitMix64(seed);
stato[2] = SplitMix64(seed);
stato[3] = SplitMix64(seed);
}

```

Dove i numeri magici rappresentano:

- *0x9e3779b97f4a7c15ULL* è la rappresentazione a 64 bit della sezione aurea di $2^{64}/\phi$. I numeri che sono multipli della sezione aurea¹⁶ sono distribuiti in modo uniforme. Ogni volta che il generatore avanza, somma questa costante garantendo che i bit cambino il più possibile ed evitando di rimanere bloccati su pattern ripetitivi.
- *0xbf58476d1ce4e5b9ULL* e *0x94d049bb133111ebULL* sono le costanti di Avalanche e servono per far sì che i bit esplodano rimescolandosi su tutta la lunghezza dei 64 bit. Sono numeri primi molto grandi con una distribuzione di bit 0 e 1 bilanciata.

La struttura dello spettro precalcola la CDF al momento della costruzione, prima dell'avvio della simulazione. Il campionamento avviene poi con ricerca binaria sulla CDF in tempo $O(\log N)$. Si estrae ξ , si trova il bin i tale che $CDF[i - 1] \leq \xi < CDF[i]$ e si aggiunge un offset uniforme per render la distribuzione continua.

```

double campiona_energia(Xoshiro256 &rng) const {
    double xi = rng();
    int indice_limite_inferiore = 0;
    int indice_limite_superiore = NUMERO_BINS_SPETTRO - 1;
    while (indice_limite_inferiore < indice_limite_superiore) {
        int punto_centrale = (indice_limite_inferiore + indice_limite_superiore) / 2;
        if (cdf[punto_centrale] < xi) {
            indice_limite_inferiore = punto_centrale + 1;
        }
        else {
            indice_limite_superiore = punto_centrale;
        }
    }
    double energia_centrale = energie[indice_limite_inferiore];
    double offset = (rng() - 0.5) * larghezza_intervallo_energetico_bin;
    double energia = energia_centrale + offset;
    if (energia < 0.01) {
        energia = 0.01;
    }
    if (energia > 6.00) {
        energia = 6.00;
    }
    return energia;
}
};

```

¹⁶ La sezione aurea o Φ è un numero irrazionale considerato come il numero più irrazionale che esista. Evita le sovrapposizioni e garantisce la massima dispersione.

5.1.3. Compton.h

Lo scattering Compton è il processo dominante nell'intervallo $0.1 - 6.0 \text{ MeV}$ in acqua utilizzato. La distribuzione angolare è descritta dalla formula di Klein-Nishina che in termini del rapporto $\tau = E'/E$ non può essere invertita analiticamente, dove:

- E' è l'energia del fotone diffuso;
- E è l'energia incidente.

Per campionarla si usa il metodo di Kahn che è un algoritmo di rejection sampling utilizzato in questo tipo di distribuzioni. La distribuzione di Klein-Nishina viene decomposta in due rami con aree diverse:

- $A_{log} = \ln\left(\frac{1}{\tau_{min}}\right)$;
- $A_{lin} = \frac{1-\tau_{min}^2}{2}$.

Il campionamento procede in due fasi: prima si seleziona il ramo in base ad ξ_1 e si genera un valore di τ candidato, poi si testa l'accettazione con il criterio $p_{acc} = 1 - \tau \sin^2 \theta / (1 + \tau^2)$. Se il test fallisce il campione viene scartato. La funzione segnala il fallimento impostando $\cos \theta = 2.0$ che è un valore fisicamente impossibile che il wrapper interpreta come segnale di ripetere il campionamento.

```
inline void metodo_kahn(double energia_mev, double xi_1, double xi_2, double xi_3, double &cos_theta, double
&energia_scatter) {
    double alpha = energia_mev / ME_C2;
    double tau_min = 1.0 / (1.0 + 2.0 * alpha);
    double area_ramo_1 = std::log(1.0 / tau_min);
    double area_ramo_2 = (1.0 - tau_min * tau_min) * 0.5;
    double area_totale = area_ramo_1 + area_ramo_2;
    double tau;
    if (xi_1 * area_totale < area_ramo_1) {
        tau = std::pow(tau_min, 1.0 - xi_2);
    } else {
        double tau_min_quadro = tau_min * tau_min;
        double tau_quadro = tau_min_quadro + xi_2*(1.0 - tau_min_quadro);
        tau = std::sqrt(std::max(tau_quadro, 1e-30));
    }
    tau = std::min(std::max(tau, tau_min), 1.0);
    cos_theta = 1.0 - (1.0 - tau) / (alpha * tau);
    cos_theta = std::min(std::max(cos_theta, -1.0), 1.0);
    energia_scatter = tau * energia_mev;
    double sin2_theta = std::max(0.0, 1.0 - cos_theta * cos_theta);
    double termine_correttivo = (tau * sin2_theta) / (1.0 + tau * tau);
    double probabilita_accettazione = 1.0 - termine_correttivo;
    probabilita_accettazione = std::max(0.0, std::min(probabilita_accettazione, 1.0));
    if (xi_3 > probabilita_accettazione) {
        cos_theta = 2.0;
    }
}
```

Il wrapper *estrazione_compton()* esegue il loop rejection finché non ottiene un valore valido. In media converge in meno di 2 iterazioni per l'intervallo considerato per cui il costo computazionale aggiuntivo

è limitato. Il ciclo infinito in questo contesto fisico è garantito che termini. L'algoritmo di Kahn, infatti, lavora su una distribuzione di probabilità nota e limitata per cui il valore valido viene estratto non appena il numero casuale è minore o uguale della probabilità di accettazione. Poiché la probabilità di successo per le energie dei fotoni usate in radioterapia è sempre relativamente alta il valore accettabile viene trovato in tempi brevissimi.

```
template<typename RNG>
inline void estrazione_compton(double energia_mev, RNG &rng, double &cos_theta, double &energia_scatter) {
    while(true){
        double xi_1 = rng();
        double xi_2 = rng();
        double xi_3 = rng();
        metodo_kahn(energia_mev, xi_1, xi_2, xi_3, cos_theta, energia_scatter);
        if (cos_theta <= 1.0)
            return;
    }
}
```

In questo metodo, l'uso del template permette alla funzione di accettare qualsiasi tipo di generatore casuale senza dover riscrivere il codice ogni volta. Il compilatore vede quale generatore si sta usando e unisce il codice del generatore direttamente nella logica compton con l'operazione di inlining. Questo è importante perché, nel caso di milioni di fotoni, si evita il ritardo causato dal dover cercare il generatore in memoria ad ogni urto compton che si ottiene.

Dopo l'effetto compton il fotone ha un nuovo angolo polare e un nuovo angolo azimutale campionato uniformemente in $[0, 2\pi)$ perché la sezione d'urto di Klein-Nishina è simmetrica attorno all'asse del fascio. La funzione di aggiornamento della traiettoria trasforma questi angoli nel nuovo versore del sistema applicando una matrice di rotazione 3D.

```
inline void aggiornamento_traiettoria(double &ux, double &uy, double &uz, double cos_theta, double phi) {
    double sin_theta = std::sqrt(std::max(0.0, 1.0 - cos_theta * cos_theta));
    double cos_phi = std::cos(phi);
    double sen_phi = std::sin(phi);
    double ux_new, uy_new, uz_new;
    if (std::fabs(uz) > 0.99999) {
        double segno = 1.0;
        if(uz > 0.0){
            segno = 1.0;
        }else{
            segno = -1.0;
        }
        ux_new = sin_theta * cos_phi;
        uy_new = sin_theta * sen_phi * segno;
        uz_new = cos_theta * segno;
    } else {
        double proiezione_xy = std::sqrt(1.0 - uz * uz);
        ux_new = sin_theta * (ux * uz * cos_phi - uy * sen_phi) / proiezione_xy + ux * cos_theta;
        uy_new = sin_theta * (uy * uz * cos_phi + ux * sen_phi) / proiezione_xy + uy * cos_theta;
        uz_new = -sin_theta * cos_phi * proiezione_xy + uz * cos_theta;
    }
}
```



```

double norm = std::sqrt(ux_new * ux_new + uy_new * uy_new + uz_new * uz_new);
if (norm > 0.0){
    ux = ux_new/norm;
    uy = uy_new/norm;
    uz = uz_new/norm;
}
}

```

Si considera, in questo metodo, anche il caso degenere e cioè quando il fotone viaggia quasi esattamente lungo l'asse Z per cui $|u_z| > 0.99999$. In questo caso la formula generale è numericamente instabile perché la proiezione sul piano XY è quasi nulla. In questo caso si evitano divisioni per valori prossimi allo 0. Infine, viene eseguita una normalizzazione per correggere gli errori di arrotondamento dei valori floating-point.

5.1.4. Phantom.h

Il modulo phantom.h si occupa della costruzione della griglia dei materiali in due modi possibili: acqua e acqua+osso.

La funzione *costruzione_phantom_acqua()* costruisce il phantom semplice composto da un cubo di $N_x \times N_y \times N_z \text{ cm}^3$ interamente composto di acqua. Questo phantom è il riferimento per la validazione fisica in quanto il PDD simulato deve riprodurre la curva caratteristica di un fascio da 6MV in acqua con il picco intorno a 3 cm e il 50% di dose intorno a 19 – 20 cm.

```

inline void costruzione_phantom_acqua(int *phantom) {
    int capienza_totale = NX * NY * NZ;
    for (int i = 0; i < capienza_totale; i++)
        phantom[i] = MATERIALE_ACQUA;
}

```

La funzione *costruzione_phantom_acquaosso()* estende il phantom omogeneo inserendo un cubo di osso al centro geometrico del volume. Il raggio dell'inserito in un'unità di voxel è stato calcolato come metà lato che sono circa 8 voxel.

```

inline void costruzione_phantom_acquaosso(int *phantom) {
    costruzione_phantom_acqua(phantom);
    int cx = NX / 2;
    int cy = NY / 2;
    int cz = NZ / 2;
    int meta_lato_inserito = (int)(2.5 / VOXEL_CM + 0.5);
    for (int iz = cz - meta_lato_inserito; iz < cz + meta_lato_inserito; iz++){
        for (int iy = cy - meta_lato_inserito; iy < cy + meta_lato_inserito; iy++){
            for (int ix = cx - meta_lato_inserito; ix < cx + meta_lato_inserito; ix++) {
                if (ix >= 0 && ix < NX && iy >= 0 && iy < NY && iz >= 0 && iz < NZ) {
                    phantom[phantom_idx(ix, iy, iz)] = MATERIALE_OSSO;
                }
            }
        }
    }
}

```

```
}
```

Il phantom eterogeneo serve a verificare la corretta gestione dell'attraversamento di interfacce tra materiali diversi. Infatti, quando un fotone passa dall'acqua all'osso il coefficiente μ cambia bruscamente perché l'osso ha un valore molto più alto a parità di energie. Deve esserci per questo un incremento locale della dose depositata seguito da un effetto di schermatura nelle regioni più profonde.

5.1.5. Output.h

Il modulo output.h contiene le funzioni di post-processing e di analisi dosimetrica. In particolare, il metodo *calcola_pdd()* descrive come la dose varia in funzione della profondità lungo l'asse del fascio, normalizzata al punto di dose massima. Per ogni slice in profondità la dose viene mediata su una finestra quadrata intorno all'asse centrale del fascio. Questa media spaziale simula la risposta integrata di una camera a ionizzazione reale.

```
inline void calcolo_pdd(const double *dose, double *pdd, double *profondita, int semiampiezza_della_media = 8) {
    int cx = NX / 2; // centro del fascio in X
    int cy = NY / 2; // centro del fascio in Y
    double max_dose = 0.0;
    for (int iz = 0; iz < NZ; iz++) {
        double valore_dose = 0.0;
        int num_voxel_sommati = 0;
        for (int ix = cx - semiampiezza_della_media; ix <= cx + semiampiezza_della_media; ix++) {
            for (int iy = cy - semiampiezza_della_media; iy <= cy + semiampiezza_della_media; iy++) {
                if (ix >= 0 && ix < NX && iy >= 0 && iy < NY) {
                    valore_dose += dose[phantom_idx(ix, iy, iz)];
                    num_voxel_sommati++;
                }
            }
        }
        if (num_voxel_sommati > 0) {
            pdd[iz] = valore_dose / num_voxel_sommati;
        } else {
            0.0;
        }
        profondita[iz] = (iz + 0.5) * VOXEL_CM;
        if (pdd[iz] > max_dose) {
            max_dose = pdd[iz];
        }
    }
    if (max_dose > 0.0)
        for (int iz = 0; iz < NZ; iz++)
            pdd[iz] = pdd[iz] / max_dose * 100.0;
}
```

La funzione *calcolo_profilo_laterale()*, invece, calcola il profilo di dose trasversale a una griglia di profondità fissa. Per ogni posizione lungo X, la dose viene mediata su ± 2 voxel attorno al centro in Y. Questo profilo permette di verificare la simmetria del fascio e la larghezza della regione piatta al centro del campo.

```

inline void calcolo_profilo_laterale(const double *dose, double *profilo_dose_relativa, double *coordinate_cm, double
profondita_scelta = 10.0, int semiampiezza_media = 2) {
    int iz = std::min((int)(profondita_scelta / VOXEL_CM), NZ - 1);
    int centro_asse_y = NY / 2;
    double dose_massima_profilo = 0.0;
    for (int ix = 0; ix < NX; ix++) {
        double somma_dose_accumulata = 0.0;
        int conteggio_voxel_validi = 0;
        for (int iy = centro_asse_y - semiampiezza_media; iy <= centro_asse_y + semiampiezza_media; iy++) {
            if (iy >= 0 && iy < NY) {
                somma_dose_accumulata += dose[phantom_idx(ix, iy, iz)];
                conteggio_voxel_validi++;
            }
        }
        if (conteggio_voxel_validi > 0){
            profilo_dose_relativa[ix] = somma_dose_accumulata / conteggio_voxel_validi;
        } else{
            profilo_dose_relativa[ix] = 0.0;
        }
        coordinate_cm[ix] = (ix + 0.5) * VOXEL_CM - PHANTOM_CM / 2.0;
        if (profilo_dose_relativa[ix] > dose_massima_profilo) {
            dose_massima_profilo = profilo_dose_relativa[ix];
        }
    }
    if (dose_massima_profilo > 0.0)
        for (int ix = 0; ix < NX; ix++)
            profilo_dose_relativa[ix] = profilo_dose_relativa[ix] / dose_massima_profilo * 100.0;
}

```

Il modulo contiene inoltre le funzioni di esportazione che salvano i risultati in CSV per effettuare poi i test fisici di validazione e in formato binario per avere la griglia 3D completa. Inoltre, la funzione *stampa_statistiche_dose()* esegue un riepilogo delle statistiche computazionali chiave, tra cui il throughput, l'energia depositata e i voxel con dose positiva.

5.1.6. Main.cpp

Il file *main.cpp* rappresenta il programma principale che gestisce l'intera simulazione. Accetta tre parametri opzionali da riga di comando che sono il numero di fotoni, il tipo di phantom e il seed del PRNG. La struttura dati fondamentale è quella del fotone che contiene lo stato completo di un fotone, ovvero la posizione in centimetri, il versore di direzione normalizzato e l'energia in MeV. La funzione *genera_fotone_iniziale()* genera ogni fotone iniziale con posizione campionata uniformemente e con direzione parallela all'asse Z ed energia estratta dallo spettro a 6MV.

```

inline Fotone genera_fotone_iniziale(const Spettro &spettro, Xoshiro256 &rng) {
    double cx = PHANTOM_CM / 2.0;
    double cy = PHANTOM_CM / 2.0;
    Fotone p;
    p.x = cx + (rng() * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
    p.y = cy + (rng() * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
    p.z = 1.0e-7;
}

```

```

p.ux = 0.0;
p.uy = 0.0;
p.uz = 1.0;
p.energia = spettro.campiona_energia(rng);
return p;
}

```

La funzione *distanza_limite_voxel()* calcola la distanza che il fotone deve percorrere nella direzione corrente prima di raggiungere il confine del voxel corrente. Lo fa indipendentemente per ciascuno dei tre assi. Per ogni componente non nulla della direzione in cui è trova il piano del voxel che interseca la traiettoria e calcola la distanza parametrica. Il minimo delle tre distanze è considerato come la distanza al confine.

```

inline double distanza_limite_voxel(double x, double y, double z, double ux, double uy, double uz, int ix, int iy, int iz) {
    double distanza_minima_confine = 1.0e30; // inizializzata a infinito
    if (std::fabs(ux) > 1.0e-12) {
        double confine_voxel_X;
        if (ux > 0){
            confine_voxel_X = (ix + 1) * VOXEL_CM;
        } else{
            confine_voxel_X = ix * VOXEL_CM;
        }
        double distanza_lineare = (confine_voxel_X - x) / ux;
        if (distanza_lineare > 1.0e-10){
            distanza_minima_confine = std::min(distanza_minima_confine, distanza_lineare);
        }
    }
    if (std::fabs(uy) > 1.0e-12) {
        double confine_voxel_Y;
        if (uy > 0){
            confine_voxel_Y = (iy + 1) * VOXEL_CM;
        } else{
            confine_voxel_Y = iy * VOXEL_CM;
        }
        double distanza_lineare = (confine_voxel_Y - y) / uy;
        if (distanza_lineare > 1.0e-10){
            distanza_minima_confine = std::min(distanza_minima_confine, distanza_lineare);
        }
    }
    if (std::fabs(uz) > 1.0e-12) {
        double confine_voxel_Z;
        if (uz > 0){
            confine_voxel_Z = (iz + 1) * VOXEL_CM;
        } else{
            confine_voxel_Z = iz * VOXEL_CM;
        }
        double distanza_lineare = (confine_voxel_Z - z) / uz;
        if (distanza_lineare > 1.0e-10) {
            distanza_minima_confine = std::min(distanza_minima_confine, distanza_lineare);
        }
    }
}

```

```

}
return distanza_minima_confine;
}

```

Nel metodo *trasporto_fotoni()* il trasporto segue la legge di Beer Lambert per il campionamento del cammino libero medio. La distanza di interazione è data da $\lambda = -\ln(\xi)/\mu$ e se $\lambda \leq \text{distanza_fisica}$ il fotone interagisce nel voxel corrente altrimenti viene traslato oltre il confine con un $\varepsilon = 10^{-7}$ cm per evitare che rimanga esattamente sul confine.

```

void trasporto_fotoni(Fotone fotone_iniziale, const int *phantom, double *dose, Xoshiro256 &rng) {
    Fotone stack[64];
    int num_particelle_stack = 0;
    stack[num_particelle_stack++] = fotone_iniziale;
    while (num_particelle_stack > 0) {
        ...
        double distanza_teorica = -std::log(xi) / mu;
        double distanza_fisica = distanza_limite_voxel(...);
        if (distanza_teorica <= distanza_fisica) {
            // Sposta la particella al punto di interazione
            ...
        } else {
            double eps = 1.0e-7;
            particella_corrente.x += particella_corrente.ux * (distanza_fisica + eps);
            particella_corrente.y += particella_corrente.uy * (distanza_fisica + eps);
            particella_corrente.z += particella_corrente.uz * (distanza_fisica + eps);
        }
    }
}
}
}

```

In questo metodo vengono gestiti i tre processi fisici:

- Effetto fotoelettrico in cui il fotone viene completamente assorbito e tutta la sua energia è depositata nel voxel corrente per cui non vengono generati fotoni secondari;

```

if (tipo_interazione == 0) {
    dose[id_voxel] += particella_corrente.energia;
    break;
}

```

- Interazione compton in cui il fotone continua con la nuova energia e la nuova direzione. Se l'energia residua scende sotto ECUT allora anche questa viene depositata localmente.

```

else if (tipo_interazione == 1) {
    double cos_theta;
    double energia_scatter;
    estrazione_compton(particella_corrente.energia, rng, cos_theta, energia_scatter);
    double energia_ceduta = particella_corrente.energia - energia_scatter;
    if (energia_ceduta > 0.0) {
        dose[id_voxel] += energia_ceduta;
    }
}

```

```

    particella_corrente.energia = energia_scatter;
    double phi = 2.0 * PI * rng();
    aggiornamento_traiettoria(particella_corrente.ux, particella_corrente.uy, particella_corrente.uz, cos_theta,
    phi);
    if (particella_corrente.energia < ECUT) {
        dose[id_voxel] += particella_corrente.energia;
        break;
    }
}

```

- Produzione di coppie in cui il fotone viene assorbito creando una coppia elettrone-positrone.

```

else {
    double energia_cinetica_residua = particella_corrente.energia - 2.0 * ME_C2;
    if (energia_cinetica_residua > 0.0) {
        dose[id_voxel] += energia_cinetica_residua;
    }
    if (ME_C2 > ECUT && num_particelle_stack + 2 <= 62) {
        double cos_theta = 2.0 * rng() - 1.0;
        double phi_a = 2.0 * PI * rng();
        double sen_theta = std::sqrt(std::max(0.0, 1.0 - cos_theta * cos_theta));
        Fotone fotone_secondario_1, fotone_secondario_2;
        fotone_secondario_1.x = fotone_secondario_2.x = particella_corrente.x;
        fotone_secondario_1.y = fotone_secondario_2.y = particella_corrente.y;
        fotone_secondario_1.z = fotone_secondario_2.z = particella_corrente.z;
        fotone_secondario_1.ux = sen_theta * std::cos(phi_a);
        fotone_secondario_1.uy = sen_theta * std::sin(phi_a);
        fotone_secondario_1.uz = cos_theta;
        fotone_secondario_2.ux = -fotone_secondario_1.ux;
        fotone_secondario_2.uy = -fotone_secondario_1.uy;
        fotone_secondario_2.uz = -fotone_secondario_1.uz;
        fotone_secondario_1.energia = fotone_secondario_2.energia = ME_C2;
        stack[num_particelle_stack++] = fotone_secondario_1;
        stack[num_particelle_stack++] = fotone_secondario_2;
    }
    break;
}

```

La gestione delle particelle secondarie usa uno stack esplicito di dimensione fissa. La scelta di avere uno stack locale, invece di avere un’allocazione dinamica serve per evitare malloc e free nel loop critico e per mantenere i dati vicini ai registri dove i tempi di accesso sono minimi. La dimensione scelta è 64 che è una dimensione sufficiente a contenere le storie dei fotoni da 6MV.

5.1.7. BeerLambert.cpp

Il file *BeerLambert.cpp* implementa una versione semplificata del simulatore *main.cpp* con lo scopo di verificare che il campionamento del cammino libero medio sia corretto. Nel trasporto semplificato il fotone percorre la distanza di $\lambda = -\ln(\xi)/\mu$ e al primo punto di interazione deposita tutta la sua energia senza simulare la natura del processo e ne generare fotoni secondari. In questo contesto ogni fotone

contribuisce esattamente una volta al suo punto di prima interazione. Il profilo di dose lungo l'asse Z, in un mezzo omogeneo, infatti, deve seguire la legge di Beer Lambert

$$D(Z) \propto e^{(-\mu z)}$$

Se il PDD prodotto segue questo andamento esponenziale il campionamento del cammino libero medio è corretto. La differenza rispetto al *main.cpp* è nel metodo di trasporto dei fotoni che è:

```
void trasporto_fotoni(Fotone p, const int *phantom, double *dose, Xoshiro256 &rng) {
    while (p.energia > ECUT && verifica_confini(p.x, p.y, p.z)) {
        int mat = phantom[phantom_idx((int)(p.x/VOXEL_CM), (int)(p.y/VOXEL_CM), (int)(p.z/VOXEL_CM))];
        double mu = calcolo_attenuazione_totale(p.energia, mat);
        double d = -std::log(rng()) / mu;
        p.x += p.ux * d;
        p.y += p.uy * d;
        p.z += p.uz * d;
        if (verifica_confini(p.x, p.y, p.z)) {
            int ix = (int)(p.x / VOXEL_CM);
            int iy = (int)(p.y / VOXEL_CM);
            int iz = (int)(p.z / VOXEL_CM);

            dose[phantom_idx(ix, iy, iz)] += p.energia;
            break;
        }
    }
}
```

5.1.8. Conclusioni

La versione sequenziale è stata validata per cui è fisicamente coerente ma non sfrutta nessun tipo di parallelismo. I punti critici principali sono rappresentati dal fatto che ogni storia del fotone è completamente indipendente dalle altre per cui si possono parallelizzare tra di loro. Si deve tenere però in considerazione che, con molti thread che scrivono sugli stessi voxel simultaneamente si generano delle race condition. I fotoni hanno, inoltre, storie di lunghezza molto variabile causando anche possibili divergenze nei thread.

5.2. Versione CPU parallela

La versione CPU parallela cerca di mitigare il fatto che ogni storia del fotone è completamente indipendente dalle altre. Non ci sono dipendenze di dati tra particelle diverse durante il trasporto. Un fotone non legge mai lo stato di un altro fotone e l'unica risorsa condivisa è la griglia di dose su cui si accumula il risultato finale.

In questa versione è stato utilizzato *std::thread* che è la libreria di threading standard di C++. In questo modo si ha un controllo esplicito su come vengono creati i thread, come viene distribuito il lavoro e come si gestisce la fase di riduzione finale. I diversi moduli fisici sono uguali a quelli della versione sequenziale CPU per cui tutta la logica di parallelizzazione è presente nel file *main.cpp* e nella versione di test semplificata *BeerLambert.cpp*.

5.2.1. Main.cpp

Nel codice sequenziale ogni fotone esegue $dose[id_voxel] += energia_ceduta$. Con N thread che eseguono simultaneamente questa operazione sullo stesso array si genera una race conditio. Se, infatti, due thread leggono il valore corrente del voxel, lo incrementano ciascuno con la propria quota di energia e poi lo riscrivono, uno dei due incrementi viene perso. Per risolvere il problema ogni thread accumula la propria dose su un array privato e alla fine lo somma all'array globale in una sezione critica. In questo modo non si ha nessuna contesa durante il calcolo e si ha un'unica sincronizzazione alla fine. Il costo che si ha è dovuto alla memoria aggiuntiva, in particolare si hanno:

$$N \text{ thread} * 10^6 \text{ voxel} * 8 \text{ byte} = N * 8 \text{ MB}$$

Per fare questo è stata definita una funzione *worker()* in cui ogni thread alloca il proprio vettore di dose privato e lo inizializza a zero. Tutta la fisica del problema viene eseguita scrivendo su questo buffer locale senza mai toccare la griglia globale durante il calcolo.

```
void worker(const ConfigurazioneWorker& cfg) {
    uint64_t seed_thread = cfg.seed_casuale + (uint64_t)cfg.id_thread * 2654435761ULL;
    Xoshiro256 generatore(seed_thread);
    Spettro spettro;
    std::vector<double> dose_locale(NX * NY * NZ, 0.0);
    for (long long i = cfg.id_thread; i < cfg.numero_fotoni; i += cfg.numero_thread) {
        Fotone fotone = genera_fotone_iniziale(spettro, generatore);
        trasporto_fotoni(fotone, cfg.phantom, dose_locale.data(), generatore);
        ...
    }
}
```

La funzione *trasporto_fotoni()* è uguale alla versione sequenziale, infatti, riceve un puntatore e ci scrive i valori sopra, sia esso un buffer globale o un buffer locale. Quando un thread ha completato tutti i suoi fotoni somma il buffer locale al buffer globale in una sezione critica protetta da mutua esclusione. Il *mutex* è dichiarato static all'interno della funzione. In questo modo esiste un'unica istanza condivisa tra tutte le invocazioni del worker senza doverle passare come parametro.

```
static std::mutex mutex_riduzione;
{
    std::lock_guard<std::mutex> lock(mutex_riduzione);
    for (int k = 0; k < NX * NY * NZ; k++)
        cfg.dose_globale[k] += dose_locale[k];
}
```

La riduzione avviene una sola volta per thread, alla fine, e il ciclo che somma i 10^6 elementi è sequenziale ma veloce. Il mutex, in questo modo viene tenuto per poco tempo, in proporzione alla durata totale del lavoro dei thread. L'uso di *std::lock_guard* permette di rilasciare automaticamente il mutex quando esce dallo scope, anche in caso di eccezione, evitando possibili deadlock.

Un modo di assegnare i fotoni ai thread è quello di assegnare dei blocchi contigui. Tuttavia, in questo modo, se i fotoni nei primi blocchi sono casualmente più veloci o più lenti i thread finiscono in tempi diversi per cui alcuni aspettano gli altri inattivi. È stata utilizzata una tecnica di round robin in cui il thread con id t simula i fotoni con indice $t, t + T, t + 2T, \dots, t + (N - 1)T$. In questo modo si distribuiscono le

variazioni di lunghezza delle storie tra tutti i thread bilanciando il carico in modo naturale senza bisogno di un task scheduler dinamico.

```
for (long long i = cfg.id_thread; i < cfg.numero_fotoni; i += cfg.numero_thread) {  
    Fotone fotone = genera_fotone_iniziale(spettro, generatore);  
    trasporto_fotoni(fotone, cfg.phantom, dose_locale.data(), generatore);  
    ...  
}
```

Un aspetto critico della parallelizzazione di simulazioni Monte Carlo è la qualità statistica dei numeri casuali generati da thread diversi. Se tutti i thread partissero dallo stesso seed produrrebbero sequenze identiche, simulando di fatto gli stessi fotoni N volte invece di N fotoni diversi, il risultato sarebbe sbagliato e la varianza non scenderebbe come $1/\sqrt{N}$. Ogni thread deve ricevere un seed derivato dal seed globale combinato con il proprio id:

```
uint64_t seed_thread = cfg.seed_casuale + (uint64_t)cfg.id_thread * 2654435761ULL;  
Xoshiro256 generatore(seed_thread);
```

Dove la costante 2654435761ULL è la rappresentazione a 32 bit della sezione aurea che garantisce che seed adiacenti producano stati iniziali molto diversi tra di loro. Ogni thread istanzia anche il loro spettro locale così da evitare accessi in lettura ad un oggetto condiviso.

Nella struttura di configurazione del worker i campi relativi al phantom e alla dose globale sono dei puntatori condivisi tra tutti i thread. Il phantom è in sola lettura durante la simulazione per cui non serve alcuna sincronizzazione per accedervi mentre la dose globale è in scrittura ma solamente nella fase di riduzione finale gestita dal mutex.

I thread vengono creati con il comando `std::thread` e memorizzati in un vettore. La chiamata `threads.emplace_back(worker, cfg)` crea un nuovo thread che esegue il worker e lo aggiunge al vettore. Il numero di thread viene determinato a runtime così da adattarsi al numero di core logici disponibili, nonostante possa anche essere ridefinito da riga di comando.

```
int num_thread = (int)std::thread::hardware_concurrency();  
if (num_thread < 1)  
    num_thread = 1;  
  
std::vector<std::thread> threads;  
threads.reserve(num_thread);  
...  
for (auto &th : threads)  
    th.join();
```

In questa implementazione la fase di join è bloccante. Il thread main, infatti, attende che tutti i worker completino il calcolo della riduzione, e solo dopo la griglia contiene il risultato completo e può essere processata.

Questa implementazione garantisce risultati uguali alla versione sequenziale. I valori non sono uguali bit per bit perché i thread elaborano fotoni in ordine diverso e usano seed diversi ma convergono agli stessi valori fisici entro l'incertezza statistica attesa. Questo è garantito dal fatto che la fisica di ogni

fotone è identica, i generatori di ciascun thread sono statisticamente indipendenti tra di loro e la dose locale viene sommata linearmente alla dose globale.

5.2.2. BeerLambert.cpp

La versione di test semplificata BeerLambert.cpp utilizza la stessa infrastruttura del file main.cpp parallelo con la stessa configurazione del worker e la stessa funzione *worker()*. L'unica differenza sta nel trasporto fotoni semplificato senza scattering per validare la legge di Beer Lambert

```
void trasporto_fotoni(Fotone p, const int *phantom, double *dose, Xoshiro256 &rng) {
    while (p.energia > ECUt && verifica_confini(p.x, p.y, p.z)) {
        int mat = phantom[phantom_idx((int)(p.x / VOXEL_CM), (int)(p.y / VOXEL_CM), (int)(p.z / VOXEL_CM))];
        double mu = calcolo_attenuazione_totale(p.energia, mat);
        double d = -std::log(rng()) / mu;
        p.x += p.ux * d;
        p.y += p.uy * d;
        p.z += p.uz * d;
        if (verifica_confini(p.x, p.y, p.z)) {
            int ix = (int)(p.x / VOXEL_CM);
            int iy = (int)(p.y / VOXEL_CM);
            int iz = (int)(p.z / VOXEL_CM);
            dose[phantom_idx(ix, iy, iz)] += p.energia;
            break;
        }
    }
}
```

Con questa versione si può validare la fisica su un numero maggiore di fotoni in tempi più accessibili, rendendo maggiormente evidenti, eventuali deviazioni rispetto all'andamento esponenziale teorico.

5.3. Versione base GPU

La versione base GPU rappresenta il primo porting del codice su architettura CUDA. Non sono state ottimizzate al massimo le prestazioni ma è stata tradotta la versione sequenziale in un kernel GPU.

5.3.1. Caratteristiche GPU

La GPU utilizzata è la NVIDIA Tesla T4 di Google Colab, le cui caratteristiche sono:

Caratteristica	Valore
Architettura	Turing SM
CUDA Cores	2560
Memoria Globale	15360 MiB
Memoria shared per SM	64 KB
Warp size	32 thread
Max thread per blocco	1024
Driver CUDA	12.8

Tabella 3 - Caratteristiche GPU

L'architettura Turing Sm 7.5 presenta atomicAdd su valori direttamente in hardware, non viene emulata tramite software, accelerando il tempo di esecuzione. Questa caratteristica consente di poter fare un accumulo diretto della dose senza costi eccessivi. Dal punto di vista della capacità di memoria, i 15 GB

disponibili sono sufficienti in quanto il phantom utilizzato di $100 \times 100 \times 100$ interi occupa circa 4 MB mentre la griglia di dose di double ne occupa circa 8MB. Entrambe le strutture dati vengono tenute memorizzate in memoria globale per tutta la durata della simulazione, senza essere trasferite o suddivise.

5.3.2. Kernel CUDA

In questo problema ogni storia di fotone è completamente indipendente dalle altre. Il fotone accede al phantom in sola lettura, e scrive, tramite scrittura condivisa, sulla griglia di dose ma non interagisce mai con gli altri fotoni durante il trasporto. Questa caratteristica rende il problema embarrassingly parallel per cui si può distribuire su un numero arbitrario di unità di calcolo indipendenti.

Nella versione CPU parallela questo si esegue assegnando a ciascun thread un sottoinsieme dei fotoni con ciascun thread che accumula la propria dose locale e poi la somma alla griglia globale in una sezione critica. Sulla GPU si lancia un thread per fotone e ogni thread gestisce l'intera vita di una particella dalla sorgente all'assorbimento. In questo modo con un milione di fotoni si lanciano un milione di thread GPU che sulla GPU Tesla T4 vengono distribuiti su 2560 CUDA core organizzati a warp da 32.

```
const int DIMENSIONE_BLOCCO = 256;
long long numero_blocchi = (num_fotoni + DIMENSIONE_BLOCCO - 1) / DIMENSIONE_BLOCCO;
mc_kernel<<<(int)numero_blocchi, DIMENSIONE_BLOCCO>>>(num_fotoni, device_panthom, device_dose, seed);
```

L'utilizzo di 256 thread per blocco consente di bilanciare l'occupancy del SM permettendo di tenere attivi più warp contemporaneamente per nascondere le latenze di memoria, con la pressione sui registri. Infatti, ogni thread ha il proprio stato fotone da tenere in vita e che consuma registri. Con 256 thread per blocco e lo stato necessario per ogni fotone, il compilatore riesce a mantenere un buon livello di occupancy senza che i registri si riversino in local memory.

Il kernel contiene immediatamente un controllo di guardia che fa terminare in thread in eccesso rispetto al numero di fotoni richiesti poiché il numero totale di thread lanciati è arrotondato al multiplo del block size più vicino:

```
__global__ void mc_kernel( long long num_fotoni, const int *phantom, double *dose, uint64_t seed_base) {
    long long indice_thread = (long long)blockIdx.x * blockDim.x + threadIdx.x;
    if (indice_thread >= num_fotoni)
        return;
    curandStatePhilox4_32_10_t stato_casuale;
    curand_init(seed_base, (unsigned long long)indice_thread, 0, &stato_casuale);
    Fotone p = genera_fotone_iniziale(&stato_casuale);
    trasporto_fotoni(p, phantom, dose, &stato_casuale);
}
```

5.3.3. Struttura dei moduli

I moduli sono stati convertiti tenendo conto che sulla GPU gira solamente ciò che viene chiamato all'interno del kernel di trasporto mentre tutto il resto rimane sulla CPU. Per questo motivo i moduli *phantom.cuh* e *output.cuh* contengono le funzioni qualificate con inline senza decoratori CUDA. Queste operazioni avvengono, infatti, una sola volta, prima e dopo la simulazione e il loro costo è trascurabile rispetto al calcolo principale.

I moduli *physics.cuh* e *compton.cuh* invece contengono delle funzioni qualificate con `__device__` poiché vengono chiamate da dentro il kernel e compilate per girare sulla GPU. La qualifica di `__device__` indica proprio al compilatore che queste funzioni esistono solo nel contesto GPU e possono essere chiamate da altri device o da kernel qualificati con `__global__`. Alcune funzioni di utilità come *phantom_idx*, *verifica_confini* e *vox* sono qualificate come `__host__ __device__` perché devono funzionare sia lato CPU durante la costruzione del phantom che lato GPU nel kernel del trasporto per cui ne vengono generate due versioni.

5.3.4. Gestione della memoria

Nella versione CPU sequenziale le tabelle di sezione d'urto erano dichiarate come *static const* e il compilatore le teneva in memoria read only con caching automatico. Sulla GPU esiste il tipo di memoria dedicata a questo scopo che è la constant memory che è una zona di memoria globale della GPU ad accesso in sola lettura e ottimizzata per il caso in cui tutti i thread leggano lo stesso indirizzo contemporaneamente. Le tabelle sono quindi dichiarate come:

```
static const int PUNTI_CAMPIONAMENTO = 28;
__constant__ double GRIGLIA_ENERGIA[PUNTI_CAMPIONAMENTO]
__constant__ double PROBABILITA_TOTALE[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO]
__constant__ double PROBABILITA_FOTOELETTRICO[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO]
__constant__ double PROBABILITA_COMPTON[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO]
__constant__ double PROBABILITA_COPPIA[NUMERO_MATERIALI][PUNTI_CAMPIONAMENTO]
```

In questo modo durante il trasporto di un fotone ogni thread chiama il metodo *interpolazione_lineare_energia()* che legge la stessa griglia energetica e la stessa tabella di sezioni d'urto. Poiché tutti i thread di un warp tendono ad avere fotoni con energie diverse, e quindi accedono a indici diversi della griglia, il pattern di accesso non è adatto ad un broadcast puro ma la constant memory è in una L1 cache dedicata per cui la ricerca binaria su 28 punti genera al massimo 5 accessi distinti alla cache per ogni interpolazione. Questo è molto più efficiente del classico accesso alla memoria globale non in cache. Lo stesso viene fatto con la CDF dello spettro energetico. Viene infatti copiata in constant memory prima del lancio del kernel:

```
double host_distribuzione_cumulata[NUMERO_BINS_SPETTRO];
calcola_FLUENZA(host_distribuzione_cumulata);
CUDA_CHECK(cudaMemcpyToSymbol(FLUENZA, host_distribuzione_cumulata, NUMERO_BINS_SPETTRO *
sizeof(double)));
```

La funzione *cudaMemcpyToSymbol* è proprio la chiamata specifica per trasferire i dati nella constant memory a partire dall'host.

5.3.5. Generazione numeri casuali

Nella versione CPU il generatore di numeri casuali utilizzato è Xoshiro256 che è un PRNG a stato a 256 bit con periodo di $2^{256} - 1$. Sulla GPU questo meccanismo non si può utilizzare. Ogni thread deve avere il proprio stato RNG indipendente e con un milione o più di thread mantenere altrettante istanze di Xoshiro256 richiederebbe $4 \times 8 \text{ byte} \times 1000000 = 32 \text{ MB}$ il che rappresenta un peso significativo per i registri e la local memory.

La soluzione utilizzata è quella di utilizzare la libreria cuRAND di NVIDIA che è specificatamente scritta per la generazione di numeri casuali su GPU. L'algoritmo scelto è il Philox 4×32 che è un counter-based PRNG che invece di mantenere uno stato che viene modificato ad ogni chiamata prende in input un counter e una chiave e calcola il numero casuale con una funzione invertibile. Lo stato è compatto, richiede circa 64 byte per thread, mentre l'inizializzazione avviene come:

```
curandStatePhilox4_32_10_t stato_casuale;  
curand_init(seed_base, (unsigned long long)indice_thread, 0, &stato_casuale);
```

Passare il parametro *indice_thread* come sequence garantisce che ogni thread parta da un punto della sequenza matematicamente distante da tutti gli altri, senza alcuna correlazione, offre la garanzia formale di indipendenza statistica offerta da cuRAND. Come nella versione sequenziale viene escluso il valore 1.0 per evitare che $-\log(1.0) = 0$ produca passi nulli:

```
__device__ inline double genera_rng(curandStatePhilox4_32_10_t *stato) {  
    double valore;  
    do {  
        valore = curand_uniform_double(stato);  
    } while (valore >= 1.0);  
    return valore;  
}
```

5.3.6. AtomicAdd su double

Nella versione CPU multi-thread per la gestione della scrittura sulla griglia di dose è stato utilizzato un buffer locale per thread con riduzione finale sotto mutex. Ogni thread scrive liberamente sul proprio array privato e sincronizza solo una volta alla fine. Questo funziona perché il numero di thread CPU è piccolo e il costo di allocare e ridurre un array da 8 MB per thread era accettabile. Sulla GPU con milioni di thread allocare milioni di copie della griglia di dose è impossibile. Per questo si accumula direttamente sulla griglia condivisa utilizzando le operazioni atomiche, in particolare:

```
atomicAdd(&dose[id], energia_ceduta);
```

L'operazione atomica garantisce che due thread che tentano di aggiornare lo stesso voxel contemporaneamente non si sovrascrivano a vicenda, a costo di una serializzazione hardware. La contesa sulle operazioni atomiche è il principale fattore limitante delle prestazioni, infatti, fotoni che attraversano la stessa regione del phantom, tipicamente il centro del fascio dove la dose è più alta, generano molte scritture concorrenti sugli stessi voxel.

5.3.7. Gestione delle particelle secondarie

La gestione delle particelle secondarie generate dalla produzione di coppie riproduce la stessa struttura della versione CPU; infatti, si ha uno stack esplicito di dimensione fissa allocato localmente per ogni thread. Nel contesto CUDA questa allocazione avviene in local memory che è fisicamente la memoria globale della GPU ma è privata per ogni thread e può andare in cache in L1.

```
Fotone stack[64];  
int num_particelle_stack = 0;  
stack[num_particelle_stack++] = fotone_iniziale;  
while (num_particelle_stack > 0) {  
    Fotone particella_corrente = stack[--num_particelle_stack];
```

...

La dimensione di 64 è dovuta al fatto che un fotone da 6MV non può generare una cascata molto profonda di coppie consecutive in quanto la produzione di coppie richiede $E > 1.022 \text{ MeV}$ e ogni coppia genera due fotoni da 0.511 MeV che sono sotto la soglia per generare nuove coppie, per cui lo stack in pratica non super mai i 3/4 elementi.

Se il compilatore riesce ad allocare lo stack nei registri del thread il costo è minimo, e questo accade quando gli accessi agli elementi è con indici costanti o facilmente analizzabili. Se invece lo stack viene riversato in local memory perché i registri sono insufficienti gli accessi diventano equivalenti a quelli sulla memoria globale ma con latenza mascherata dalla pipeline. In entrambi i casi, utilizzando fasci da 6MV non si creano dei colli di bottiglia.

5.3.8. Misurazione del tempo

Per separare il tempo di trasferimento dati dal tempo di calcolo vero e proprio sono stati utilizzati i CUDA Events invece dei timer lato CPU. I timer CPU misurano il tempo percepito dal main thread che include latenze di scheduling e attese implicite; i CUDA Events invece sono timestamp inseriti nella coda di comandi della GPU e misurati con il clock interno del device che ha risoluzione al microsecondo:

```
cudaEvent_t inizio_copia, fine_copia, inizio_kernel, fine_kernel, inizio_totale, fine_totale;
cudaEventCreate(&inizio_copia); cudaEventCreate(&fine_copia);
cudaEventCreate(&inizio_kernel); cudaEventCreate(&fine_kernel);
cudaEventCreate(&inizio_totale); cudaEventCreate(&fine_totale);
cudaEventRecord(inizio_copia);
CUDA_CHECK(cudaMemcpy(device_phantom, host_phantom, NX * NY * NZ * sizeof(int), cudaMemcpyHostToDevice));
cudaEventRecord(fine_copia);
cudaEventSynchronize(fine_copia);
...
cudaEventElapsedTime(&ms_copia_a_gpu, inizio_copia, fine_copia);
cudaEventElapsedTime(&ms_calcolo_gpu, inizio_kernel, fine_kernel);
cudaEventElapsedTime(&ms_copia_da_gpu, inizio_totale, fine_totale);
double tempo_calcolo_secondi = ms_calcolo_gpu / 1000.0;
double tempo_totale_secondi = (ms_copia_a_gpu + ms_calcolo_gpu + ms_copia_da_gpu) / 1000.0;
cudaEventDestroy(inizio_copia); cudaEventDestroy(fine_copia);
cudaEventDestroy(inizio_kernel); cudaEventDestroy(fine_kernel);
cudaEventDestroy(inizio_totale); cudaEventDestroy(fine_totale);
```

Questa suddivisione permette di distinguere i tre contributi fondamentali al tempo totale che sono:

- Host to Device, in particolare nel tempo di copia del phantom della RAM della CPU alla memoria GPU;
- Il tempo del kernel, ovvero il tempo di simulazione vero e proprio che dipende dal numero di fotoni e dalla complessità delle storie;
- Device to Host, in particolare il tempo di copia della griglia di dose dalla GPU alla RAM.

I valori vengono scritti, come nel caso della CPU, in un file di log testuale per essere poi confrontati con le altre implementazioni.

5.3.9. Versione Beer Lambert

La versione semplificata BeerLambert.cu implementa il trasporto di fotoni senza lo scattering. Ogni fotone viene trasportato in linea retta e il primo punto di interazione viene campionato con la legge di Beer Lambert e li viene depositata tutta l'energia. In questa implementazione non viene generata nessuna particella secondaria e non viene aggiornata la direzione:

```
__global__ void mc_beer_lambert_kernel( long long num_fotoni, const int *phantom, double *dose, uint64_t seed_base){
    long long indice_thread = (long long)blockIdx.x * blockDim.x + threadIdx.x;
    if (indice_thread >= num_fotoni)
        return;
    curandStatePhilox4_32_10_t stato_casuale;
    curand_init(seed_base, (unsigned long long)indice_thread, 0, &stato_casuale);
    double cx = PHANTOM_CM / 2.0;
    double cy = PHANTOM_CM / 2.0;
    Fotone p;
    p.x = cx + (curand_uniform_double(&stato_casuale) * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
    p.y = cy + (curand_uniform_double(&stato_casuale) * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
    p.z = 1.0e-7;
    p.ux = 0.0; p.uy = 0.0; p.uz = 1.0;
    p.energia = campiona_energia(&stato_casuale);
    while (p.energia > ECUT && verifica_confini(p.x, p.y, p.z)) {
        int materiale = phantom[phantom_idx(vox(p.x), vox(p.y), vox(p.z))];
        double mu = calcolo_attenuazione_totale(p.energia, materiale);
        double distanza_percorsa = -log(genera_rng(&stato_casuale)) / mu;
        p.x += p.ux * distanza_percorsa;
        p.y += p.uy * distanza_percorsa;
        p.z += p.uz * distanza_percorsa;
        if (verifica_confini(p.x, p.y, p.z)) {
            int id = phantom_idx(vox(p.x), vox(p.y), vox(p.z));
            atomicAdd(&dose[id], p.energia);
            break;
        }
    }
}
```

Dal punto di vista dell'implementazione CUDA in questo kernel non si ha nessuno stack, nessun branch per selezionare il tipo di interazione e nessun aggiornamento della direzione. Il flusso di controllo è uniforme tra tutti i thread il che riduce la warp divergence rispetto all'implementazione del kernel completo.

5.3.10. Criticità e analisi implementativa

Questa implementazione non presenta nessun tipo particolare di parallelismo ma solamente una trascrizione in una versione base del Monte Carlo per radioterapia. Tuttavia, presenta già due criticità che sono la warp divergence e la contesa sulle operazioni atomiche.

In un warp da 32 thread ogni thread simula un fotone con una storia diversa, per cui c'è chi subisce un effetto fotoelettrico e quindi termina subito e c'è chi subisce uno scattering Compton e continua. Questo significa che i thread dello stesso warp divergono rapidamente nei loro branch e la GPU deve

eseguire ogni ramo serialmente mascherando i thread che non lo percorrono. In un problema come il Monte Carlo in cui la stocasticità è intrinseca rappresenta un problema, nonché limite, importante.

Per quanto riguarda la contesa sulle operazioni atomiche ogni volta che un thread deposita energia in un voxel può collidere con altri thread che stanno cercando di scrivere sullo stesso voxel. Questa contesa è maggiore soprattutto nelle regioni ad alta fluenza che sono il centro del campo e i primi centimetri di profondità dove la dose è massima, e aumenta con il numero di thread attivi. Questa problematica viene gestita con `atomicAdd` che consente un accesso atomico alla variabile.

Il kernel di questa versione viene lanciato con tanti thread quanti sono i fotoni e per ognuno di questi viene eseguita l'inizializzazione di `CuRand`. Per il generatore `Philox` si deve derivare lo stato iniziale della sequenza a partire dal seed e dall'indice richiedendo diversi cicli di clock ancor prima che inizi il trasporto fisico del fotone stesso.

Ogni thread riceve esattamente un fotone da simulare, tuttavia la durata della simulazione è intrinsecamente variabile perché dipende dall'energia iniziale campionata dallo spettro, dal numero e dal tipo di interazioni subite. Questa variabilità introduce uno sbilanciamento del carico tra thread dello stesso blocco riducendo di molto l'efficienza.

5.4. Versione 2 GPU

5.4.1. Considerazioni e profilazione della versione base

La versione base su GPU implementa un modello di parallelismo in cui ogni thread CUDA simula l'intera traiettoria di un singolo fotone. Per ottimizzare le prestazioni base i dati fisici invarianti sono ospitati in Constant Memory per sfruttare la cache di broadcasting, mentre la coerenza dei dati nella griglia di dose è garantita dall'uso di operazioni atomiche.

Per effettuare la profilazione del codice sono stati utilizzati `Nsight Compute` e `nvprof` messi a disposizione nelle librerie NVIDIA. Con `Nsight Compute` sono state raccolte le metriche hardware interne eseguendo il kernel diverse volte, in questo caso 10. `Nvprof`, invece, misura una singola esecuzione reale senza alterare il comportamento per ottenere le prestazioni effettive. Considerando un'esecuzione con 10 milioni di fotoni i risultati sono:

Attività	Tempo	Percentuale sul totale
<code>mc_kernel</code>	9.154 s	99.94%
<code>cudaMemcpy da device a host</code>	4.587 ms	0.05%
<code>cudaMemcpy da host a device</code>	0.761 ms	0.01%
<code>cudaMemset</code>	0.027 ms	< 0.01%
Throughput	1.092 MP/s	
Tempo per fotone	0.9 μ s	

Tabella 4 - Risultati profilazione versione base GPU

La percentuale rappresentante il tempo totale è tutta sull'esecuzione del kernel, per cui qualsiasi ottimizzazione che non tocchi il kernel non migliora di molto le prestazioni.

5.4.2. Ottimizzazione con Work Stealing

Nella versione 2 come prima ottimizzazione è stato invertito il rapporto tra i thread e il lavoro, infatti, invece di creare tanti thread per quanti sono i fotoni si crea un numero fisso di thread che rimangono

attivi per tutta la durata del kernel e si autoprogrammano il lavoro dinamicamente tramite una work queue condivisa.

```
const int DIMENSIONE_BLOCCO = 256;
const int NUMERO_BLOCCHI = 1024;
mc_kernel<<<NUMERO_BLOCCHI, DIMENSIONE_BLOCCO>>>( num_fotoni, device_phantom, device_dose,
contatori_fotoni_procesati, seed);
```

In questo modo si hanno $1024 \times 256 = 262144$ *thread* che sono indipendenti dai fotoni da simulare. Considerando che la Tesla T4 contiene 40 SM con 16 blocchi attivi per SM con questo numero di thread la GPU si satura ma non si sovraccarica lo scheduler. Riducendo drasticamente il numero di blocchi si riduce lo sforzo del warp scheduler e si maschera la latenza poiché i thread sono già in esecuzione e non devono essere continuamente creati e distrutti dall'hardware.

La work queue è implementata considerando un singolo contatore intero in memoria globale inizializzato a zero prima del lancio del kernel:

```
unsigned long long *contatori_fotoni_procesati;
```

In questo modo, il kernel di ogni thread diventa un loop in cui si acquisisce automaticamente il prossimo fotone disponibile

```
_global_ void mc_kernel(long long num_fotoni, const int *phantom, double *dose, unsigned long long
*contatori_fotoni_procesati, uint64_t seed_base){
    while (true) {
        unsigned long long id_fotone = atomicAdd(contatori_fotoni_procesati, 1ULL);
        if ((long long)id_fotone >= num_fotoni)
            break;
        curandStatePhilox4_32_10_t stato_rng;
        curand_init(seed_base, id_fotone, 0, &stato_rng);
        Fotone p = genera_fotone_iniziale(&stato_rng);
        trasporto_fotoni(p, phantom, dose, &stato_rng);
    }
}
```

atomicAdd in questo caso restituisce il valore precedente del contatore che diventa l'indice univoco del fotone assegnato a quel thread in quella interazione. Quando l'indice restituito supera il numero di fotoni la coda è esaurita e il thread esce.

In questo modo, un thread che finisce una storia corta non aspetta nessuno ma esegue immediatamente atomicAdd e prende il prossimo fotone dalla coda, mentre un thread con una storia lunga continua a lavorare in parallelo. La GPU, in questo modo, rimane uniformemente occupata per tutta la durata del kernel.

La chiamata di curand_init viene effettuata ogni volta dentro il loop, per cui viene fatta per ogni fotone. Spostarla fuori dal loop e quindi inizializzare lo stato RNG una volta sola per thread e riutilizzarlo per i fotoni successivi ridurrebbe il numero di chiamate, tuttavia, la sequenza casuale di ogni fotone dipenderebbe da quanti fotoni quel thread ha già processato per cui la distribuzione fisica di dose diventerebbe non deterministica. Dipenderebbe, infatti, da quanti fotoni quel thread ha già processato e quindi dall'ordine non deterministico di scheduling dei thread. Tenendo questa inizializzazione nel

loop ogni fotone ha sempre la stessa sequenza casuale qualunque sia il thread che lo processi e in qualunque ordine.

Il costo dell'atomicAdd sul contatore globale è trascurabile perché avviene una volta sola per fotone mentre la simulazione di un singolo fotone richiede migliaia di cicli tra campionamento, accessi al phantom e le interpolazioni fisiche nelle tabelle.

5.4.3. Versione Beer Lambert

Anche la versione semplificata Beer Lambert presenta lo stesso pattern di scheduling. Il kernel, infatti, è definito come:

```
__global__ void mc_beer_lambert_kernel( long long num_fotoni, const int *phantom, double *dose, unsigned long long
*contatore_fotoni_processati, uint64_t seed_base)
{
    while (true) {
        unsigned long long indice_thread = atomicAdd(contatore_fotoni_processati, 1ULL);
        if ((long long)indice_thread >= num_fotoni)
            break;
        curandStatePhilox4_32_10_t stato_casuale;
        curand_init(seed_base, indice_thread, 0, &stato_casuale);
        double cx = PHANTOM_CM / 2.0;
        double cy = PHANTOM_CM / 2.0;
        double x = cx + (curand_uniform_double(&stato_casuale) * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
        double y = cy + (curand_uniform_double(&stato_casuale) * 2.0 - 1.0) * SEMI_AMPIEZZA_CAMPO;
        double z = 1.0e-7;
        double ux = 0.0, uy = 0.0, uz = 1.0;
        double energia = campiona_energia(&stato_casuale);
        while (energia > ECUT && verifica_confini(x, y, z)) {
            int ix = vox(x);
            int iy = vox(y);
            int iz = vox(z);
            int materiale = phantom[phantom_idx(ix, iy, iz)];
            double mu = calcolo_attenuazione_totale(energia, materiale);
            double distanza_percorsa = -log(genera_rng(&stato_casuale)) / mu;
            x += ux * distanza_percorsa;
            y += uy * distanza_percorsa;
            z += uz * distanza_percorsa;
            if (verifica_confini(x, y, z)) {
                int id = phantom_idx(vox(x), vox(y), vox(z));
                atomicAdd(&dose[id], energia);
                break;
            }
        }
    }
}
```

Il lancio anche qui avviene con $1024 \text{ blocchi} \times 256 \text{ thread}$, tutti con lo stesso contatore atomico work queue. Anche in questa versione Beer Lambert non si ha nessuna selezione del tipo di interazione, nessuno scatter e nessuna particella secondaria. Il fotone viene trasportato in linea retta e al primo punto di interazione deposita tutta l'energia nel voxel corrispondente.

5.5. Versione 3 GPU

5.5.1. Criticità versione 2

Nell'implementazione della versione 2 rimangono ancora non risolte le principali criticità che sono:

- Warp divergence;
- Accessi non coalescenti al phantom;
- Uso eccessivo dei double.

Per risolvere il problema della warp divergence l'unico modo è quello di raggruppare i fotoni per tipo di interazione prima di eseguire il calcolo così che thread dello stesso warp percorrano lo stesso ramo del codice eliminando la differenza. Questo si potrebbe fare creando una pipeline in cui un kernel trasporta i fotoni fino al punto di interazione in cui vengono smistati in codice separare per tipo, quindi fotoelettrico, compton e coppie, e kernel specializzati elaborano ciascuna coda. Il problema è che questo approccio introduce costi che superano il beneficio. Questo perché, in questo modo, invece di tenere lo stato del fotone nei registri del thread si deve scrivere lo stato in memoria globale ogni volta che il fotone viene messo in una coda e si deve rileggere quando viene estratto dal kernel successivo. La memoria globale ha una latenza maggiore rispetto a quello dei registri per cui fare queste operazioni comporta maggior tempo rispetto all'attesa minima durante le divergenze.

Questa soluzione comporta anche una sorta di sincronizzazione tra CPU e GPU; infatti, si devono determinare quanti fotoni sono finiti in ogni coda dopo ogni step e questo richiede di leggere i contatori dalla GPU il che impone una barriera di sincronizzazione. La CPU aspetta che la GPU termini e viceversa azzerando il parallelismo. Un fotone può avere decine di interazioni per cui si creerebbe un rimbalzo tra CPU e GPU più costoso della divergenza stessa.

Questo tipo di ottimizzazione è efficace quando le code sono grandi e i kernel sono computazionalmente pesanti. Nel Monte Carlo per radioterapia i fotoni hanno un numero molto variabile di interazione e passi molto leggeri dal punto di vista computazionale per cui l'overhead di gestione delle code peggiora rispetto al guadagno di coerenza nel warp. La divergenza di questo problema introduce un peggioramento di esecuzione irrilevante per cui non viene risolta.

Un altro problema riscontrato nella versione 2 è l'utilizzo di double per tutte le variabili del kernel. La GPU Tesla T4 presenta un throughput FP64 di circa 0.25 TFLOPS mentre ha un throughput FP32 di circa 8.1 TFLOPS. Passare tutta l'esecuzione da double a float comporta una perdita di precisione per cui tutta la fisica viene invalidata. Alcune variabili è necessario che rimangano a doppia precisione come quella rappresentante l'energia del fotone, le coordinate di posizione e l'accumulo della dose. Le tabelle di sezione d'urto e di interpolazione lineare operano su valori che devono essere confrontati con soglie fisiche precise per cui errori di arrotondamento in float comporterebbero un non raggiungimento della soglia ECUT alterando la distribuzione della dose. Per quanto riguarda le coordinate, considerando che il phantom ha lato 30 cm con voxel di 3mm un valore float a 32 bit ha una precisione di 10^{-7} che copre la risoluzione dei 3mm. Tuttavia, le coordinate vengono aggiornate sommando passi successivi e l'accumulo di errori di arrotondamento su tanti passi comporta il fatto che un fotone attraversi un confine di voxel sbagliato o rimanga bloccato sul confine. Per lo stesso motivo anche l'accumulo della dose rimane in double. La dose in un voxel, infatti, è la somma di migliaia di contributi piccoli per cui ogni singolo errore di arrotondamento contribuisce a rendere meno accurata la griglia. L'approssimazione float si può utilizzare per le operazioni intermedie dove la precisione a 32 bit è sufficiente e non a perdere di accuratezza.

Il terzo problema riscontrato è quello degli accessi non coalescenti. Nella versione 2 il phantom viene memorizzato come un array di interi in memoria globale. Affinchè la GPU raggruppi 32 letture di un warp in un'unica transazione di memoria i 32 thread devono leggere 32 indirizzi contigui e allineati. In un array lineare questo avviene quando thread con indici consecutivi leggono elementi consecutivi. Nel Monte Carlo per radioterapia questo non accade quasi mai. Ogni thread simula un fotone in una posizione arbitraria del phantom e dopo anche pochi assi di scattering compton i fotoni di thread adiacenti si trovano in posizione di volume completamente scorrelate. Quando 32 thread leggono il materiale del loro voxel corrente gli indirizzi nella griglia del phantom sono in posizioni casuali, per cui la GPU esegue fino a 32 transazioni di memoria separate invece di una sola. Una soluzione potrebbe essere quella di riorganizzare i fotoni in modo tale che thread vicini siano spazialmente vicini nel phantom ma questo, come nel caso della divergenza nel warp, introduce costi che superano il beneficio ottenuto.

5.5.2. Conversione float

In float sono state convertite tutte le operazioni che non comportano perdita di accuratezza. Anche il generatore di numeri casuali cuRAND in versione float produce valori nell'intervallo $[0,1)$ con 23 bit di mantissa che sono sufficienti per questo campionamento stocastico. Nella versione 3 sono state infatti inserite due versioni separate del campionamento di numeri casuali;

```
__device__ inline float genera_rng_f(curandStatePhilox4_32_10_t *stato) {
    float valore;
    do {
        valore = curand_uniform(stato);
    } while (valore >= 1.0f);
    return valore;
}
__device__ inline double genera_rng_d(curandStatePhilox4_32_10_t *stato) {
    double valore;
    do {
        valore = curand_uniform_double(stato);
    } while (valore >= 1.0);
    return valore;
}
```

La versione in double viene utilizzata quando l'errore di arrotondamento è critico, come nel caso del campionamento del cammino libero dove valori vicini a 0 o a 1 producono distanze molto grandi o molto piccole.

Per questa conversione sono stati modificati anche alcune funzioni dei moduli, in particolare il modulo compton di cui è stata fatta la versione *compton_float.cuh* e il modulo physics di cui è stata fatta la versione *physics_float.cuh*. Nel modulo compton l'intera funzione di aggiornamento della traiettoria viene fatta in float convertendo in double solamente al momento di scrivere i risultati finali nelle componenti di direzione del fotone.

```
__device__ inline void aggiornamento_trajettoria(double &ux, double &uy, double &uz, double cos_theta, float phi) {
    float sin_theta = sqrtf(fmaxf(0.0f, 1.0f - (float)(cos_theta * cos_theta)));
    float cos_phi = cosf(phi);
    float sin_phi = sinf(phi);
    float ux_new, uy_new, uz_new;
```

```

float fux = (float)ux;
float fuy = (float)uy;
float fuz = (float)uz;
float fcos = (float)cos_theta;

...
float norm = sqrtf(ux_new*ux_new + uy_new*uy_new + uz_new*uz_new);
if (norm > 0.0f) {
    ux = (double)(ux_new / norm);
    uy = (double)(uy_new / norm);
    uz = (double)(uz_new / norm);
}
}

```

Lo stesso per il metodo di Kahn che opera in float mantenendo in double solamente il coseno e l'energia di scatter che servono poi per aggiornare lo stato del fotone:

```

__device__ inline void metodo_kahn(double energia_mev, float xi_1, float xi_2, float xi_3, double &cos_theta, double
&energia_scatter) {
    double alpha = energia_mev / ME_C2;
    double tau_min = 1.0 / (1.0 + 2.0 * alpha);
    float area_ramo_1 = (float)log(1.0 / tau_min);
    float area_ramo_2 = (float)((1.0 - tau_min * tau_min) * 0.5);
    float area_totale = area_ramo_1 + area_ramo_2;
    float tau;
    if (xi_1 * area_totale < area_ramo_1) {
        tau = (float)pow(tau_min, 1.0 - (double)xi_2);
    } else {
        float tau_min_quadro = (float)(tau_min * tau_min);
        float tau_quadro = tau_min_quadro + xi_2 * (1.0f - tau_min_quadro);
        tau = sqrtf(fmaxf(tau_quadro, 1e-30f));
    }
    tau = fminf(fmaxf(tau, (float)tau_min), 1.0f);
    cos_theta = 1.0 - (1.0 - (double)tau) / (alpha * (double)tau);
    cos_theta = fmin(fmax(cos_theta, -1.0), 1.0);
    energia_scatter = (double)tau * energia_mev;
    float sin2_theta = fmaxf(0.0f, 1.0f - (float)(cos_theta * cos_theta));
    float termine_correttivo = (tau * sin2_theta) / (1.0f + tau * tau);
    float probabilita_accettazione = fmaxf(0.0f, fminf(1.0f - termine_correttivo, 1.0f));
    if (xi_3 > probabilita_accettazione)
        cos_theta = 2.0;
}

```

5.5.3. Utilizzo texture memory

Per risolvere il problema degli accessi non coalescenti e ridurre la latenza della memoria globale viene utilizzata la texture memory. La texture memory presenta una cache limitata che è diversa dalla cache L1; infatti, è spazialmente cosciente in 2D e 3D. Quando un thread legge un voxel alle coordinate (i_x, i_y, i_z) la texture cache carica in anticipo anche i voxel nelle tre dimensioni vicine. Se un altro thread poco dopo legge un voxel nelle vicinanze lo trova già in cache.

Un fotone percorre una traiettoria nel volume e i voxel letti in passi successivi sono spazialmente vicini. Anche thread warp diversi che si trovano in regioni diverse del phantom beneficiano del fatto che la cache spaziale è condivisa, infatti i voxel già caricati da thread precedenti restano in cache per quelli successivi che transitano nella stessa regione. La texture memory è read only per cui, considerando che il phantom viene creato una volta sola sulla CPU, non crea nessun problema.

Nella versione 3 il phantom non viene più copiato come array lineare ma come un array CUDA tridimensionale e viene acceduto tramite l'oggetto texture *cudaTextureObject_t*. Per fare questo come prima cosa si alloca l'array tridimensionale che è il contenitore ottimizzato per la texture 3D:

```
cudaArray_t device_phantom_array;  
cudaChannelFormatDesc tipoCanale = cudaCreateChannelDesc<int>();  
cudaExtent estensioneVolume = make_cudaExtent(NX, NY, NZ);  
CUDA_CHECK(cudaMalloc3DArray(&device_phantom_array, &tipoCanale, estensioneVolume));
```

Successivamente viene copiato il phantom dall'host all'array gestendo il layout di memoria in modo ottimizzato:

```
cudaMemcpy3DParms parametriCopia3D = {0};  
parametriCopia3D.srcPtr = make_cudaPitchedPtr((void*)host_phantom, NX * sizeof(int), NX, NY);  
parametriCopia3D.dstArray = device_phantom_array;  
parametriCopia3D.extent = estensioneVolume;  
parametriCopia3D.kind = cudaMemcpyHostToDevice;  
CUDA_CHECK(cudaMemcpy3D(&parametriCopia3D));
```

Viene creato poi il descrittore della risorsa che punta all'array e il descrittore della texture che specifica le modalità di accesso:

```
cudaResourceDesc sorgenteTexture = {};  
sorgenteTexture.resType = cudaResourceTypeArray;  
sorgenteTexture.res.array.array = device_phantom_array;  
cudaTextureDesc texDesc = {};  
texDesc.addressMode[0] = cudaAddressModeClamp;  
texDesc.addressMode[1] = cudaAddressModeClamp;  
texDesc.addressMode[2] = cudaAddressModeClamp;  
texDesc.filterMode = cudaFilterModePoint;  
texDesc.readMode = cudaReadModeElementType;  
texDesc.normalizedCoords = 0;
```

Il parametro *cudaAddressModeClamp* fa sì che qualsiasi tentativo di accedere a una posizione fuori dal limite venga bloccato sul bordo restituendo il valore del voxel più vicino. Questo evita comportamenti imprevedibili del sistema e garantisce una protezione utile contro gli errori di arrotondamento. L'opzione *cudaFilterModePoint* invece disattiva la sfumatura e la media tra i punti adiacenti facendo ottenere un valore esatto e puro del materiale. Il parametro *normalizedCoords = 0* invece stabilisce che le coordinate per muoversi nella griglia siano espresse direttamente con il numero effettivo di voxel anziché utilizzare una percentuale.

Si crea poi l'oggetto texture che viene poi utilizzato come parametro del kernel:

```
cudaTextureObject_t texture_phantom = 0;
```

```
CUDA_CHECK(cudaCreateTextureObject(&texture_phantom, &sorgenteTexture, &texDesc, NULL));
```

Nel kernel il trasporto e la logica implementativa sono uguali a quelli della versione 2, l'unica cosa che cambia è il passaggio dell'oggetto texture al posto dell'array classico.

5.5.4. Versione Beer Lambert

Anche nella semplificazione Beer Lambert sono state utilizzate le stesse ottimizzazioni, float e texture memory della versione completa:

```
__global__ void mc_beer_lambert_kernel( long long num_fotoni, cudaTextureObject_t texture_phantom, double *dose,
unsigned long long *contatore_fotoni_processati, uint64_t seed_base)
{
    while (true) {
        unsigned long long indice_thread = atomicAdd(contatore_fotoni_processati, 1ULL);
        if ((long long)indice_thread >= num_fotoni)
            break;
        curandStatePhilox4_32_10_t stato_casuale;
        curand_init(seed_base, indice_thread, 0, &stato_casuale);
        double cx = PHANTOM_CM / 2.0;
        double cy = PHANTOM_CM / 2.0;
        double x = cx + (double)(genera_rng_f(&stato_casuale) * 2.0f - 1.0f) * SEMI_AMPIEZZA_CAMPO;
        double y = cy + (double)(genera_rng_f(&stato_casuale) * 2.0f - 1.0f) * SEMI_AMPIEZZA_CAMPO;
        double z = 1.0e-7;
        double ux = 0.0, uy = 0.0, uz = 1.0;
        double energia = campiona_energia(&stato_casuale);
        while (energia > ECUT && verifica_confini(x, y, z)) {
            int ix = vox(x);
            int iy = vox(y);
            int iz = vox(z);
            int materiale = tex3D<int>(texture_phantom, ix + 0.5f, iy + 0.5f, iz + 0.5f);
            double mu = calcolo_attenuazione_totale(energia, materiale);
            double distanza_percorsa = -log(genera_rng_d(&stato_casuale)) / mu;
            x += ux * distanza_percorsa;
            y += uy * distanza_percorsa;
            z += uz * distanza_percorsa;
            if (verifica_confini(x, y, z)) {
                int id = phantom_idx(vox(x), vox(y), vox(z));
                atomicAdd(&dose[id], energia);
                break;
            }
        }
    }
}
```

Per il cammino libero medio si mantiene la generazione casuale in double perché la distribuzione esponenziale con logaritmo è sensibile agli arrotondamenti e nella versione Beer Lambert il cammino libero è l'unica variabile fisica campionata.

5.6. Profiling Nsight

Per effettuare un'analisi delle prestazioni in esecuzione delle diverse versioni GPU è stato effettuato il profiling. Per fare questo sono stati utilizzati due diversi tool di profiling messi a disposizione da NVIDIA che sono NCU o Nsight Compute e NVPROF. Il primo è un profiling interattivo di NVIDIA per i kernel CUDA che esegue un'analisi approfondita avanzata riga per riga. In particolare:

```
!ncu --set roofline \  
--metrics sm_warps_active.avg.pct_of_peak_sustained_active,l1tex_t_sector_hit_rate.pct,lts_t_sector_hit_rate.pct \  
--kernel-name mc_kernel \  
--launch-count 1 \  
--export ./profiling/v1_full_report \  
--force-overwrite \  
./GPU_V1/mc_gpu $num_fot_profiling 0 42
```

Questo comando attiva l'analisi *roofline model* che mostra le prestazioni del kernel in relazione ai limiti massimi teorici di calcolo e di larghezza di banda della Tesla T4. Vengono valutati il numero di cache hit della cache di primo e secondo livello e la percentuale di warp attivi.

NVPROF invece, sebbene sia deprecato, consente di ottenere una panoramica completa di tempi di esecuzione, di trasferimenti Host to Device e Device to Host:

```
!nvprof \  
--print-gpu-summary \  
./GPU_V1/mc_gpu $num_fot_profiling 0 42
```

Questi due strumenti sono complementari: NVPROF fornisce il tempo reale di esecuzione mentre Nsight fornisce le metriche hardware interne che giustificano i tempi di esecuzione.

5.6.1. Analisi risultati NVPROF

Il profiling è stato eseguito con un numero di fotoni pari a 10.000.000 e con il phantom omogeneo composto da acqua. Ne risulta:

Metrica	Versione 1	Versione 2	Versione 3
Tempo kernel	9.208 s	5.957 s	4.619 s
Throughput	1.086 MP/s	1.678 MP/s	2.165 MP/s
Tempo per fotone	0.9 μ s	0.6 μ s	0.5 μ s
Host to Device (ms)	1.032	0.939	0.012
Device to Host (ms)	6.294	5.627	5.882
Speedup V1	-	X1.54	X1.99
Speedup V2	-	X1.54	X1.29

Tabella 5 - Risultati analisi NVPROF

Il guadagno complessivo dalla versione base alla versione 3 è quasi del x2 sul tempo del kernel. Ognuna delle tre versioni porta miglioramenti alla precedente pur mantenendo gli stessi risultati fisici, stesso PDD e stesso numero di voxel con dose >0. Inoltre, nella versione 3 il tempo di trasferimento Host to Device è minore rispetto alle altre versioni in quando viene diviso in tempo di trasferimento del contatore atomico e in tempo di trasferimento dell'array.

Dall'analisi delle attività della GPU risulta:

		<i>mc_kernel</i>	<i>Device to Host</i>	<i>Host to Device</i>	<i>cudaMemset</i>	<i>Host to Array</i>
<i>Versione 1</i>	Tempo %	99.94%	0.05%	0.01%	0%	-
	Tempo	9.208 s	4.969 ms	0.750 ms	0.028 ms	-
<i>Versione 2</i>	Tempo %	99.91%	0.07%	0.01%	0%	-
	Tempo	5.957s	4.455 ms	0.700 ms	0.028 ms	-
<i>Versione 3</i>	Tempo %	99.88%	0.10%	-	0%	0.02%
	Tempo	4.619s	4.750 ms	-	0.028 ms	0.800 ms

Tabella 6 - Analisi attività GPU

La versione 3 presenta il tempo di copia verso l'array CUDA ma non verso il device per l'implementazione con texture memory. Inoltre, questa presenta una diminuzione della percentuale di tempo del kernel non perché è aumentato il tempo di Device to Host ma perché il kernel è diventato più veloce e quindi il suo peso totale diminuisce.

5.6.2. Analisi risultati Nsight

Per quanto riguarda le metriche raccolte i risultati ottenuti sono:

<i>Metrica</i>	<i>Versione 1</i>	<i>Versione 2</i>	<i>Versione 3</i>
<i>Occupancy Warp</i>	71.87%	99.85%	61.43%
<i>Hit Rate cache L1</i>	34.24%	29.43%	91.13%
<i>Hot Rate cache L2</i>	92.26%	85.28%	75.48%

Tabella 7 - Risultati metriche Nsight

Per quanto riguarda la percentuale di warp attivi rispetto al picco teorico sostenibile tra la versione 1 e la versione 2 si registra un importante salto. Questo dimostra come mantenendo un numero fisso e ottimizzato di thread attivi per tutta la durata del kernel, si azzerano le perdite causate dallo sbilanciamento del carico della versione 1. Nella terza versione questo valore scende perché, sostituendo gran parte delle operazioni in doppia precisione con la precisione singola ed eliminando la memoria globale il kernel diventa più veloce e i thread completano le loro istruzioni molto prima riducendo il numero di warp contemporaneamente residenti pur raddoppiando il throughput.

Per la cache hit L1 che misura la percentuale di accessi alla memoria che trovano i dati già nella cache, si passa da 34-29% delle prime due versioni dove il phantom è memorizzato come un array in memoria globale al 91% della versione 3. Questo avviene perché la cache detta texture memory è ottimizzata per il pattern di accesso 3D poiché sono precaricati in cache tutti i voxel spazialmente confinanti con quello acceduto.

Nel cache hit L2 in accordo al miglioramento sulla cache L1 si passa dal 92-85% delle prime due versioni al 75% della terza versione. Questo calo è proprio dovuto al successo delle richieste nella cache L1, infatti, siccome il 91% circa di richieste viene soddisfatto immediatamente a livello 1 alla cache di livello 2 arrivano meno richieste, dovute ad accessi sparsi il cui pattern rimanente è meno predicibile, riducendo di conseguenza la percentuale di hit locale.

5.6.3. Analisi complessiva dei risultati

Tra la versione 1 e la versione 2 si ha un guadagno perché, mentre con il modello di un thread per fotone blocchi da 256 thread si liberano solo quando tutti hanno terminato la propria storia, indipendentemente da quanto questa sia variabile, con il persistent thread pool i thread che finiscono

prima acquisiscono immediatamente il prossimo fotone e rimangono attivi. Si elimina, pertanto, tutto il processo di creazione e distruzione del thread ogni volta.

Il miglioramento invece che si ha nella versione 3 rispetto alla versione 2 è dovuto alla sostituzione di alcune operazioni FP64 in FP32 riducendo la pressione sulle unità di calcolo che nella versione a 64 hanno un throughput inferiore di 32 volte rispetto alla versione FP32. Inoltre, con il meccanismo di texture memory si hanno cache hit di livello 1 maggiori e si evitano accessi alla memoria globale che ha una latenza maggiore. Nella terza versione il numero di voxel con una dose maggiore di 0 è 998.629 contro 998.633 delle altre due. Questo è dovuto al cambio di precisione tra le versioni che porta un certo numero di fotoni a cadere in voxel adiacenti rispetto a quelli corretti. Questa differenza è inferiore allo 0.0004% per cui è fisicamente accettabile.

La versione 1 rappresenta la scrittura base del codice CUDA di Monte Carlo per Radioterapia; nella versione 2 viene risolto lo sbilanciamento del carico tra i thread mentre nella versione 3 viene ridotto il costo computazionale delle operazioni FP64 e viene migliorata l'efficienza di accesso al phantom ottenendo un kernel che processa quasi il doppio dei fotoni per unità di tempo rispetto alla versione base mantenendo risultati fisici coerenti.

6. Test dei risultati

La simulazione Monte Carlo per radioterapia produce una distribuzione di dose numerica che per essere valida deve essere confrontata con i dati sperimentali o con le leggi fisiche del problema stesso. Sono stati per questo scritti dei test, in particolare 10, che controllano che ogni versione del simulatore, indipendentemente dall'implementazione hardware può essere eseguita sullo stesso codice di validazione cambiando solo le costanti relative ai file di output garantendo che le ottimizzazioni introdotte nelle diverse versioni non alterino la fisica.

Per effettuare i singoli test sono stati utilizzati dei valori di riferimento della letteratura coerenti con il simulatore implementato. È stato definito lo spettro energetico a 6 MV come nel modulo physics con i valori derivanti da Sheikh-Bagheri e Rogers. Da questi deriva il valore di energia media che è di 1.96 MeV che viene poi utilizzato nel Test 5 per calcolare l'energia totale immessa nel sistema. Dalla stessa pubblicazione sono stati presi anche i valori di PDD di riferimento clinico che rappresentano il Percent Depth Dose per il fascio da 6MV che vengono utilizzati nel test 6 per il confronto visuale e nel test 9 per il calcolo del gamma index.

Sono state definite anche delle funzioni di utilità come il calcolo del coseno medio teorico e la simulazione degli urti compton. In particolare:

```
def calcola_coseno_medio_teorico(energia_mev, num_punti=20_000):
    alfa = energia_mev / MASSA_ELETTRONE_MEV
    coseni = np.linspace(-1.0, 1.0, num_punti)
    tau = 1.0 / (1.0 + alfa * (1.0 - coseni))
    probabilita = 0.5 * tau**2 * (tau + 1.0/tau - 1.0 + coseni**2)
    area = np.trapezoid(probabilita, coseni)
    probabilita_norm = probabilita / area
    coseno_medio = np.trapezoid(coseni * probabilita_norm, coseni)
    return float(coseno_medio)
```

Che calcola analiticamente il coseno medio dell'angolo di scattering Compton integrando numericamente la sezione d'urto differenziale di Klein Nishina in forma adimensionale. Il valore restituito è il valore teorico esatto con cui poi verranno confrontati i campioni generati dal metodo di Kahn della simulazione nel test 4.

La funzione di simulazione degli urti compton, definita come:

```
def simula_urti_compton(energia_mev, generatore_casuale, n_campioni=50_000):
    alfa = energia_mev / MASSA_ELETTRONE_MEV
    tau_min = 1.0 / (1.0 + 2.0 * alfa)
    limite_accettazione_1 = np.log(1.0 / tau_min)
    limite_accettazione_2 = (1.0 - tau_min**2) * 0.5
    totale_limiti = limite_accettazione_1 + limite_accettazione_2
    coseni_generati = []
    tentativi_totali = 0
    while len(coseni_generati) < n_campioni:
        tentativi_totali += 1
        r1, r2, r3 = generatore_casuale.random(3)
        if r1 * totale_limiti < limite_accettazione_1:
            tau = tau_min**(1.0 - r2)
        else:
            tau = np.sqrt(tau_min**2 + r2 * (1.0 - tau_min**2))
        tau = np.clip(tau, tau_min, 1.0)
        coseno = 1.0 - (1.0 - tau) / (alfa * tau)
        coseno = np.clip(coseno, -1.0, 1.0)
        seno2 = max(0.0, 1.0 - coseno**2)
        filtro = 1.0 - (tau * seno2) / (1.0 + tau**2)
        if r3 <= filtro:
            coseni_generati.append(coseno)
    efficienza = (len(coseni_generati) / tentativi_totali) * 100
    return np.array(coseni_generati), efficienza
```

È un'implementazione della logica del metodo di Kahn presente nel modulo compton che serve a verificare che l'implementazione del campionamento compton della simulazione si comporti in modo equivalente all'implementazione di riferimento. Serve anche a verificare che il rejection sampling non si sia degradato.

La stessa suite di test viene eseguita su tutte le versioni del simulatore cambiando solamente il blocco di costanti, che rappresentano gli eseguibili, all'inizio di ogni sessione.

6.2. Test 1 – Analisi del coefficiente di attenuazione lineare

Il primo test serve a verificare che i fotoni simulati si attenuino nel modo corretto. La legge di Beer Lambert afferma che l'intensità di un fascio di fotoni monoenergetici decresce esponenzialmente con la profondità

$$I(z) = I_0 * \exp(-\mu z)$$

Dove μ è il coefficiente di attenuazione lineare del mezzo. Per un fascio da 6 MV i fotoni a bassa energia però, vengono assorbiti prima e lo spettro si indurisce con la profondità. Tuttavia, nelle regioni oltre il

massimo di dose che sono approssimativamente da 5 a 20 cm, il comportamento è approssimato da un'esponenziale con un valore di μ effettivo che rappresenta l'energia media del fascio penetrante. Il test esegue una regressione lineare sul logaritmo del PDD in questa regione e ricava il valore μ della pendenza:

```
profondita, dose = load_pdd(PDD_WATER)
if profondita is None:
    return False
filtro_zona_fit = (profondita >= 5.0) & (profondita <= 20.0) & (dose > 0)
z_da_analizzare = profondita[filtro_zona_fit]
dose_da_analizzare = dose[filtro_zona_fit]
pendenza, intercetta = np.polyfit(z_da_analizzare, np.log(dose_da_analizzare), 1)
mu_misurato = -pendenza
MU_MIN = 0.030
MU_MAX = 0.055
test_superato = MU_MIN <= mu_misurato <= MU_MAX
```

Il range di accettabilità è definito nella letteratura scientifica. Un valore fuori da questo range indica un errore nell'attenuazione o nel campionamento del cammino libero. Il coefficiente di determinazione della regressione in questo test misura quanto bene i dati seguono una legge esponenziale, infatti, un valore vicino a 1 indica che il comportamento è coerente con la fisica attesa mentre un valore basso indica errori nel profilo di dose.

6.3. Test 2 – Validazione della legge di Beer Lambert

Il secondo test usa i risultati della simulazione Beer Lambert semplificata invece del kernel completo. Questo test serve a valutare il caso limite e quindi se si rimuove completamente la fisica dello scattering e si lascia solo l'attenuazione esponenziale con deposizione locale dell'energia. In questo caso il PDD risultante deve corrispondere esattamente alla curva teorica definita, con il valore di μ definito in letteratura nel database NIST, come:

$$\exp(-\mu z)$$

In particolare, il confronto avviene punto per punto su sette profondità di riferimento con una tolleranza di $\pm 2\%$ per ciascun punto.

```
MU_TEORICO = 0.04942
TOLLERANZA_DOSE = 2.0
profondita, dose_mc = load_pdd(PDD_WATER_BL)
dose_teorica = 100.0 * np.exp(-MU_TEORICO * profondita)
esiti_punti = []
profondita_controllo = [1.5, 3.0, 5.0, 10.0, 15.0, 20.0, 25.0]
for z in profondita_controllo:
    idx = np.abs(profondita - z).argmin()
    d_sim = dose_mc[idx]
    d_teo = 100.0 * np.exp(-MU_TEORICO * z)
    differenza = abs(d_sim - d_teo)
    punto_ok = differenza < TOLLERANZA_DOSE
    esiti_punti.append(punto_ok)
    stato = "BUONO" if punto_ok else "ERRORE"
    print(f"{z:>10.1f} | {d_sim:>12.2f} | {d_teo:>12.2f} | {stato:>8} ({differenza:.3f})")
```

```

filtro = (profondita >= 5.0) & (profondita <= 20.0)
pendenza, _ = np.polyfit(profondita[filtro], np.log(dose_mc[filtro]), 1)
mu_misurato = -pendenza
errore_mu = abs(mu_misurato - MU_TEO_RICO) / MU_TEO_RICO * 100
mu_ok = errore_mu < 2.0
esiti_punti.append(mu_ok)
test_superato = all(esiti_punti)

```

Questo test verifica indipendentemente sia che il campionamento del cammino libero medio e la deposizione dell'energia locale siano corretti sia che siano isolati dalla complessità dello scattering. Se il kernel Beer Lambert produce un profilo esponenziale sbagliato si ha un problema nel campionamento di base mentre se il kernel Beer Lambert è corretto ma il kernel completo no, allora si ha qualche problema nella fisica dello scattering.

6.4. Test 3 – Verifica delle sezioni d'urto e probabilità di interazione

Il terzo test non è dipendente dalla simulazione ma serve a validare la logica del database NIST. Deve infatti risultare che a 2.0 MeV in acqua la scomposizione delle sezioni d'urto NIST è netta, ovvero:

- L'effetto compton domina con quasi il 100% delle interazioni;
- L'effetto fotoelettrico è trascurabile;
- La produzione di coppie non è ancora attiva ma diventa significativa solo oltre i 5 MeV.

```

mu_fotoelettrico = 2.200e-7
mu_compton = 0.04942
mu_coppie = 0.0
mu_totale = mu_fotoelettrico + mu_compton + mu_coppie
perc_fotoelettrico = (mu_fotoelettrico / mu_totale) * 100
perc_compton = (mu_compton / mu_totale) * 100
perc_coppie = (mu_coppie / mu_totale) * 100

```

Queste verifiche devono dimostrare che l'effetto compton superi il 99.9%, quello fotoelettrico sia sotto lo 0.01% e la produzione di coppie sia esattamente 0, mantenendo la coerenza matematica. Questo test è deterministico e serve a verificare che le costanti fisiche siano coerenti con i valori tabulati dal NIST.

6.5. Test 4 – Validazione stocastica della deviazione compton

Il metodo di Kahn implementato nel modulo di compton è un algoritmo di rejection sampling che deve riprodurre la distribuzione angolare predetta dalla formula di Klein-Nishina. Il test verifica che questa corrispondenza sia quantitativamente corretta confrontando il coseno medio dei campioni generati con il valore teorico integrato:

```

energie_test = [0.5, 1.0, 2.0, 4.0, 6.0]
generatore_casuale = np.random.default_rng(42)
numero_campioni = 50_000
limite_errore = 5.0
esiti = []
dati_per_grafico = {}
for E in energie_test:
    media_teorica = calcola_coseno_medio_teorico(E)

```

```

angoli_campionati, efficienza = simula_urti_compton(E, generatore_casuale, n_campioni=numero_campioni)
media_mc = float(angoli_campionati.mean())
errore_relativo = abs(media_mc - media_teorica) / abs(media_teorica) * 100.0
ok = errore_relativo < limite_errore
esiti.append(ok)
dati_per_grafico[E] = angoli_campionati

```

Vengono utilizzate cinque energie campione da 0.5 MeV a 6.0 MeV così da coprire l'intero range dello spettro. A energie basse lo scattering è isotropo, quindi il coseno medio è circa 0.3, a energie alte invece è fortemente avanti, quindi il coseno è circa 0.85.

Verificare che la distribuzione sia corretta a entrambi gli estremi garantisce che l'algoritmo di Kahn funzioni anche in tutto il range energetico. L'efficienza di campionamento indica, invece, la correttezza del rejection sampling. Se questa è troppo bassa vuol dire che si stanno rifiutando candidati corretti; se al contrario è troppo alta si stanno accettando candidati non corretti. Questo test serve a verificare che l'algoritmo implementato poi nelle diverse versioni sia corretto matematicamente.

6.6. Test 5 – Analisi del bilancio energetico globale

Il bilancio energetico è il principio di conservazione applicato alla simulazione. Secondo questo l'energia totale emessa dalla sorgente deve essere uguale alla somma di energia depositata nel phantom più energia trasportata fuori dai fotoni trasmessi e diffusi, ovvero:

```

energia_media_singolo_fotone = ENERGIA_MEDIA_6MV
dose_3d = load_dose_bin(DOSE_WATER)
if dose_3d is None:
    return False
energia_totale_depositata = float(dose_3d.sum())
energia_totale_emessa = N * energia_media_singolo_fotone
frazione_depositata = energia_totale_depositata / energia_totale_emessa
voxel_colpiti = int((dose_3d > 0).sum())
percentuale_volume = (voxel_colpiti / dose_3d.size) * 100

```

Per un phantom d'acqua cubico di lato 30 cm con fascio da 6MV la frazione di energia depositata attesa è compresa tra il 30% e il 90%. Questo range è così ampio sia perché i fotoni ad alta energia passano attraverso il volume senza fermarsi sia perché i fotoni diffusi lateralmente escono dai lati del phantom. L'obiettivo del test non è predire la frazione depositata ma per verificar che la simulazione sia termodinamicamente consistente.

6.7. Test 6 – Validazione della forma del PDD clinico

Il PDD è la quantità clinica più importante per la pianificazione della radioterapia. Per un fascio da 6MV in acqua la forma attesa deve avere una zona di build-up nei primi centimetri, dove gli elettroni secondari depositano energia lontano dal loro punto di liberazione, e un massimo di dose intorno a 1.5 cm e un decremento monotono per profondità maggiori. Il test verifica tre parametri quantitativi;

```

indice_picco = np.argmax(dose)
z_picco = profondita[indice_picco]
valore_picco = dose[indice_picco]
dose_10 = dose[np.abs(profondita - 10.0).argmin()]
dose_20 = dose[np.abs(profondita - 20.0).argmin()]

```

```

rapporto_10_max = dose_10 / valore_picco
rapporto_20_10 = dose_20 / dose_10
controlli = [
    ("Profondità del picco (0-2 cm) :", 0.0 <= z_picco <= 2.0, f"{z_picco:.2f} cm"),
    ("Rapporto Dose 10cm / Max   :", 0.60 <= rapporto_10_max <= 0.82, f"{rapporto_10_max:.3f}"),
    ("Rapporto Dose 20cm / 10cm  :", 0.45 <= rapporto_20_10 <= 0.72, f"{rapporto_20_10:.3f}")
]

```

I valori dei controlli sono derivati dai parametri dosimetrici standard usati nella fisica medica per caratterizzare la qualità del fascio. Il range di accettabilità è ampio così da tenere conto delle approssimazioni del simulatore, come l'assenza del trasporto esplicito degli elettroni secondari, ma anche abbastanza stretto da escludere eventuali errori di notevole importanza. Il test viene completato con un confronto visuale con i dati di Sheikh Bagheri tramite interpolazione cubica che mostra l'errore medio assoluto tra la curva simulata e quella di riferimento.

6.8. Test 7 – Verifica dell'eterogeneità acqua-osso

Questo test serve a verificare che il phantom eterogeneo produca effetti dosimetrici fisicamente corretti. I due effetti fisici attesi sono:

- Maggiore dose dentro l'osso, ovvero a parità di profondità l'osso deposita più energia per unità di volume rispetto all'acqua a causa della sua maggiore densità e sezione d'urto fotoelettrica;
- Effetto ombra a valle, ovvero l'osso attenua il fascio primario più dell'acqua riducendo la dose nelle regioni oltre l'inserito.

```

prof_acqua, pdd_acqua = load_pdd(PDD_WATER)
prof_osso, pdd_osso = load_pdd(PDD_HETERO)
fetta_inizio, fetta_fine = int(12.5 / VOXEL_CM), int(17.5 / VOXEL_CM)
dose_media_acqua = pdd_acqua[fetta_inizio:fetta_fine].mean()
dose_media_osso = pdd_osso[fetta_inizio:fetta_fine].mean()
fetta_20cm = int(20.0 / VOXEL_CM)
dose_dopo_acqua = pdd_acqua[fetta_20cm]
dose_dopo_osso = pdd_osso[fetta_20cm]
diff_dentro = dose_media_osso - dose_media_acqua
diff_dopo = dose_dopo_acqua - dose_dopo_osso
controlli = [
    ("Maggiore assorbimento nell'osso", diff_dentro > 0),
    ("Presenza effetto ombra dopo l'osso", diff_dopo > 0),
    ("Entità dell'ombra significativa (>3%)", diff_dopo > 3.0)
]

```

La soglia del 3% è dovuta all'effetto ombra a 20 cm. Infatti, con un inserto osseo da 5 cm a densità quasi doppia dell'acqua l'attenuazione aggiuntiva deve essere misurabile e significativa a 20 cm di profondità. Il trasporto corretto prevede un aggiornamento del materiale del voxel corrente ad ogni step del trasporto, non solo all'inizio della storia. Questa funzione serve pertanto a validare proprio le funzioni di trasporto fotoni e del calcolo della distanza limite del voxel se che sbagliati produrrebbero un PDD eterogeneo identico a quello omogeneo.

6.9. Test 8 – Analisi della penombra laterale e diffusione compton

La penombra laterale è la zona ai bordi del campo di radiazione dove la dose passa dal valore centrale a zero. La sua larghezza aumenta con la profondità sia per la diffusione laterale dei fotoni compton che sposta la dose lateralmente rispetto alla direzione originale, sia per la divergenza geometrica del fascio. In questo test viene considerato il profilo laterale della dose a diverse profondità e misura la larghezza della penombra come distanza tra i punti all'80% e al 20% del massimo profilo, che sono i valori standard della fisica medica:

```
profondita_controllo = [3.0, 5.0, 10.0, 15.0, 20.0]
misure_penombra = []
profili_grafico = {}
posizioni_x = (np.arange(NX) + 0.5) * VOXEL_CM - (PHANTOM_CM / 2.0)
def penombra_subvoxel(profilo_norm, pos_x):
    ...
    p80 = crossover(p_r, x_r, 80.0)
    p20 = crossover(p_r, x_r, 20.0)
    ...
    return abs(p20 - p80)
```

La penombra deve aumentare monotonamente con la profondità e il rapporto tra la penombra a 20 cm e quella a 2 cm deve essere maggiore del 10%. Questo test serve quindi sia a verificare l'andamento qualitativo e quindi come la penombra cresce con la profondità, sia quello quantitativo e quindi rapporto e pendenza. Un errore nella diffusione Compton invaliderebbe i profili laterali alterando il risultato del test.

6.10. Test 9 – Gamma Index

Il Gamma Index è lo standard internazionale per il confronto tra le distribuzioni di dose nella fisica medica. Questo meccanismo, invece di confrontare la dose solo punto per punto combina simultaneamente la differenza di dose e la distanza spaziale al punto più simile secondo il criterio geometrico espresso come:

$$\gamma(r_{ref}) = \min \sqrt{\left(\frac{\Delta D}{\Delta D_{tol}}\right)^2 + \left(\frac{\Delta r}{\Delta r_{tol}}\right)^2}$$

Un punto passa il test se esiste un punto nella distribuzione di riferimento con cui la differenza sia contemporaneamente entro la tolleranza di dose e di distanza. Viene usato il criterio del 3%/3mm ovvero della tolleranza di 3% sulla dose e 3mm sulla distanza ed è applicato nella zona di 8-12 cm di profondità. Viene utilizzata questa zona perché è quella di Charged Particle Equilibrium o CPE dove la dose depositata localmente è proporzionale alla fluenza dei fotoni e l'assenza del trasporto esplicito degli elettroni secondari del simulatore emette meno bias.

```
profondita_mc, dose_mc_grezza = load_pdd(PDD_WATER)
profondita_rif = PROFONDITA_RIF
dose_rif_storica = DOSE_RIF
idx_10cm_mc = np.argmin(np.abs(profondita_mc - 10.0))
idx_10cm_rif = np.argmin(np.abs(profondita_rif - 10.0))
valore_10cm_mc = dose_mc_grezza[idx_10cm_mc]
valore_10cm_rif = dose_rif_storica[idx_10cm_rif]
```



```

dose_mc_percentuale = (dose_mc_grezza / valore_10cm_mc) * 100.0
dose_rif_percentuale = (dose_rif_storica / valore_10cm_rif) * 100.0
tolleranza_dose = 3.0 # %
tolleranza_distanza = 0.3 # cm
def calcola_gamma_index(d_test, d_rif, z_test, z_rif):
    ...
    profondita_min_valida = 8.0
    profondita_max_valida = 12.0
    maschera_zona_valida = (profondita_rif >= profondita_min_valida) & (profondita_rif <= profondita_max_valida)
    punti_ok = valori_gamma[maschera_zona_valida] <= 1.0
    percentuale_passati = np.mean(punti_ok) * 100
    soglia_minima = 90.0
    test_superato = percentuale_passati >= soglia_minima

```

La soglia del 90% è il criterio standard per la verifica dei piani di trattamento clinici definiti dall'AAPM (American Association of Physicist in Medicine).

6.11. Test 10 – Validazione della convergenza statistica

Il metodo Monte Carlo per radioterapia presenta un'incertezza statistica intrinseca che decresce come $1/\sqrt{N}$ all'aumentare del numero di storie simulate. La deviazione standard della media di N campioni indipendenti è:

$$\sigma_{media} = \frac{\sigma}{\sqrt{N}}$$

Questo test serve a verificare sia la correttezza delle implementazioni parallele; quindi, che non portino dietro correlazione fisica tra le storie, sia a verificare la generazione dei numeri casuali. Il test esegue più simulazioni indipendenti per diversi valori di N e misura la deviazione standard dell'energia depositata per fotone tra le ripetizioni:

```

configurazioni = [(10000, 20), (100000, 20), (1000000, 20)]
if not salta_lento:
    configurazioni.append((10000000, 5))
lista_sigma = []
lista_N = []
for N, n_ripetizioni in configurazioni:
    risultati_energia = []
    for i in range(n_ripetizioni):
        if lancia_simulazione(N, tipo_phantom=0, seed=500 + i):
            dati_dose = load_dose_bin(DOSE_WATER)
            if dati_dose is not None:
                risultati_energia.append(dati_dose.sum() / N)
    if len(risultati_energia) < 3:
        continue
    media = np.mean(risultati_energia)
    dev_standard = np.std(risultati_energia, ddof=1)
    lista_sigma.append(dev_standard)
    lista_N.append(N)
log_N = np.log10(lista_N)
log_sigma = np.log10(lista_sigma)

```

```
pendenza, intercetta = np.polyfit(log_N, log_sigma, 1)
test_superato = abs(pendenza - (-0.5)) < 0.15
```

Una pendenza significativamente diversa da 0.5 indica una correlazione tra le storie e quindi errori nella gestione dei numeri casuali. Questo test è quello più costoso in quanto per ciascuna ripetizione servono diverse esecuzioni complete del simulatore.

7. Risultati e metriche di prestazione

Le prestazioni del sistema implementato sono state valutate scalando il carico da 100.000 fotoni fino a 50.000.000 sia sulle configurazioni omogenee che su quelle eterogenee della versione kernel completa. In questo genere di sistemi la valutazione dell'implementazione avviene attraverso due metriche fondamentali che sono lo strong scaling che richiede di mantenere fisso il carico di lavoro aumentando il numero di risorse computazionali e il weak scaling che prevede di scalare la dimensione del problema proporzionalmente all'aumento delle risorse. Tuttavia, a causa dei vincoli dell'ambiente di esecuzione non è stato possibile effettuare queste valutazioni, infatti, l'ambiente Colab fornisce un'allocazione fissa che comprende una singola GPU Tesla T4 e due CPU Intel. La valutazione del sistema viene fatta considerando solamente la scalabilità rispetto al carico.

Per poter derivare le metriche prestazionali come i GFLOPS e la larghezza di banda effettiva è necessario sapere il peso operativo associato al singolo fotone. Per fare questo sono stati estratti i coefficienti tramite il profiling del codice sulla GPU analizzando un campione di 100.000 fotoni. Le metriche sono state definite come:

```
df["flop_per_fotone"] = FLOP_PER_FOTONE_COMPLETA
df["byte_per_fotone"] = BYTE_PER_FOTONE_COMPLETA

df["gflops"] = df["n_fotoni"] * df["flop_per_fotone"] / df["tempo_medio_s"] / 1e9
df["banda_gbs"] = df["n_fotoni"] * df["byte_per_fotone"] / df["tempo_medio_s"] / 1e9
```

Dove i parametri sono stati definiti come:

```
FLOP_PER_FOTONE_COMPLETA = 29113
BYTE_PER_FOTONE_COMPLETA = 3633
```

Nel kernel completo sono richiesti campionamenti stocastici continui, soprattutto nel metodo di Kahn, oltre al tracciamento e alla risoluzione delle collisioni. Dalla profilazione considerando operazioni in doppia precisione, quindi FP64, per 100.000 fotoni si hanno:

Operazione	Valore
Addizioni (<i>daad</i>)	218.578.841
Moltiplicazioni (<i>dmul</i>)	226.292.929
Fused Multiply Add (<i>dfma</i>)	1.233.193.679

Il volume totale di operazioni in virgola mobile, considerando che FMA esegue due operazioni matematiche per singolo ciclo di clock, è:

$$FLOP_{tot} = daad + dmul + (2 \times dfma) = 218.578.841 + 226.292.929 + (2 \times 1.233.193.679) = 2.911.259.128$$

Considerando che sono stati calcolati su 100.000 fotoni si ottiene un carico di 29.113 FLOP per fotone. Per quanto riguarda invece il calcolo dei byte utile per le metriche della DRAM, sulla base dei dati di profiling si ottiene:

Operazione	Valore
DRAM Read	214,52 MB
DRAM Write	148,77 MB

$$Byte_{tot} = 214,52 MB + 148,77 MB = 363,29 MB \approx 363.290 Byte$$

Considerando che sono stati calcolati su 100.000 fotoni si ottiene un traffico medio pari a 3.633 byte per fotone che comprende la lettura dei dati della particelle, le letture del phantom e delle costanti fisiche che non si trovano nella L1 e nella L2 e le scritture atomiche contese sulla griglia di dose.

7.1. Tempo di esecuzione

Il tempo di esecuzione rappresenta la misura diretta dell'efficienza della computazione e indica il tempo di calcolo reale impiegato per simulare il sistema. Per le GPU viene considerata la separazione tra il tempo di computazione del kernel e il tempo completo che include anche i trasferimenti. È stato calcolato come:

```
tempo_kernel = riga["tempo_medio_s"].values[0]
if versione.startswith("GPU") and not np.isnan(riga["h2d_medio_ms"].values[0]):
    tempo_e2e = (riga["h2d_medio_ms"].values[0] + riga["kernel_medio_ms"].values[0] +
                riga["d2h_medio_ms"].values[0]) / 1000.0
else:
    tempo_e2e = tempo_kernel
```

I risultati sono:

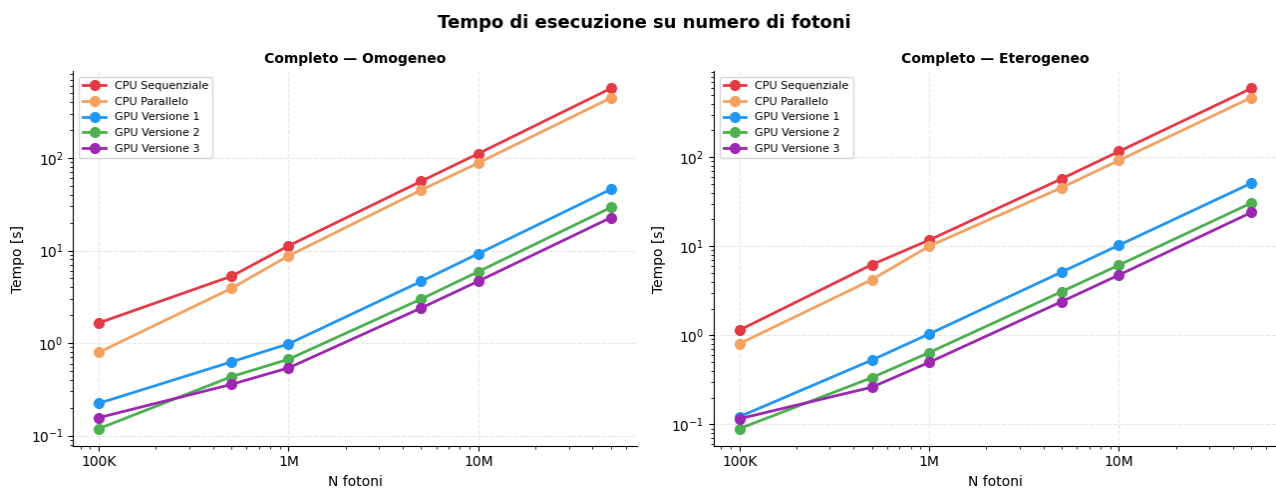


Figura 1 - Tempo di esecuzione

Il grafico mostra un andamento di esecuzione lineare rispetto al numero di fotoni, questo perché il problema presenta una complessità computazionale di $O(N)$. In entrambi i tipi di phantom si ha che la versione CPU sequenziale è quella che impiega maggiore tempo computazionale mentre la versione 3 della GPU presenta tempo di esecuzione molto più basso.

7.2. Speedup assoluto

Lo Speedup assoluto rappresenta quante volte l'implementazione parallela è più veloce rispetto all'esecuzione base GPU. Si calcola come:

$$Speedup = \frac{T_{SEQ}}{T_{PAR}}$$

Nel calcolo il valore viene estratto considerando il tempo sequenziale e quello parallelo sullo stesso numero di fotoni:

```
speedup_vs_seq = tempo_cpu_seq / tempo_kernel if not np.isnan(tempo_cpu_seq) else np.nan
speedup_e2e_vs_seq = tempo_cpu_seq / tempo_e2e if not np.isnan(tempo_cpu_seq) else np.nan
speedup_vs_par = tempo_cpu_par / tempo_kernel if not np.isnan(tempo_cpu_par) else np.nan
speedup_e2e_vs_par = tempo_cpu_par / tempo_e2e if not np.isnan(tempo_cpu_par) else np.nan
```

I risultati sono:

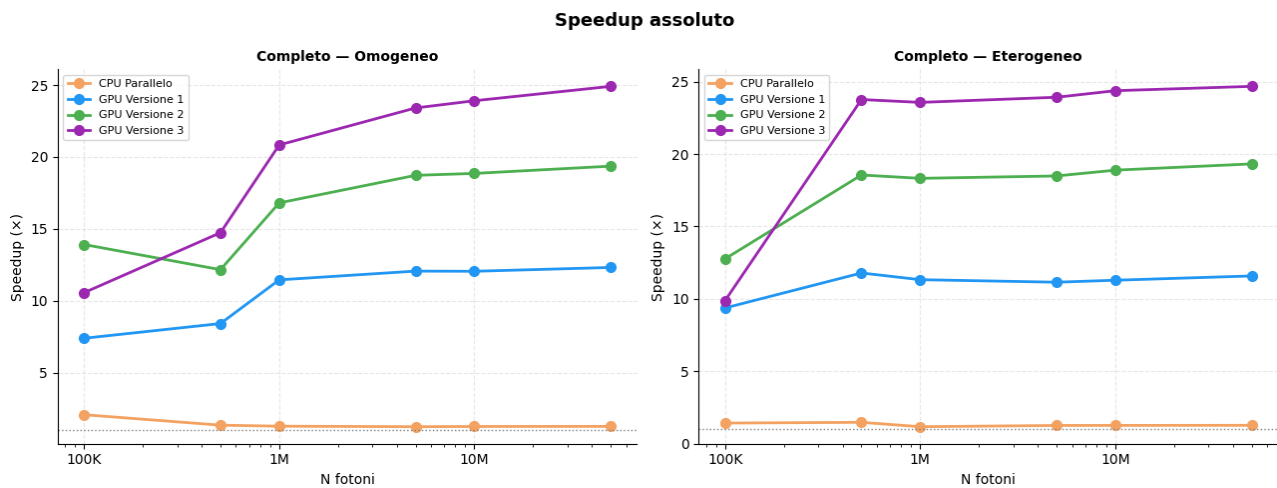


Figura 2 - Speedup assoluto

Per bassi carichi di lavoro le GPU restituiscono accelerazioni moderate perché i costi fissi di lancio del kernel e inizializzazione sovrastano il lavoro e guadagno utile e la GPU non si satura. Aumentando il valore di N le curve crescono fino a raggiungere, quasi, un valore asintotico, che è quello in cui la GPU raggiunge la saturazione degli Streaming Multiprocessor. In questa fase la versione 3 della GPU presenta una capacità di calcolo di quasi 25x rispetto alla versione CPU sequenziale. La versione CPU parallela rimane molto bassa a causa del numero ridotto di core fisici disponibili su Colab.

7.3. Speedup relativo

Lo Speedup relativo rappresenta i miglioramenti prestazionali che ogni versione GPU introduce rispetto alla versione precedente. Questo è utile per valutare se l'ottimizzazione implementata ha portato o meno miglioramenti. Viene calcolata come:

$$Speedup_{relativo} = \frac{T_{GPU_{precedente}}}{T_{GPU_{corrente}}}$$

Viene calcolata dinamicamente come:

```
speedup_relativo = tempo_gpu_precedente_kernel / tempo_kernel
```

```
speedup_relativo_e2e = tempo_gpu_precedente_e2e / tempo_e2e
```

I risultati sono:

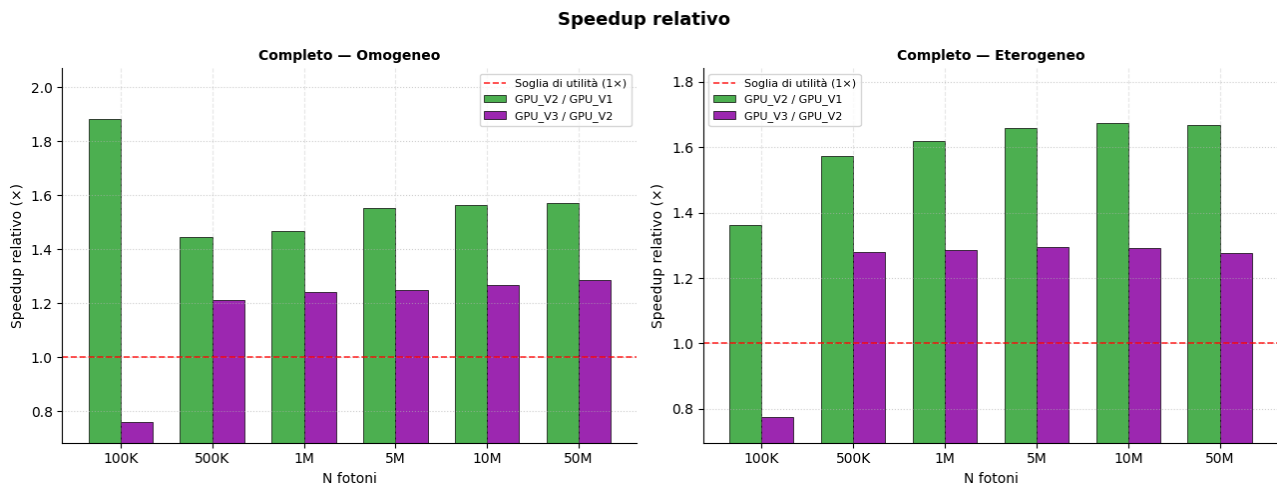


Figura 3 - Speedup relativo

In questo grafico vengono rappresentate separatamente i diversi impatti delle ottimizzazioni. Mostra come il passaggio dalla versione 1 alla versione 2 porta ad un aumento delle performance notevoli con miglioramenti che vanno da 1.4x a 1.9x. Il passaggio invece dalla versione 2 alla versione 3 aggiunge dei miglioramenti seppur non notevoli. Per valori di fotoni considerevoli tutti i miglioramenti sono sopra la soglia utile di 1x.

7.4. Throughput

Il throughput misura il volume di carico reale processato per unità di tempo. A differenza dello speedup che è adimensionale, da un risultato sulla base del numero di fotoni calcolati al secondo. Viene calcolato come:

$$Throughput = \frac{N_{fotoni}}{T_{esecuzione} * 10^6}$$

Nel codice viene calcolato come:

```
df["milioni_fotoni_s"] = df["n_fotoni"] / df["tempo_medio_s"] / 1e6
```

I risultati sono:

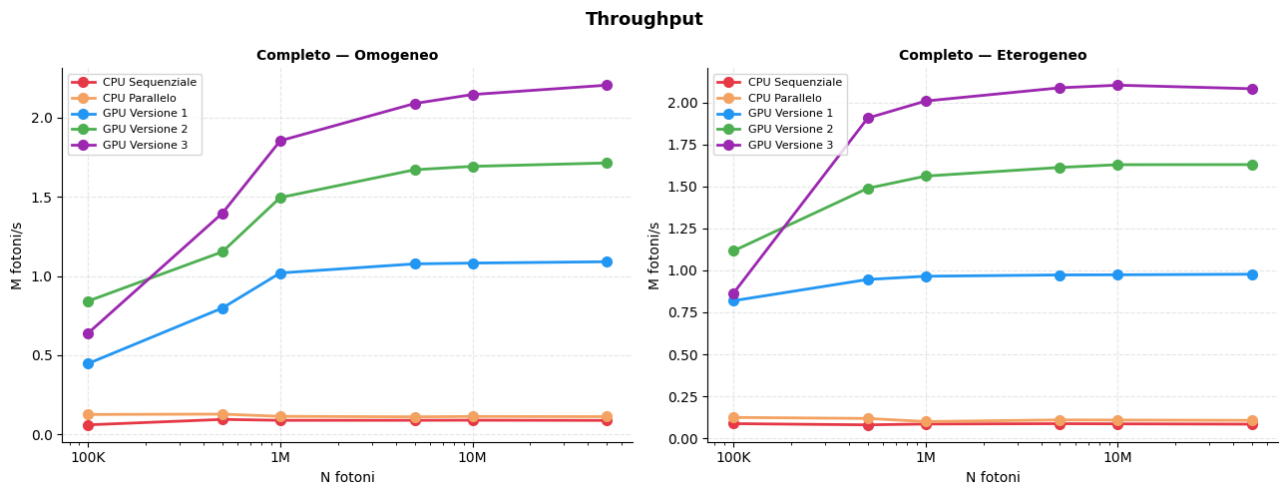


Figura 4 – Throughput

Tutte le versioni, quando si ha un numero di fotoni limitato, partono con valori bassi per poi subire un'accelerazione importante fino a stabilizzarsi sul limite massimo di assorbimento. La versione 3 presenta un valore di throughput massimale di circa $2.2Mfot/s$.

7.5. Heatmap dello Speedup assoluto

È stato generato un heatmap per rappresentare, sul numero massimo di fotoni testato, l'andamento delle diverse versioni rispetto alla base sequenziale. Si ottiene:

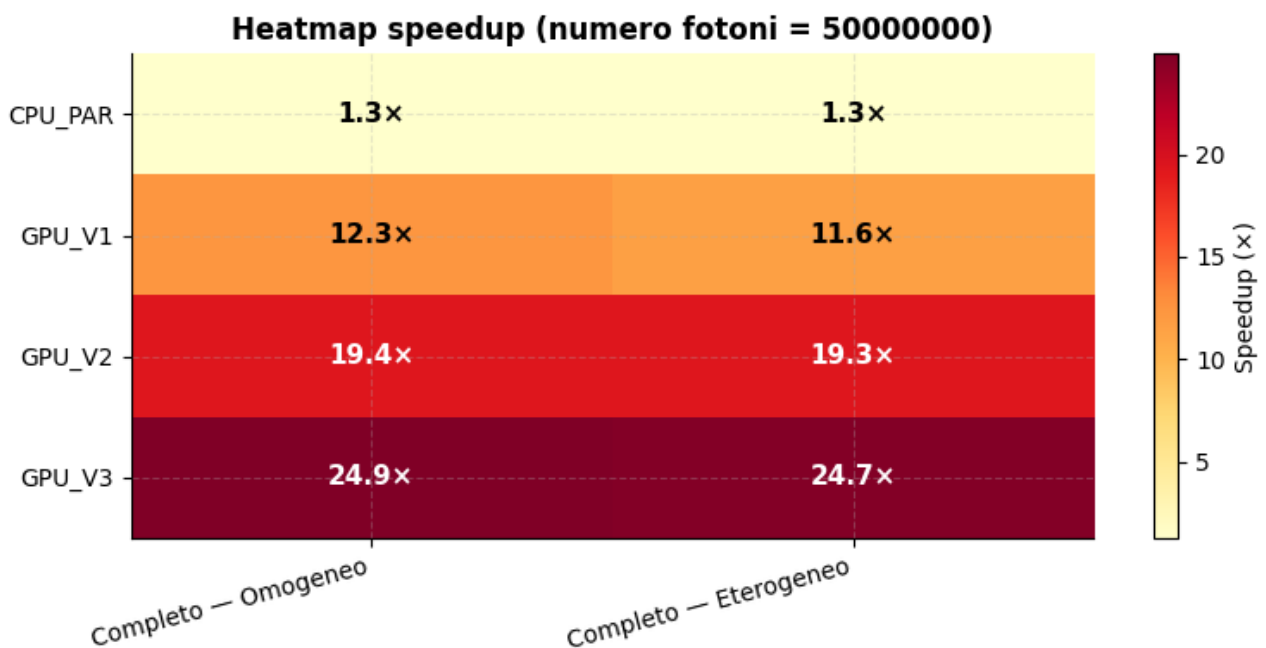


Figura 5 - Heatmap speedup

7.6. Ripartizione del tempo GPU

In ambienti HPC con acceleratori separati dall'host la latenza di trasferimento sul bus può rappresentare dei colli di bottiglia. Con la ripartizione del tempo si divide il tempo totale di esecuzione in:

- Tempo di trasferimento del phantom dalla RAM alla VRAM;

- Tempo di simulazione del kernel Monte Carlo;
- Tempo di trasferimento della griglia di dose calcolata verso la CPU.

Il grafico ottenuto è:

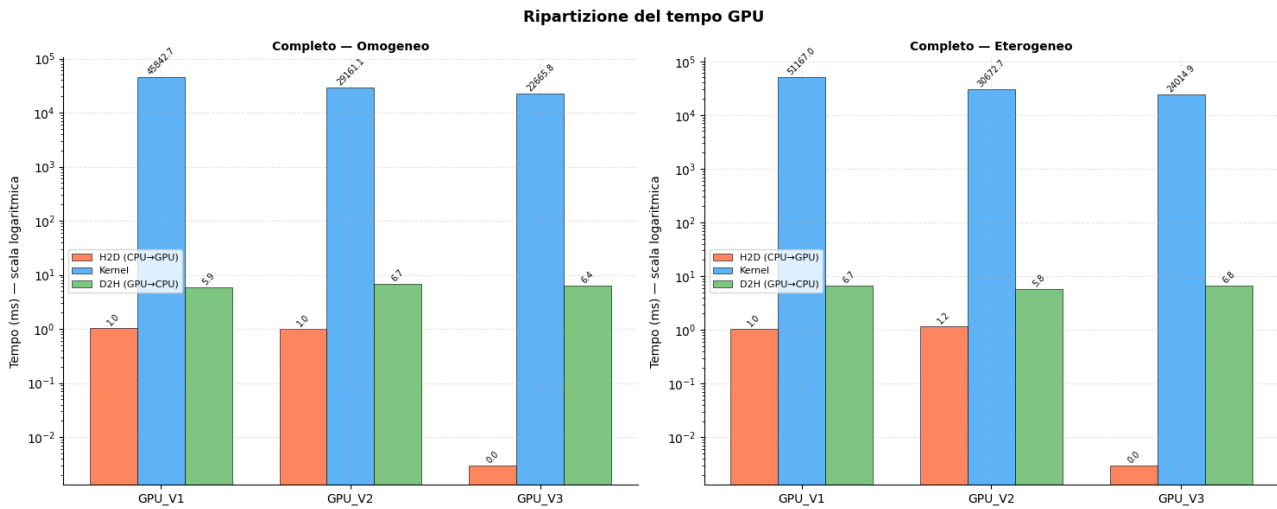


Figura 6 - Ripartizione del tempo GPU

I tempi di copia della dose e di trasferimento dell'array finale sono irrilevanti rispetto al tempo di calcolo del kernel per cui non si hanno colli di bottiglia dovuti a trasferimenti e il peso computazionale risiede interamente nel kernel.

7.7. Riepilogo dei risultati

Dall'analisi con carico massimo si evince che la CPU sequenziale genera circa 2.5 *GFLOPS* mentre la versione 3 raggiunge circa 64.2 *GFLOPS*. La stessa differenza si ha anche sul livello di banda che va dai circa 3.9 *GB/s* della versione 1 a circa 8.0 *GB/s* della versione 3. Dal calcolo della legge di Amdahl empirica viene stimata la frazione teorica di codice effettivamente parallelizzata per calcolare il limite di speedup massimo. Il valore di circa 0.98 della versione 3 mostra proprio il grado di parallelizzazione della simulazione.

Tramite le risoluzioni dei problemi della versione sequenziale attraverso la warp divergence, il bilanciamento del carico e il caching tramite texture memory è stata ottenuta una simulazione che riesce ad elaborare oltre 2 milioni di fotoni per secondo senza perdere l'accuratezza fisica del problema.

8. Visualizzazione dei risultati

Per la validazione del simulatore Monte Carlo per radioterapia è stato costruito sulla base dei risultati lo spazio della dose e sono state estratte le metriche cliniche standard. Per fare questo vengono prima convertiti i dati grezzi in grandezze interpretabili, vengono estratti i piani ortogonali e vengono calcolati i profili di dose. Tutti i dati vengono normalizzati in percentuale rispetto alla dose massima per consentire il tracciamento delle linee isodose¹⁷.

¹⁷ Linee che collegano i punti nello spazio all'interno di un corpo con la stessa dose di radiazione.

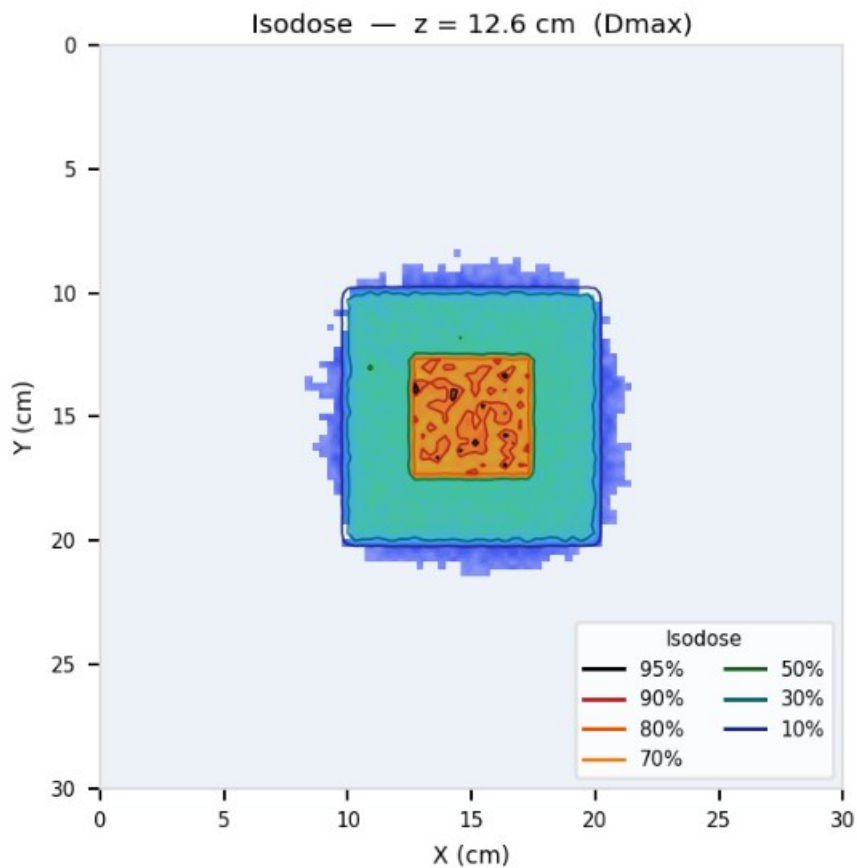


Figura 7 - Curve isodose

Con la simulazione a regime eseguita su un phantom eterogeneo contenente un inserto osseo centrato di spessore pari a ± 2.5 cm sono stati calcolati i tre parametri clinici principali:

- Profondità di dose massima $D_{max} = 12.75$ cm;
- Raggio terapeutico $R_{50} = 17.45$ cm;
- Larghezza di penombra laterale = 2.7 cm.

Tramite la curva del PDD viene mostrato l'andamento della dose lungo l'asse centrale di penetrazione del fascio. Si ottiene:

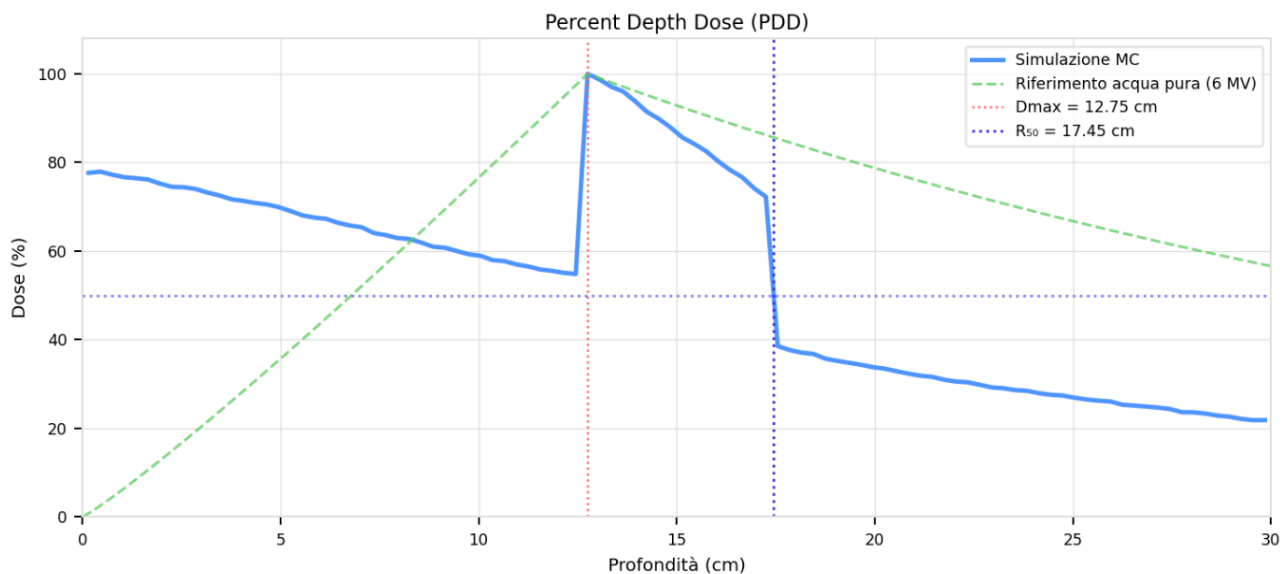


Figura 8 - Curva PDD

Da questa curva si evince come nei primi centimetri di penetrazione la dose non ha un massimo superficiale ma cresce progressivamente fino a raggiungere il picco di massima profondità. Questo dimostra come i fotoni primari interagiscono con i voxel superficiali producendo elettroni secondari carichi tramite effetto Compton e fotoelettrico. Superato il punto di massima profondità di dose si ha un decadimento progressivo controllato dal coefficiente di attenuazione lineare del mezzo. Questo dimostra come la probabilità di interazione calcolata ad ogni passo riduce lo spettro energetico dei fotoni in funzione del materiale.

Il profilo laterale della dose, invece, mostra una distribuzione con una netta degradazione ai bordi del campo geometrico:

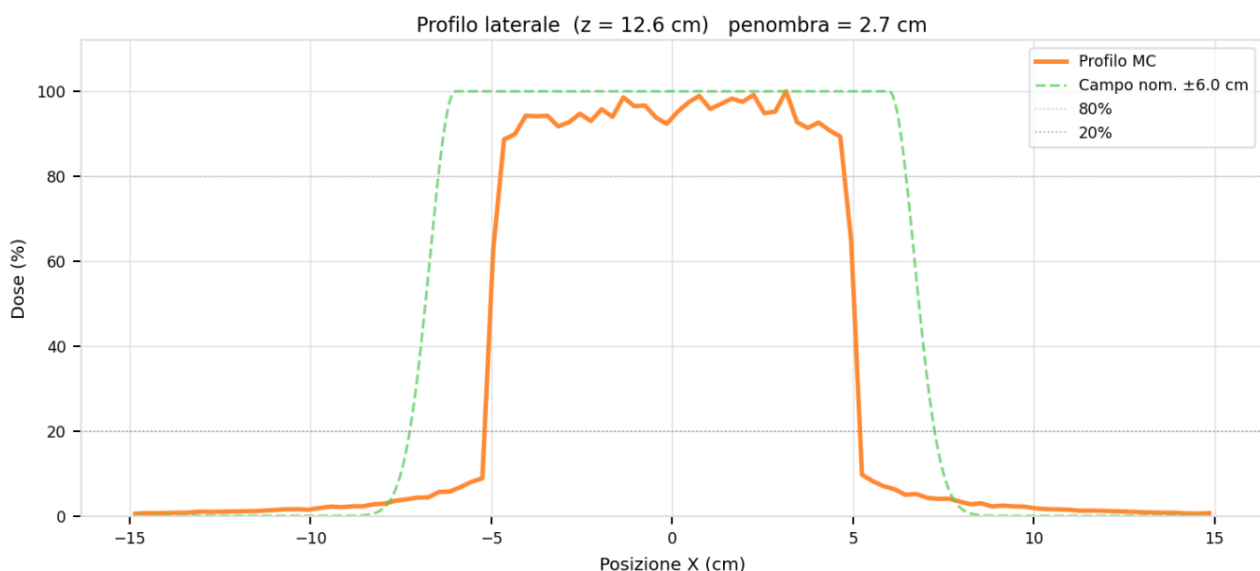


Figura 9 - Profilo laterale dose

Il fatto che il profilo non presenti un gradino netto a ridosso del campo dimostra il corretto campionamento angolare della sezione d'urto di Klein Nishina per l'effetto Compton. I fotoni deviati depositano, infatti energia al di fuori del cono del fascio primario generando la penombra.

Dai piani visivi per le sezioni assiale¹⁸, coronale¹⁹ e sagittale²⁰ viene mostrato l’impatto del deposito della dose sull’inserto osseo.

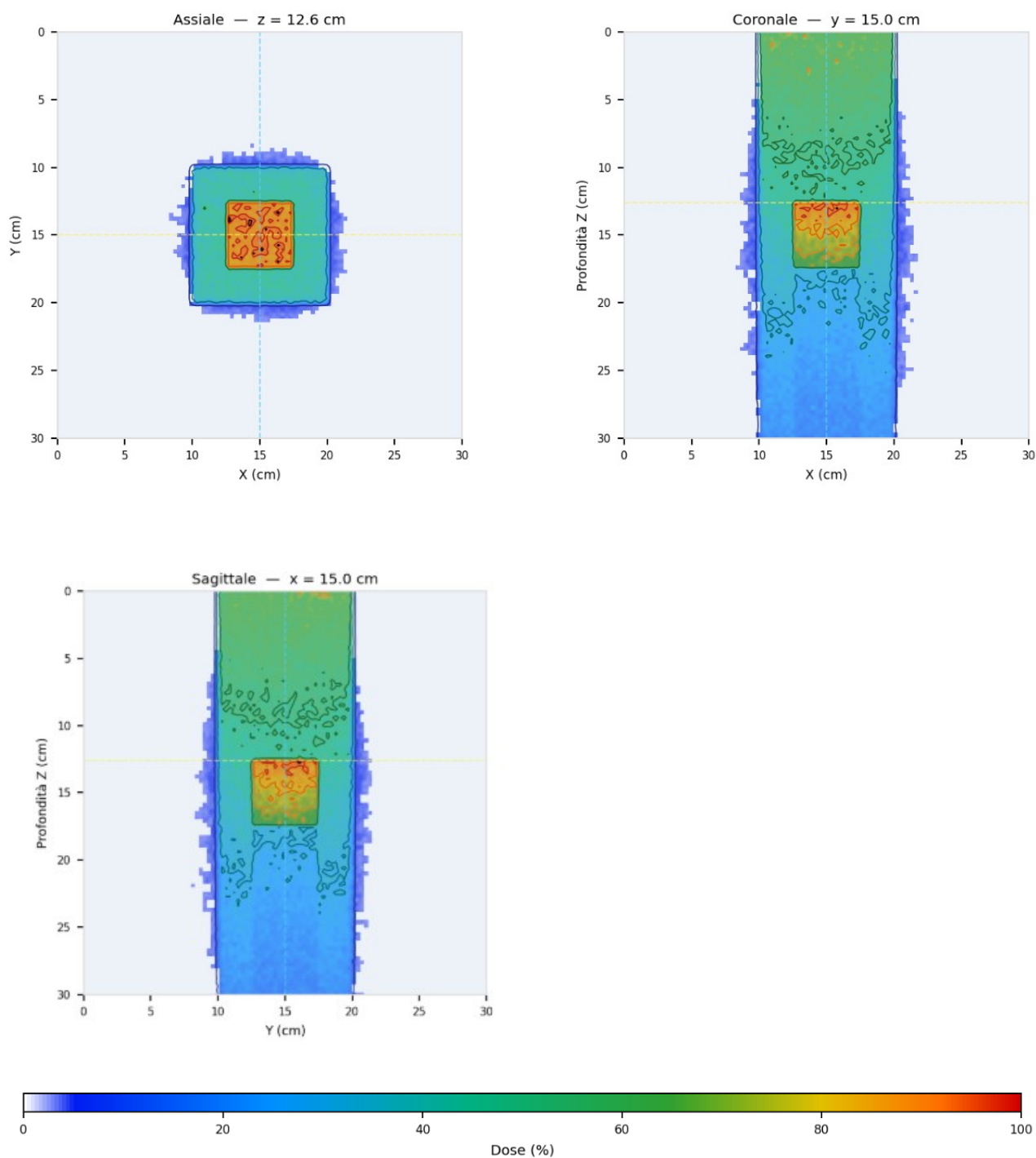


Figura 10 - Effetto eterogeneità phantom

Poiché l’osso presenta una densità fisica e un numero atomico più elevato rispetto all’acqua il coefficiente di attenuazione locale subisce un incremento. La dose decade più rapidamente all’interno dell’osso mentre si ha una riduzione di dose nei voxel di acqua situati a valle dell’eterogeneità.

¹⁸ Piano assiale divide orizzontalmente la struttura in parte superiore e parte inferiore.

¹⁹ Piano coronale divide verticalmente da sinistra a destra la struttura.

²⁰ Piano sagittale divide verticalmente la struttura tra parte anteriore e posteriore.

9. Conclusioni

Il presente lavoro rappresenta l'implementazione e la validazione di un simulatore Monte Carlo per il calcolo della dose in radioterapia su architetture parallele GPU tramite implementazione CUDA. L'intero sviluppo è stato fatto in maniera incrementale cercando di risolvere le problematiche in termini di prestazioni delle versioni già sviluppate. Si è partiti da una versione CPU sequenziale che ha rappresentato il punto di validazione dell'accuratezza fisica e della conversione alle versioni GPU.

La prima versione GPU base ha mostrato delle criticità dovute al trasporto delle particelle quali l'elevata latenza dovuta agli accessi non coalescenti sulla griglia voxelizzata e la perdita di efficienza per la warp divergence. Per risolvere queste criticità nella versione 2 GPU è stato effettuato un bilanciamento dinamico del carico basato su work stealing ottimizzando la distribuzione delle storie attive tra i thread del warp. Infine, nella versione 3 è stata ridefinita la gestione della memoria e dei tipi di dato attraverso la conversione a precisione singola e la mappatura del phantom nella texture memory della GPU massimizzando il throughput computazionale.

Dal profiling GPU condotto tramite i tool NVIDIA Nsight e NVPROF sono state estratte le metriche fondamentali, il throughput e la ripartizione del tempo di esecuzione dei kernel CUDA. Dal profiling emerge l'efficienza delle ottimizzazioni introdotte, soprattutto nella gestione delle operazioni atomiche di accumulo della dose.

Il simulatore è stato sottoposto a 10 test di validazione fisica e clinica che confermano l'estrazione stocastica accurata dei coefficienti di attenuazione del database NIST XCOM, la corretta gestione delle eterogeneità e la convergenza statistica dell'errore. Il confronto con le curve di dose in profondità e dei profili laterali rispetto allo standard di riferimento DOSXYZnrc validato quantitativamente tramite la metrica del Gamma Index, ne fornisce una corrispondenza e ne garantisce il rispetto dei requisiti di accuratezza imposti dal contesto clinico.

9.1. Sviluppi futuri

L'attuale implementazione adotta il modello del KERMA locale assumendo che l'energia trasferita dal fotone agli elettroni secondari venga depositata interamente nel punto di interazione. Come sviluppo futuro si potrebbe introdurre il trasporto esplicito delle particelle cariche al di sotto della soglia energetica di cutoff per modellare il non equilibrio elettronico.

Si potrebbe introdurre, per supportare simulazioni con più fotoni una programmazione distribuita MPI+CUDA così da suddividere lo spettro delle particelle primarie su più nodi di calcolo.

Indice delle figure

Figura 1 - Tempo di esecuzione	75
Figura 2 - Speedup assoluto	76
Figura 3 - Speedup relativo	77
Figura 4 – Throughput	78
Figura 5 - Heatmap speedup.....	78
Figura 6 - Ripartizione del tempo GPU	79
Figura 7 - Curve isodose	80
Figura 8 - Curva PDD	81
Figura 9 - Profilo laterale dose.....	81
Figura 10 - Effetto eterogeneità phantom.....	82

Indice delle tabelle

Tabella 1 - Tabella dei parametri.....	30
Tabella 2 - Moduli versione CPU sequenziale.....	34
Tabella 3 - Caratteristiche GPU	50
Tabella 4 - Risultati profilazione versione base GPU	56
Tabella 5 - Risultati analisi NVPROF	64
Tabella 6 - Analisi attività GPU	65
Tabella 7 - Risultati metriche Nsight.....	65

Bibliografia

- [1] I. C. o. R. U. a. Measurements, *ICRU Report 83, 2010*.
- [2] B. A. N. B. Näfstadius P, *Computer assisted dosimetry of scanned electron and photon beams for radiation therapy*.
- [3] B. R. C. J. S. P. Van Dyk J, *Commissioning and quality assurance of treatment planning computers*.
- [4] F. H. (. Attix, *Introduction to Radiological Physics and Radiation Dosimetry*.
- [5] O. N. Y. (. Klein, *On the Scattering of Radiation by Free Electrons according to Dirac's New Relativistic Quantum Dynamics*.
- [6] D. E. e. a. (. Cullen, *EPDL97: The Evaluated Photon Data Library. Lawrence Livermore National Laboratory*.
- [7] I. K. L. (. Lux, *Monte Carlo Particle Transport Methods: Neutron and Photon Calculations. CRC Press*.
- [8] B. R. K. I. R. D. W. O. (. Walters, *DOSXYZnrc Users Manual. NIST PIRS-794*.

- [9] I. R. 4. (1989), *Tissue Substitutes in Radiation Dosimetry and Measurement*, ICRU.
- [10] D. a. R. D. Sheikh-Bagheri, *Monte Carlo calculation of nine megavoltage photon beam spectra using the BEAM code*, <https://doi.org/10.1118/1.1445413>, 2002.
- [11] J. & W. A. Amanatides, *A Fast Voxel Traversal Algorithm for Ray Tracing*, 1987.